Built - in 12 Bit ADC / Touch Key / LCD、LED Driver / 1T 8051 Flash MCU

MJ603 增强型系列 MCU
中文用户手册

REV 1.2

深圳晓芯科技有限公司

邮箱：yequnsong@51bots.com

http://www.51bots.com/

地址：深圳市宝安区航城大道 U8 智造产业园 U6 栋 2F

目录

1 概述 **6**
2 基本特性 **6**
3 芯片型号功能介绍 **9**
4 系统框图 **10**
5 引脚封装及其描述 **11**
5.1 封装定义 11
5.2 引脚描述 14
6 中央处理器（**CPU**） **17**
6.1 CPU 功能简介 17
6.2 寄存器描述 17
7 存储器系统 **21**
7.1 随机数据存储器（RAM） 21
7.2 特殊功能寄存器（SFR） 21
7.3 Flash 存储器 22
7.3.1 功能简介 22
7.3.2 Flash 存储器组织结构 23
7.3.3 Flash 寄存器描述 24
7.3.4 Flash 控制例程 28
7.4 外部 RAM 映射为程序空间 31
8 中断系统 **33**
8.1 功能简介 33
8.2 中断逻辑 33
8.3 中断向量表 34
8.4 中断控制寄存器 34
8.5 外部中断 38
8.5.1 外部中断介绍 38
8.5.2 外部中断寄存器 38
8.5.3 外部中断控制方法及例程 41
9 时钟系统 **43**
9.1 时钟系统介绍 43
9.1.1 时钟专用名称定义 43
9.1.2 内置 2 - 4MHz RC 振荡器（IRCH） 44
9.1.3 外部 32.768KHz 晶体谐振器（XOSCL） 44
9.1.4 内置 131KHz RC 振荡器（IRCL） 44
9.1.5 内置 4MHz RC 振荡器（TFRC） 44
9.1.6 锁相环 PLL 44
9.1.7 外部高速 RC 振荡器（ERC） 44
9.2 时钟控制寄存器描述 45
9.3 系统时钟 48
9.3.1 系统时钟结构图 48
9.3.2 系统时钟控制寄存器描述 48
9.3.3 系统时钟控制方法及例程 50

9.4 内部 RC 振荡器校正 52
9.4.1 校正模块介绍 52
9.4.2 校正模块控制寄存器 53
9.4.3 校正模块控制例程 55
9.5 外部时钟监控 58
9.5.1 功能描述 58
9.5.2 外部时钟监控控制寄存器 58
10 供电和复位系统 **60**
10.1 供电系统 60
10.1.1 O 功能简介 61
10.1.2 O 控制寄存器 61
10.2 复位系统 63
11 功耗管理 **65**
11.1 LE 模式 65
11.2 STOP 模式 65
11.3 低速运行模式 66
11.4 低功耗相关寄存器描述 66
11.5 低功耗模式控制例程 69
12 通用定时器（定时器 **0,**定时器 **1,**定时器 **2**） **71**
12.1 定时器 0 71
12.1.1 定时器 0 介绍 71
12.1.2 定时器 0 寄存器描述 72
12.2 定时器 1 74
12.2.1 定时器 1 介绍 74
12.2.2 定时器 1 寄存器描述 75
12.3 定时器 2 76
12.3.1 功能简介 76
2.3.2 定时器 2 寄存器描述 77
13 *看门狗定时器（**WDT**）* ***80***
13.1 看门狗定时器(WDT)功能简介 80
13.2 看门狗定时器(WDT)寄存器描述 80
13.3 看门狗定时器控制例程 82
14 *实时定时器（**RTC**）* ***83***
14.1 RTC 功能简介 83
14.2 RTC 寄存器描述 84
14.3 RTC 控制例程 88
15 *通用输入输出口（**GPIO**）及复用定义* ***91***
15.1 功能简介 91
15.2 引脚寄存器描述 93
15.3 GPIO 引脚控制例程 101
16 *通用异步收发器（**UART**）* ***103***
16.1 UART0 103
16.1.1 功能简介 103
16.1.2 寄存器描述 107
16.2 UART1 108
16.2.1 介绍 108
16.2.2 UART1 寄存器描述 109

17 I²C 接口 *111*
17.1 功能简介 111
17.2 I2C 主要特点 111
17.3 I2C 功能描述 111
17.4 I2C 通信引脚的映射 113
17.5 寄存器描述 114
17.6 I2C 控制例程 118
18 LCD/LED 驱动 **124**
18.1 LCD 驱动 124
18.1.1 功能简介 124
18.1.2 D 偏压 125
18.1.3 D 功能描述 126
18.2 LED 驱动 127
18.2.1 功能简介 127
18.2.2 D 功能描述 128
18.3 LCD/LED 寄存器描述 128
18.4 LCD 驱动控制例程 131
18.5 LED 驱动控制例程 132
19 PWM **134**
19.1 PWM 功能简介 134
19.2 PWM 功能描述 134
19.3 PWM 寄存器描述 139
19.4 PWM 功能控制例程 144
20 模/数字转换器（ADC） *148*
20.1 功能简介 148
20.2 主要特性 148
20.3 结构框图 148
20.4 功能描述 149
20.5 寄存器描述 151
20.6 ADC 控制例程 153
21 触摸按键（TOUCH KEY） **154**
21.1 功能简介 154
21.2 主要特性 154
21.3 结构图 155
21.4 功能描述 155
21.4.1 手动模式和自动模式 155
21.4.2 触摸时钟预分频 155
21.4.3 低功耗模式 156
21.4.4 触摸按键共用 LED 驱动 156
21.4.5 触摸内部基准和内部运放 157
21.4.6 触摸防水补偿机制 157
21.5 寄存器描述 157
21.6 触摸控制例程 165

22 低电压检测（**LVD**） **166**
22.1 功能简介 166
22.2 功能描述 166
22.3 寄存器描述 166
22.4 LVD 控制例程 167
23 程序下载和仿真 **169**
23.1 程序下载 169
23.2 在线仿真 169
24 电气特性 **170**
24.1 极限参数 170
24.2 直流电气特性 170
24.3 交流电气特性 172
24.4 ADC 电气特性 173
25 封装类型 **174**
26 典型应用参考电路 **181**
27 附录 **184**
附录 1 指令集速查表 184

1 概述

MJ603 增强型系列芯片是基于 1T 8051 内核的 8 位微控制器，通常情况下，运行速度比传统的 8051 芯片快 10 倍，性能更加优越。内置 8/16/32K Flash 程序存储器，可多次重复编程的特性给用户开发带来了极大的方便。不仅保留了传统 8051 芯片的基本特性，还集成了 ADC、LCD/LED 驱动、Touch Key、PWM、 UART、RTC、低电压检测(LVD)等功能模块。支持 IDLE、STOP 和低速运行三种省电模式以适应不同功耗要求的应用。强大的功能及优越的抗干扰性能使其可广泛应用于各种车载音响、家用音响、小家电、蓝牙音箱、汽车电子、数码电机、运动器材、马达控制、医疗保健、仪器仪表、安防、电源控制、工业控制及门铃产品中。

2 基本特性

- 内核
  - CPU：1T 8051, 最高速度比传统 8051 快 10 倍
  - 兼容 8051 指令集, 双 DPTR 工作模式
  - CPU 频率：最高可支持 27MHz
- 存储器
  - ROM: Flash : 8 / 16 / 32K 字节，支持多次重复擦写
  - Flash 可划分为程序空间和数据空间，数据空间可用于存储掉电需要保存数据，可省略 EEPROM
  - RAM:256 字节内部 RAM，2K 字节外部 RAM
- 工作电压
  - 工作电压：1.8 - 5.5V
- 中断系统
  - 15 个有效中断源
  - 两级中断优先级，支持中断嵌套
  - 10 个外部中断源，每个外部中断都可配置任意信号引脚作为中断输入脚
- 时钟系统
  - 外部 RTC 振荡器：32.768KHz（仅部分型号支持）
  - 内置低速 RC 振荡器：131KHz
  - 内置 PLL：倍频倍数为 2 - 10 倍，参考时钟为 2 - 4MHz 内置 RC 振荡器
  - 内置高速 RC 振荡器：2 - 4MHz，精度可达 1% (出厂初始频率为 3.6864MHz@3.3V/25℃)
  - 内嵌外部时钟监控模块，可有效监控外部各个时钟工作状态，避免因外部时钟停振而造成死机
- RTC 功能
  - 内置 RTC 模块可计时、分、秒、星期、天数，支持闹钟功能
  - 支持毫秒、半秒中断
- 通用输入输出口（GPIO）
  - 最多支持 26 个 GPIO 口（不同型号会有不同）
  - 支持推挽、开漏、强上拉、弱上拉、强下拉、弱下拉、高阻模式
  - 推挽模式下可设置不同驱动强度和翻转速度
- 定时器
  - 3 个 16 位通用定时器: 定时器 0，定时器 1，定时器 2

- 看门狗
  - 27 位看门狗定时器，16 位调节精度，可配置看门狗复位或中断
- 同步/异步通讯接口（UART）
  - 最多支持 2 个 UART 接口（不同型号会有不同）
  - 支持 1 字节接收缓存
- I^2C 接口
  - 内置 1 路 I^2C 接口，支持主从模式，支持标准/快速/高速模式。
  - I2C 可设置数字滤波，增强 I2C 抗干扰性能。
- LCD 驱动
  - 最大可支持 5com x 8seg、4com x 9seg、3com x 10seg（不同型号会有不同）
  - 可配置占空比：1/2、1/3、1/4、1/5 Duty
  - 可配置偏压：1/2、1/3、1/4 Bias
  - 支持 8 级对比度调整
  - 支持 3 种等级驱动电流，用户可根据不同的 LCD 屏进行调整
- LED 驱动
  - 最大可支持 5com x 8seg、4com x 9seg、3com x 10seg（不同型号会有不同）
  - 支持 8 级亮度调节
  - LED COM 引脚可设置强灌电流模式，实现 LED 高亮度显示效果。
- 模/数转换器（**ADC**）
  - 支持 8 通道 12 位 SAR ADC（不同型号会有不同）
  - 支持 3 种基准电压源：VDD、内部基准、外部基准
  - 选择内部电压为基准电压时可测量 VDD 电压
  - 内置运放，支持检测信号缩小功能，缩小倍数可选
- **PWM**
  - 支持 6 通道 PWM，在 16 位范围内可任意配置周期和占空比（不同型号会有不同）
  - 支持互补模式和死区控制，可用于驱动直流无刷电机
  - 支持可设置边沿对齐和中心对齐模式
  - 支持可直接输出内部时钟功能
  - 支持 PWM 中断
- 触摸按键（**Touch Key**）
  - 内置触摸感应控制器
  - 最大支持 20 通道触摸（不同型号会有不同）
  - 触摸可设置内部充电和内部基准，可有效抑制电源低频干扰
  - 支持触摸引脚与 LED 驱动引脚复用
  - 内置防水补偿机制
  - 高抗干扰性，符合 EMC(CS)标准
  - 支持触摸省电模式，最低功耗小于 10uA
- 低电压检测（LVD）
  - 可配置电压检测范围 1.8 - 4.8V
  - 可设置低电压复位或中断
- 复位模式
  - 芯片支持多种复位源：硬复位，软复位，看门狗复位，低电压检测复位，上电掉电复位
- 低功耗
  - STOP 模式，电流<7uA

- IDLE 模式，电流<12uA
- 低速运行模式，电流<20uA

◆ 程序下载和仿真

- 支持 ISP 和 IAP
- 支持在线仿真功能

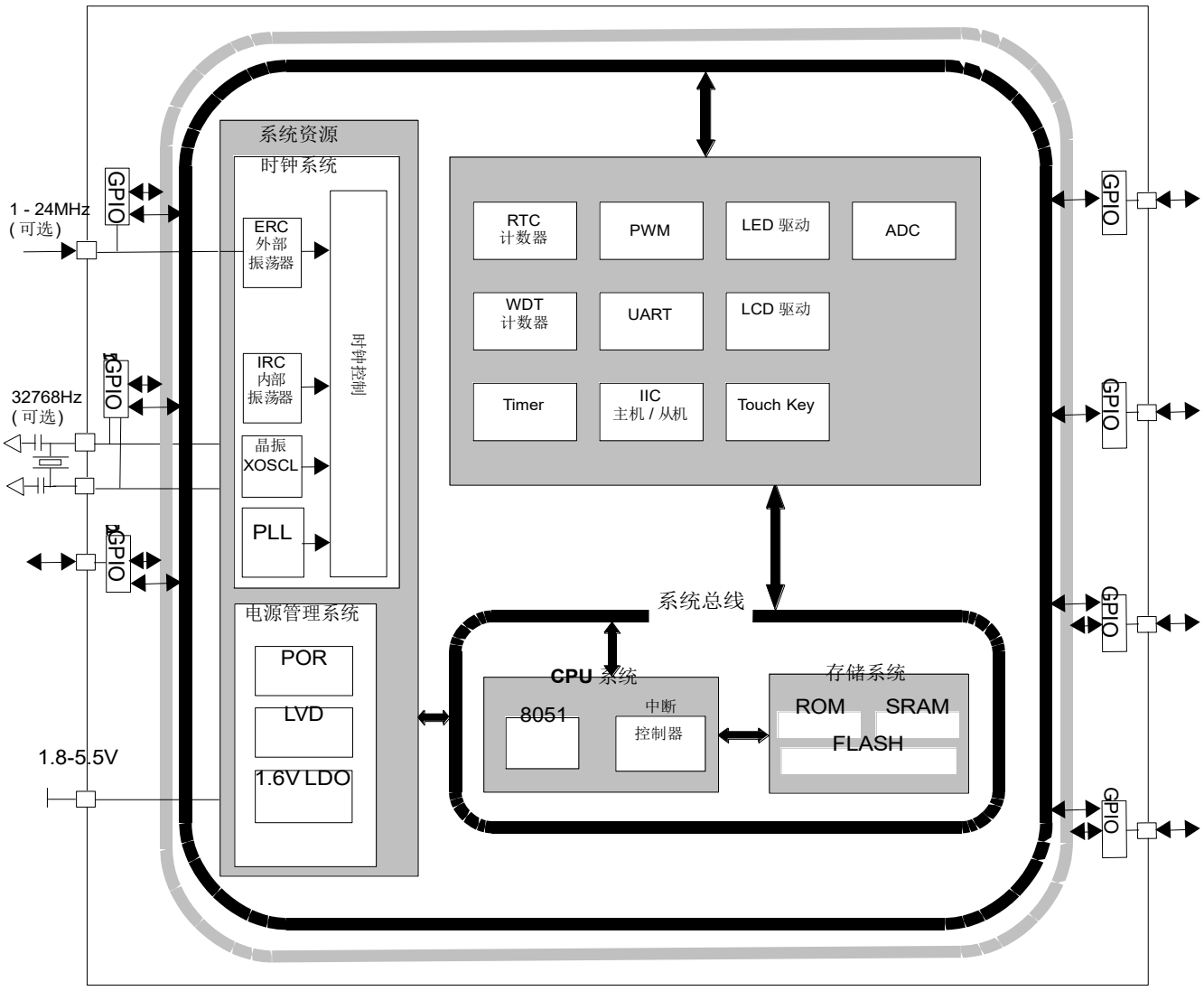
◆ 封装类型：SOP8/SOP16/SOP20/SSOP20/QFN20/SOP28/SSOP28

3 芯片型号功能介绍

表 **3-1** **MJ603** 系列具体型号功能特点

芯片型号	Flash 量容 [BYTE]	Ram[BYTE] 部外	振晶速低部外[32.768KHz]	GPIO 量数	UART 量数	I²C	16 bit PWM 量数道通	12 bit ADC 量数道通	触摸按键数	LCD 动驱[comx seg]	LED 驱动[comx seg]	真仿上片	封装形式
MJ6031S1	8K	2K	--	6	1	√	2	3	5	--	--	√	SOP8
MJ6031S3	8K	2K	--	14	2	√	6	5	11	---	---	√	SOP16
MJ6031S4	8K	2K	--	18	2	√	6	7	15	3X5 4X4	3X5 4X4	√	SOP20
MJ6031P4	8K	2K	--	18	2	√	6	7	15	3X5 4X4	3X5 4X4	√	SSOP20
MJ6031S6	8K	2K	√	26	2	√	6	8	20	3X10 4X9 5X8	3X10 4X9 5X8	√	SOP28
MJ6031A6	8K	2K	√	26	2	√	6	8	20	3X10 4X9 5X8	3X10 4X9 5X8	√	SSOP28
MJ6032N2	16K	2K	--	18	2	√	5	7	15	--	--	√	QFN20
MJ6032S4	16K	2K	--	18	2	√	6	7	15	3X5 4X4	3X5 4X4	√	SOP20
MJ6032P4	16K	2K	--	18	2	√	6	7	15	3X5 4X4	3X5 4X4	√	SSOP20
MJ6030S6	32K	2K	√	26	2	√	6	8	20	3X10 4X9 5X8	3X10 4X9 5X8	√	SOP28
MJ6030A6	32K	2K	√	26	2	√	6	8	20	3X10 4X9 5X8	3X10 4X9 5X8	√	SSOP28

4 系统框图


系统资源
时钟系统
ERC
外部
振荡器
IRC
内部
振荡器
晶振
XOSCL
PLL
时钟控制
电源管理系统
POR
LVD
1.6V LDO
1 - 24MHz
(可选)
32768Hz
(可选)
1.8-5.5V
GPIO
RTC
计数器
PWM
LED 驱动
ADC
WDT
计数器
UART
LCD 驱动
Timer
IIC
主机 / 从机
Touch Key
系统总线
CPU 系统
8051
中断
控制器
存储系统
ROM
SRAM
FLASH

5 引脚封装及其描述

5.1 封装定义

图 **5-1-1 SOP8** 封装图

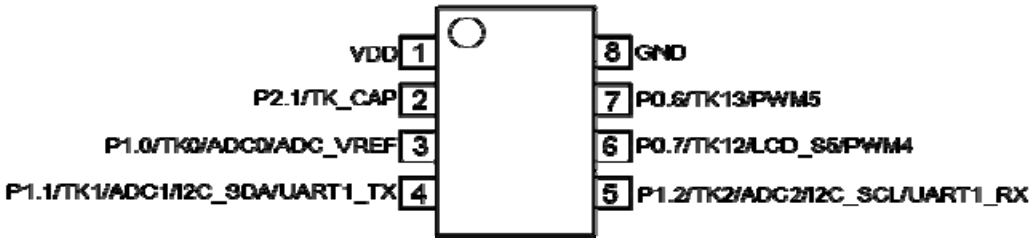


图 **5-1-2 SOP16** 封装图

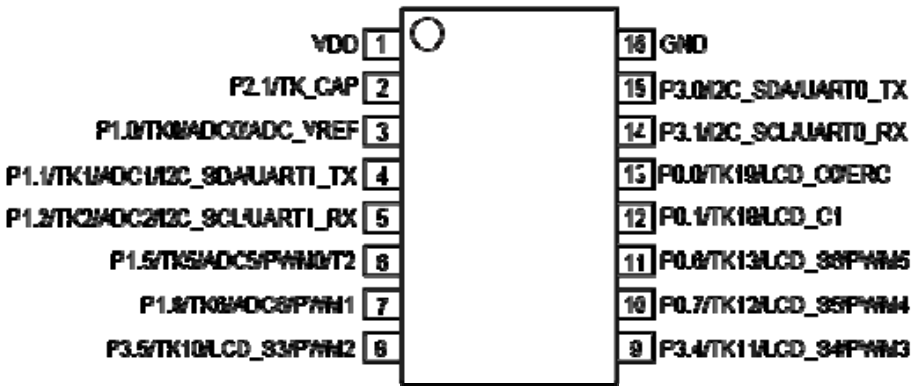


图 **5-1-3　SOP20 / SSOP20** 管脚定义图

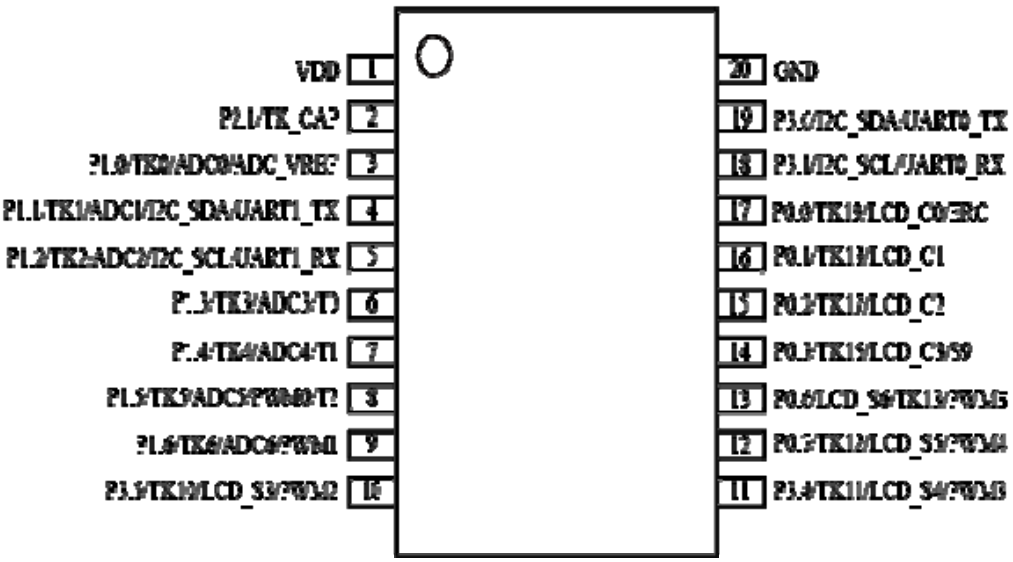


图 **5-1-4　QFN20** 管脚定义图

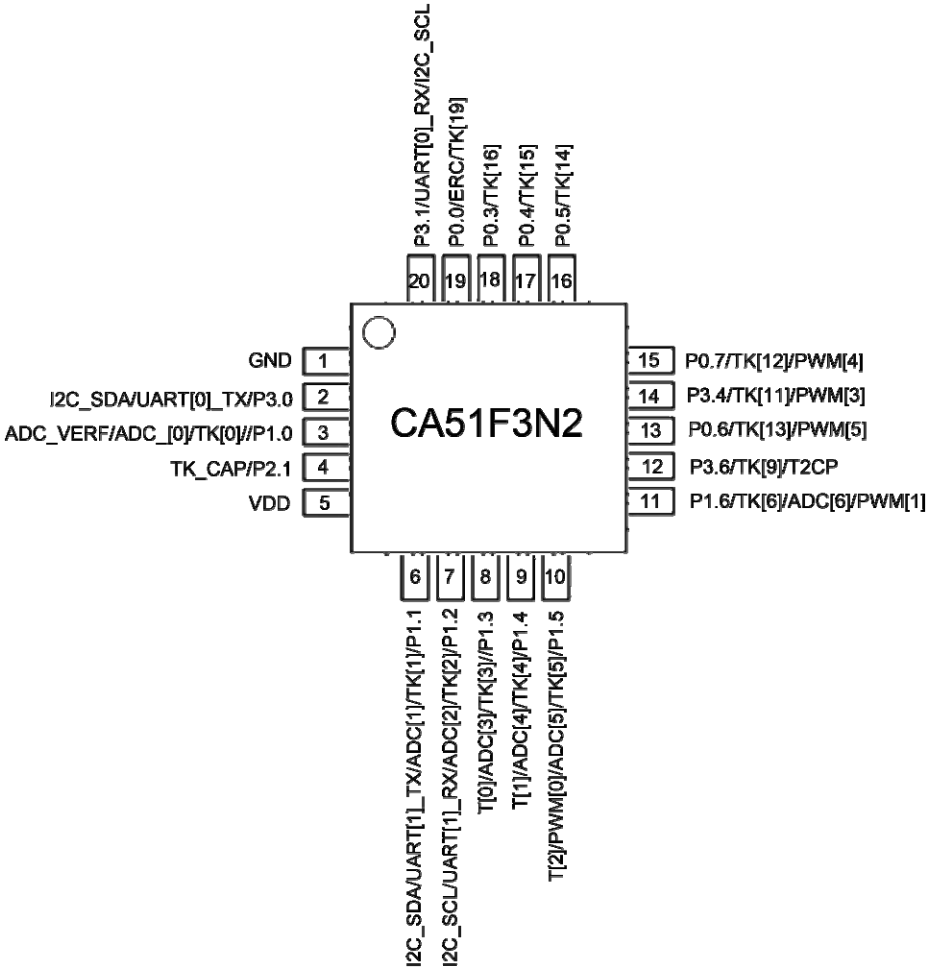
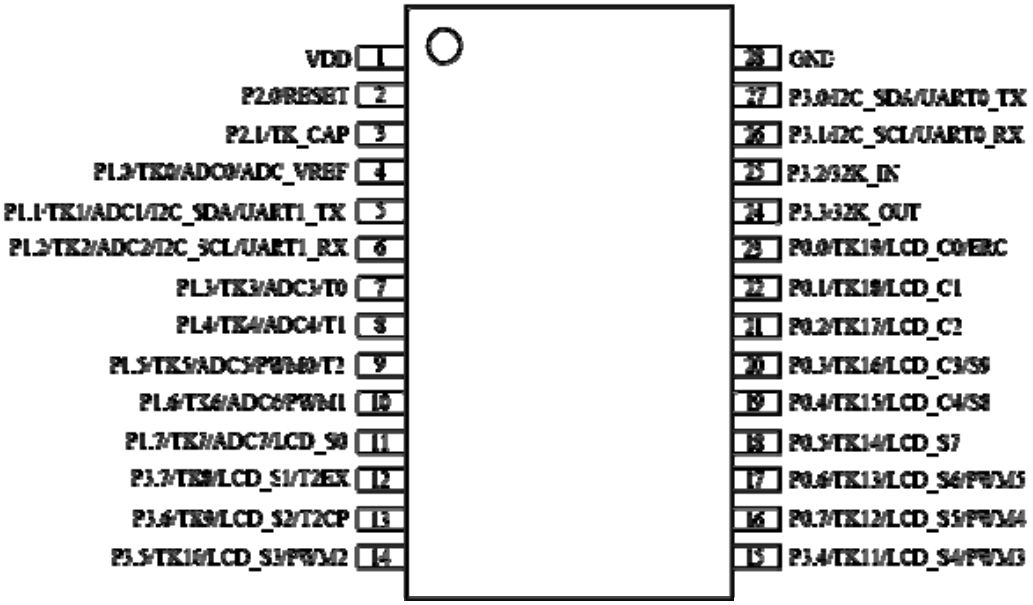


图 **5-1-5　SOP28/ SSOP28** 管脚定义图



5.2 引脚描述

表 **5-2-1** 引脚描述

引脚序号						管脚名称	管脚功能	默认功能
SOP28	SSOP28	SOP20	SSOP20	SOP16	QFN20	SOP8			
1		1		1	5	1	VDD	芯片供电管脚	芯片供电管脚
2		-		-	-	-	P2.0/RESET	通用双向 I/O 口 硬件复位脚	硬件复位脚
3		2		2	4	2	P2.1/TK_CAP	通用双向 I/O 口 触摸参考电容模拟输入	通用双向 I/O 口
4		3		3	3	3	P1.0/TK[0]&LED[0]/ADC_CH0/ADC_VREF	通用双向 I/O 口 触摸通道输入复用 LED 输出 ADC 模拟通道输入 ADC 外部参考电压输入	通用双向 I/O 口
5		4		4	6	4	P1.1/UART1_TX/TK[1]&LED[1]/ADC_CH1/ I2C_SDA	通用双向 IO 口 串口 1 数据发送端口 I2C 数据传输端口 触摸通道输入复用 LED 输出 ADC 模拟通道输入 ADC 外部参考电压输入	I2C 数据传输端口
6		5		5	7	5	P1.2/UART1_RX/TK[2]&LED[2]/ADC_CH2 /I2C_SCL	通用双向 IO 口 串口 1 数据接收端口 I2C 时钟传输端口 触摸通道输入复用 LED 输出 ADC 模拟通道输入	I2C 时钟传输端口
7		6		-	8	-	P1.3/TK[3]&LED[3]/ADC_CH3/T0	通用双向 IO 口 T0 输入端口 触摸通道输入复用 LED 输出 ADC 模拟通道输入	通用双向 IO 口
8		7		-	9	-	P1.4/TK[4]&LED[4]/ADC_CH4/T1	通用双向 IO 口 T1 输入端口 触摸通道输入复用 LED 输出 ADC 模拟通道输入	通用双向 IO 口
9		8		6	10	-	P1.5/TK[5]&LED[5]/ADC_CH5/PWM0/T2	通用双向 IO 口 T2 输入端口 PWM 输出端口 触摸通道输入复用 LED 输出 ADC 模拟通道输入	通用双向 IO 口
10		9		7	11	-	P1.6/TK[6]&LED[6]/ADC_CH6/PWM1	通用双向 IO 口 PWM 输出端口 触摸通道输入复用 LED 输出	通用双向 IO 口

						ADC 模拟通道输入	
11	-	-	-	-	P1.7/SEG0/TK[7]&LED[7]/ADC_CH7	通用双向 IO 口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出 ADC 模拟通道输入	通用双向 IO 口
12	-	-	-	-	P3.7/SEG1/TK[8]&LED[8]/T2EX	通用双向 IO 口 T2EX 输入端口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
13	-	-	12	-	P3.6/SEG2/TK[9]&LED[9]/T2CP	通用双向 IO 口 T2CP 输入端口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
14	10	8	-	-	P3.5/SEG3/TK[10]&LED[10]/PWM2	通用双向 IO 口 PWM 输出端口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
15	11	9	14	-	P3.4/SEG4/TK[11]&LED[11]/PWM3	通用双向 IO 口 PWM 输出端口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
16	12	10	15	6	P0.7/SEG5/TK[12]&LED[12]/PWM4	通用双向 IO 口 PWM 输出端口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
17	13	11	13	7	P0.6/SEG6/TK[13]&LED[13]/PWM5	通用双向 IO 口 PWM 输出端口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
18	-	-	16	-	P0.5/SEG7/TK[14]&LED[14]	通用双向 IO 口 LED SEG 输出端口 LCD SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
19	-	-	17	-	P0.4/COM4/TK[15]&LED[15]/SEG8	通用双向 IO 口 LED COM/SEG 输出端口 LCD COM/SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口

20	14	-	18	-	P0.3/COM3/TK[16]&LED[16]/SEG9	通用双向 IO 口 LED COM/SEG 输出端口 LCD COM/SEG 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
21	15	-	-	-	P0.2/COM2/TK[17]&LED[17]	通用双向 IO 口 LED COM 输出端口 LCD COM 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
22	16	12	-	-	P0.1/COM1/TK[18]&LED[18]	通用双向 IO LED COM 输出端口 LCD COM 模拟输出 触摸通道输入复用 LED 输出	通用双向 IO 口
23	17	13	19	-	P0.0/COM0/TK[19]&LED[19]/ERC	通用双向 IO LED COM 输出端口 LCD COM 模拟输出 触摸通道输入复用 LED 输出 ERC 模拟端口	通用双向 IO 口
24	-	-	-	-	P3.3/ XTAL_OUT_32K	通用双向 IO 32K 外部晶振输出	32K 外部晶振输出
25	-	-	-	-	P3.2/ XTAL_IN_32K	通用双向 IO 32K 外部晶振输入	32K 外部晶振输入
26	18	14	20	-	P3.1/UART0_RX/I2C_SCL	通用双向 IO 串口 0 数据接收端口 I2C 时钟传输端口	I2C 时钟传输端口
27	19	15	2	-	P3.0/UART0_TX/I2C_SDA	通用双向 IO 串口 0 数据输出端口 I2C 数据传输端口	I2C 数据传输端口
28	20	16	1	8	VSS	电源地引脚	电源地引脚

备注：信号引脚复用功能设置方法详见表 *15-2-5* 和表 *15-2-6*。

6 中央处理器（CPU）

6.1 CPU 功能简介

MJ603 系列芯片采用单周期 8051 CPU，与原来的 MCS-51 指令集完全兼容。CPU 采用流水线结构，通常情况下，单周期 8051 CPU 的运行速度比标准 8051 处理器快 10 倍。

CPU 有以下特性：

- 1T 8051 CPU
- 兼容 8051 指令集，见指令集附录
- 双 DPTR，可用于数据快速搬移

6.2 寄存器描述

程序计数器 **PC**

程序计数器 PC 寄存器为 16 位，是专门用来控制指令执行顺序的寄存器，它没有寄存器地址。单片机上电或复位后，PC 值为 0，单片机从零地址开始执行程序。

累加器 **ACC**

累加器 ACC 是一个常用的专用寄存器，指令系统中采用 A 作为累加器的助记符，常用于存放算术或逻辑运算的操作数及运算结果。

通用寄存器 **B**

B 在乘除法运算中需要和 ACC 配合使用。MUL AB 指令把 ACC 和 B 中 8 位无符号数相乘， 所得的 16 位乘积的低字节存放在 A 中，高字节存放在 B 中。DIV AB 指令用 B 除以 A，整数商存放在 A 中，余数存放在 B 中。寄存器 B 还可以用作通用暂存寄存器。

堆栈指针 **SP**

堆栈指针 SP 是一个 8 位专用寄存器。它指示出堆栈顶部在内部 RAM 块中的位置。系统复位后，SP 初始化位 07H，使得堆栈事实上由 08H 单元开始，考虑 08H~1FH 单元分别 属于工作寄存器组 1~3，若在程序设计中用到这些区，则最好 SP 改变为 80H 或更大的为宜。

数据指针 **DPTR**

数据指针 DPTR0/DPTR1 是两个 16 位专用寄存器，它们的高位字节寄存器用 DP0H/DP1H 表示，低位字节寄存器用 DP0L/DP1L 表示，通过 DPS(PSW.1)可选择使用 DPTR0/DPTR1。每个 DPTR 既可以作为一个 16 位寄存器来处理，也可以作为 2 个独立的 8 位寄存器 DP0H/DP1H 和 DP0L/DP1L 来处理。

状态寄存器 **PSW**

状态寄存器 PSW 是 CPU 的状态寄存器。在 CPU 做算术运算或者逻辑运算时，对应的 PSW 状态位会发生改变。

表 **6-2-1** 累加器 **ACC**

E0H	7	6	5	4	3	2	1	0
ACC	ACC[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

表 **6-2-2** 通用寄存器 **B**

F0H	7	6	5	4	3	2	1	0
B	B[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

表 **6-2-3** 堆栈指针 **SP**

81H	7	6	5	4	3	2	1	0
SP	SP[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	1	1	1

表 **6-2-4** 数据指针 **DP0L**

82H	7	6	5	4	3	2	1	0
DP0L	DP0L[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

表 **6-2-5** 数据指针 **DP0H**

83H	7	6	5	4	3	2	1	0
DP0H	DP0H[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

表 **6-2-6** 数据指针 **DP1L**

84H	7	6	5	4	3	2	1	0
DP1L	DP1L[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

表 **6-2-7** 数据指针 **DP1H**

85H	7	6	5	4	3	2	1	0
DP1H	DP1H[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

表 **6-2-8** 状态寄存器 **PSW**

D0H	7	6	5	4	3	2	1	0
PSW	CY	AC	F0	RS[1:0]		OV	DPS	P
R/W	R/W	R/W	R/W	R/W		R/W	R	R
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	CY	进位标志位 0：算术或逻辑运算中，没有进位或借位发生 1：算术或逻辑运算中，有进位或借位发生
6	AC	辅助进位标志位 0：算术或逻辑运算中，没有辅助进位或借位发生 1：算术或逻辑运算中，有辅助进位或借位发生
5	F0	F0 标志位 用户自定义标志位
4~3	RS	R0~R7 寄存器页选择位 00：页 0（映射到 00H-07H) 01：页 1（映射到 08H-0FH) 10：页 2（映射到 10H-17H) 11：页 3（映射到 18H-1FH)
2	OV	溢出标志位 0：没有溢出发生 1：有溢出发生
1	DPS	DPTR 选择寄存器，0 为选择 DPTR0，1 为选择 DPTR1
0	P	奇偶校验位 0：累加器 A 值为 1 的位数为偶数 1：累加器 A 值为 1 的位数为奇数

表 **6-2-9** 寄存器 **SPMAX**

8100H	7	6	5	4	3	2	1	0
SPMAX	SPMAX[7:0]							
R/W	R	R	R	R	R	R	R	R
初始值	0	0	0	0	0	1	1	1
位编号	位符号	说明						
7~0	SPMAX	寄存器 SPMAX 用于记录 SP 的最大值，用户在应用程序中可查看此寄存器来判断堆栈有没有溢出风险						

7 存储器系统

7.1 随机数据存储器（RAM）

MJ603 系列芯片提供了 256 字节内部 RAM 和 2K 字节外部 RAM，存储器地址分配如下：

- 低位 128 字节的内部 RAM（地址：00H ~ 7FH）可直接寻址或间接寻址。
- 高位 128 字节的内部 RAM（地址：80H ~ FFH）只能间接寻址。
- 外部 2K 字节外部 RAM（地址：0000H ~ 07FFH）可通过 MOVX 指令间接寻址。

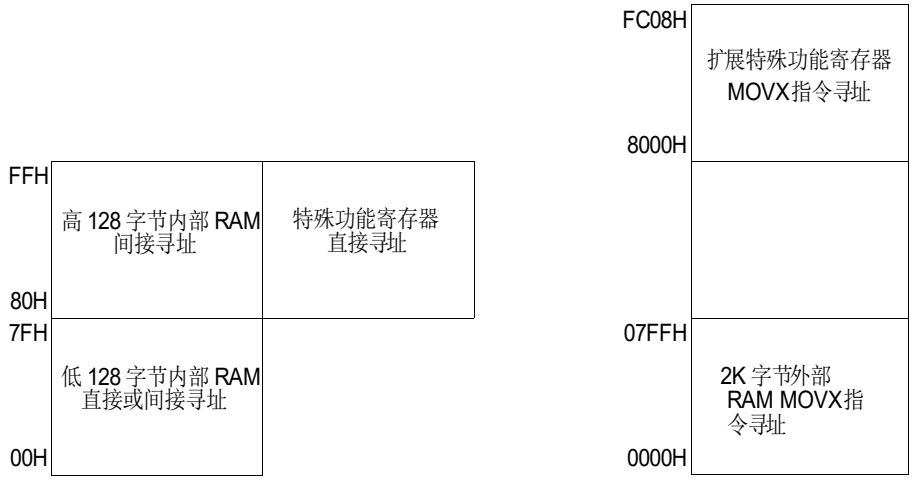


图 **7-1-1** **RAM** 组织结构图

7.2 特殊功能寄存器（SFR）

MJ603 系列芯片提供了兼容传统 8051 的 SFR 分布，SFR 和高 128 字节内部 RAM 共用地址 80H ~ FFH，只能直接寻址，SFR 映射如表 7-2-1 所示。

表 **7-2-1** 特殊功能寄存器（**SFR**）映射表

	可位寻址	不可位寻址						
	0/8	1/9	2/A	3/B	4/C	5/D	6/E	7/F
F8H	EXIP	EPIE	EPIF	EPCON	IDLSTL	IDLSTH	STPSTL	STPSTH
F0H	B	RTCON	RTCS	RTCM	RTCH	RTCDL	RTCDH	-
E8H	EXIE	RTCSS	RTAS	RTAM	RTAH	RTMSS	RTCIF	LVDCON
E0H	ACC	LXCON	LXCFG	LXDAT	LXDIVL	LXDIVH	ATKNL	ATKNH
D8H	PWMEN	PWMUPD	PWMCMX	PWMCON	PWMCFG	PWMDIVL	PWMDIVH	PWMDUTL
D0H	PSW	PWMDUTH	PWMAIF	PWMBIF	PWMCIF	TKMAXF	TKMINF	HVTH
C8H	T2CON	T2MOD	T2CL	T2CH	TL2	TH2	TKMSL	TKMSH
C0H	INDEX	TKCON	TKCFG	TKMTS	TKCHS	ATKCL	ATKCH	TKIF

B8H	IP	ADCON	ADCFGL	ADCFGH	ADCDL	ADCDH	CKMON	CKMIF
B0H	P3	I2CCON	I2CADR	I2CADM	I2CCCR	I2CDAT	I2CSTA	I2CFLG
A8H	IE	WDCON	WDFLG	WDVTHL	WDVTHH	PLLCON	-	TFCFG
A0H	P2	CKCON	CKSEL	CKDIV	IHCFGL	IHCFGH	ILCFGL	ILCFGH
98H	S0CON	S0BUF	S1CON	S1BUF	S1RELL	S1RELH	RCMSHL	RCMSHH
90H	P1	RCCON	VCKDL	VCKDH	RCTAGL	RCTAGH	RCMSLL	RCMSLH
88H	TCON	TMOD	TL0	TL1	TH0	TH1	IT1CON	IT0CON
80H	P0	SP	DP0L	DP0H	DP1L	DP1H	PWCON	PCON

由于 SFR 地址空间有限，MJ603 系列芯片在外部 RAM 地址空间增加了扩展特殊功能寄存器，扩展特殊功能寄存器映射如图表 7-2-2 所示。

表 **7-2-2** 扩展特殊功能寄存器映射表

	0/8	1/9	2/A	3/B	4/C	5/D	6/E	7/F
8000H	P00F	P01F	P02F	P03F	P04F	P05F	P06F	P07F
8008H	P10F	P11F	P12F	P13F	P14F	P15F	P16F	P17F
8010H	P20F	P21F	-	-	-	-	-	-
8018H	P30F	P31F	P32F	P33F	P34F	P35F	P36F	P37F
8020H	ADCALL	ADCALH	-	-	-	-	-	-
8100H	SPMAX	I2CIOS	-	TKPWC	-	-	TLEN	TLDAT
8108H	TLCON	TLFLG	TLCKS	TLCNTKL	TLCNTKH	TLCNTLL	TLCNTLH	TLDIV
8110H	TLCOMS	-	-	-	-	-	-	LXCAD
8118H	-	-	-	-	-	-	-	-
8120H	P00C	P01C	P02C	P03C	P04C	P05C	P06C	P07C
8128H	P10C	P11C	P12C	P13C	P14C	P15C	P16C	P17C
8130H	P20C	P21C	-	-	-	-	-	-
8138H	P30C	P31C	P32C	P33C	P34C	P35C	P36C	P37C
FC00H	MECON	FSCMD	FSDAT	LOCK	PADRD	PTSL	PTSH	REPSET

7.3 Flash 存储器

7.3.1 功能简介

MJ603 系列芯片提供 8 / 16/ 32K 字节 Flash 存储器（不同型号容量有所不同），Flash 存储器可重复擦写。Flash 存储器由一组特定的寄存器控制，用户可用这些寄存器进行读写擦、设置写保护等操作。

7.3.2 Flash 存储器组织结构

- Flash 由若干个扇区组成，扇区是进行擦除操作的最小单位，每个扇区大小为 128 字节
- Flash 可以按功能划分为程序区和数据区，划分单位为 256 字节，程序区用于存储用户的程序，数据区是用于存储一些掉电需要保存的数据。



图 **7-3-1　8K Flash** 存储器结构



图 **7-3-2　16K Flash** 存储器结构



图 **7-3-3　32K Flash** 存储器结构

7.3.3 Flash 寄存器描述

表 **7-3-3-1** 寄存器 **MECON**

FC00H	7	6	5	4	3	2	1	0
MECON	REMAP	DPSTB	-	-		-	BOOT[1:0]	
R/W	R/W	R/W	-	-		-	R/W	
初始值	0	0	-	-		-	0	0

位编号	位符号	说明
7	REMAP	XRAM 地址映射控制位 0：地址映射关闭 1：地址映射使能，2K XRAM 映射到程序空间地址为 8000H~87FFH。
6	DPSTB	IDLE/STOP 模式下 Flash 进入睡眠模式控制位 0：IDLE/STOP 模式下，Flash 处于正常工作模式 1：IDLE/STOP 模式下，Flash 进入睡眠模式 备注：如果 *DPSTB=1*，当芯片进入 *IDLE/STOP* 模式，*Flash* 也同时进入睡眠模式，*Flash* 在睡眠模式的功耗为 *50nA*，当芯片退出 *IDLE/STOP* 模式，*Flash* 也同时退出睡眠模式。
5~2	-	-
1~0	BOOT	设置软复位后程序启动空间选择位域 01：软复位后程序从 XRAM 启动运行 10：软复位后程序从 FLASH 启动运行

表 **7-3-3-2** 寄存器 **FSCMD**

FC01H	7	6	5	4	3	2	1	0
FSCMD	**IFEN**	-	-	-	CMD[3:0]			
R/W	R/W	-	-	-	R/W			
初始值	0	-	-	-	0	0	0	0
位编号	位符号	说明						
7	IFEN	信息区访问使能位，访问时需将此位置位						
6~4	-	-						
3~0	CMD	命令寄存器 0000：无操作 0100：Flash 整片擦除 0001：读 Flash 数据区 0010：写 Flash 数据区 0011：扇区擦 Flash 数据区 1011：块擦 Flash 数据区(每块为 512 字节) 0101：读 Flash 程序区 0110：写 Flash 程序区 0111：扇区擦 Flash 程序区 1111：块擦 Flash 程序区(每块为 512 字节) 备注： *1.* 擦除命令执行后 *CMD* 自动清零。 *2.* 读和写命令写入后 *CMD* 保持不变然后通过读写 *FSDAT* 完成。						

表 **7-3-3-3** 寄存器 **FSDAT**

FC02H	7	6	5	4	3	2	1	0
FSDAT	FSDAT[7:0]							
R/W	R/W							
初始值	-	-	-	-	-	-	-	-
位编号	位符号	说明						
7~0	FSDAT	Flash 数据寄存器						

表 **7-3-3-4** 寄存器 **LOCK**

FC03H	7	6	5	4	3	2	1	0
LOCK								
R		-			FLKF	PLKF	DLKF	ILKF
W	LOCK[7:0]							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
写操作		
7~0	LOCK	28H：对 Flash 可编程区解锁 29H：对 Flash 程序区解锁 2AH：对 Flash 数据区解锁 AAH：Flash 加锁，不能进行写擦操作
读操作		
7	-	
6	-	-
5	-	
4	-	
3	FLKF	可编程区解锁标志，1 表示已解锁
2	PLKF	程序区解锁标志，1 表示已解锁
1	DLKF	数据区解锁标志，1 表示已解锁
-	-	-

表 **7-3-3-5** 寄存器 **PADRD**

FC04H	7	6	5	4	3	2	1	0
PADRD	PADRD[7:0]							
R/W	R/W							
初始值	1	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	PADRD	程序区和数据区划分配置寄存器 程序区和数据区以 256 字节为单位进行划分，当 PADRD>0 时， 程序区的地址空间为: 0 ~ (PADRD×256 - 1), 数据区的地址空间 为: (PADRD×256) ~ 1FFFH/3FFFH/7FFFH. 备注： 1. 当 PADRD=0 时，整个 Flash 空间都是数据空间。 2. 1FFFH/3FFFH/7FFFH 分别表示 8K/16K/32K Flash 的最大地址。 3. 当 Flash 容量为 8K/16K/32K 时，PADRD 的最大值分别为 20H/40H/80H。PADRD 的设置值不能超过最大值。

表 **7-3-3-6** 寄存器 **PTS**

FC05H	7	6	5	4	3	2	1	0
PTSL	PTS[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
FC06H	7	6	5	4	3	2	1	0
PTSH		PTS[14:8]						
R/W	-	R/W						
初始值		0	0	0	0	0	0	0

位编号	位符号	说明
15	-	
14~0	PTS	目标地址指针寄存器

7.3.4 Flash 控制例程

- **Flash** 划分程序区和数据区

  例如，32K 的 Flash 空间划分最后 1024 字节为数据空间，其余为程序空间，程序如下：

```
---------------------------------------------------------------------------------
PADRD = 124; //程序区空间地址为：0~0x7BFF,数据区空间地址为：0x7C00~0x7FFF
备注：以上设置数据区在 FLASH 中的物理地址是 0x7C00~0x7FFF，但是逻辑地址是 0x0000~0x03FF，读写数据区时应填写逻辑地址。
---------------------------------------------------------------------------------
```

- 数据空间扇区擦除

  例如，需要擦除数据空间扇区 n(扇区序号 *n=0*、*1*、*2……)*，程序如下：

```
---------------------------------------------------------------------------------
FSCMD = 0;                                  //设置 CMD 为 0
LOCK = 0x2A;                                //数据空间解锁
PTSH = (unsigned char)((n*0x80)>>8);  //设置扇区高位地址
PTSL = (unsigned char)(n*0x80);         //设置扇区低位地址
FSCMD = 3;                                  //设置擦除命令
LOCK = 0xAA;                                //FLASH 加锁
---------------------------------------------------------------------------------
```

- 数据空间写入数据

  例如，往数据空间地址为 n~(n+100)写入数据 0xAA，程序如下：

```
---------------------------------------------------------------------------------
unsigned char i;
FSCMD = 0;          //设置 CMD 为 0
LOCK = 0x2A;  //数据空间解锁
PTSH = (unsigned char)(n>>8); //设置数据首地址高 8 位
PTSL = (unsigned char)n;          //设置数据首地址低 8 位
FSCMD = 2;    //设置写命令
for(i=0;i<100;i++)
{
    FSDAT = 0xAA; //连续写入数据
}
LOCK = 0xAA;  //FLASH 加锁
---------------------------------------------------------------------------------
```

备注：

1．当连续写入数据时，只需设置首地址，每次写 *FSDAT* 后，数据指针寄存器 *PTS* 会自动累加。

2．读写数据区时，设置的地址是数据区的逻辑地址，而不是 *FLASH* 的物理地址，逻辑地址是从 *0* 开始的。

◆ 数据空间读出数据

例如，从数据空间地址为 n~(n+100)读出数据到指针 pBuf，程序如下：

```
---------------------------------------------------------------------------------
unsigned char i，*pBuf;
FSCMD = 0;          //设置 CMD 为 0
LOCK = 0x2A;  //数据空间解锁
PTSH = (unsigned char)(n>>8); //设置数据首地址高 8 位
PTSL = (unsigned char)n;          //设置数据首地址低 8 位
FSCMD = 1;    //设置读命令
for(i=0;i<100;i++)
{
    *pBuf++ = FSDAT ;  //连续写入数据
}
LOCK = 0xAA;  //FLASH 加锁
---------------------------------------------------------------------------------
```

备注：当连续读出数据时，只需设置首地址，每次读 *FSDAT* 后，数据指针寄存器 *PTS* 会自动累加。

◆ 程序空间扇区擦除

例如，需要擦除程序空间扇区 n(扇区序号 *n=0*、*1*、*2……)*，程序如下：

```
---------------------------------------------------------------------------------
FSCMD = 0;          //设置 CMD 为 0
LOCK = 0x29;  //程序空间解锁
PTSH = (unsigned char)((n*0x80)>>8); //设置扇区高位地址
PTSL = (unsigned char)(n*0x80);          //设置扇区低位地址
FSCMD = 7;    //设置擦除命令
LOCK = 0xAA;  //FLASH 加锁
---------------------------------------------------------------------------------
```

◆ 程序空间写入数据

例如，往程序空间地址为 n~(n+100)写入数据 0xAA，程序如下：

```
---------------------------------------------------------------------------------
unsigned char i;
FSCMD = 0;          //设置 CMD 为 0
LOCK = 0x29;  //程序空间解锁
PTSH = (unsigned char)(n>>8); //设置数据首地址高 8 位
PTSL = (unsigned char)n;          //设置数据首地址低 8 位
FSCMD = 6;    //设置写命令
for(i=0;i<100;i++)
{
    FSDAT = 0xAA; //连续写入数据
}
LOCK = 0xAA;  //FLASH 加锁
---------------------------------------------------------------------------------
```

备注：当连续写入数据时，只需设置首地址，每次写 *FSDAT* 后，数据指针寄存器 *PTS* 会自动累加。

◆ 程序空间读出数据

例如，从程序空间地址为 n~(n+100)读出数据到指针 pBuf，程序如下：

```
---------------------------------------------------------------------------------
unsigned char i，*pBuf;
FSCMD = 0;          //设置 CMD 为 0
LOCK = 0x29;  //程序空间解锁
PTSH = (unsigned char)(n>>8); //设置数据首地址高 8 位
PTSL = (unsigned char)n;          //设置数据首地址低 8 位
FSCMD = 5;    //设置读命令
for(i=0;i<100;i++)
{
     *pBuf++ = FSDAT ;  //连续写入数据
}
LOCK = 0xAA;  //FLASH 加锁
---------------------------------------------------------------------------------
```

备注：当连续读出数据时，只需设置首地址，每次读 *FSDAT* 后，数据指针寄存器 *PTS* 会自动累加。

7.4 外部 RAM 映射为程序空间

外部 2K RAM 可以映射为程序空间使用,当 REMAP=0（详见寄存器 MECON）时，映射地址为 0000H~07FFH（执行软复位从 XRAM 开始运行才有效）；当 REMAP=1 时，映射地址为 8000H~87FFH，映射图如图 7-5-1 所示。用户可以下载程序到外部 RAM 空间，当程序运行时设置 REMP=1 后，再执行跳转指令跳到映射程序区执行。同样效果，也可把 BOOT[1:0]（详见寄存器 MECON）的值设置为 01，然后执行软复位，复位后程序从外部 RAM 空间开始执行（此时映射地址为 0000H~07FFH）。映射程序区用来实现 IAP 等功能特别方便。

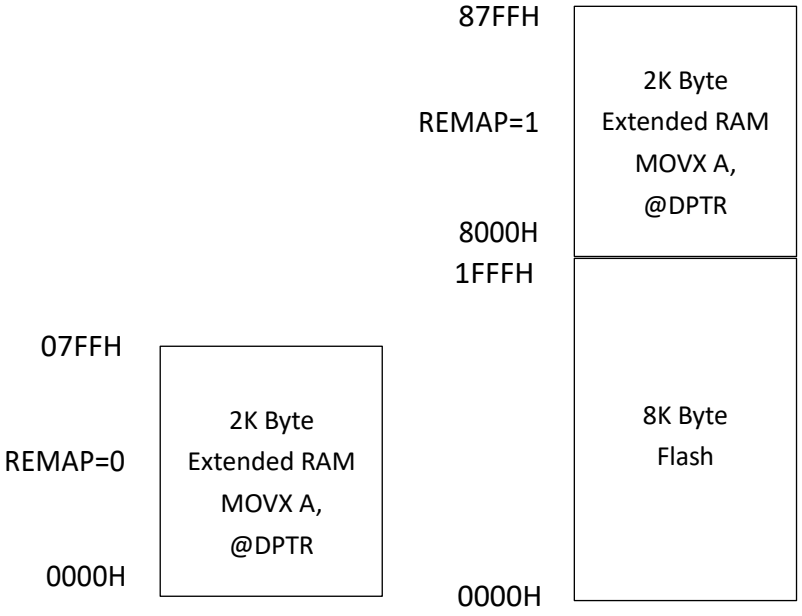


图 **7-4-1 XRAM** 地址映射图（**Flash** 容量为 **8K**）

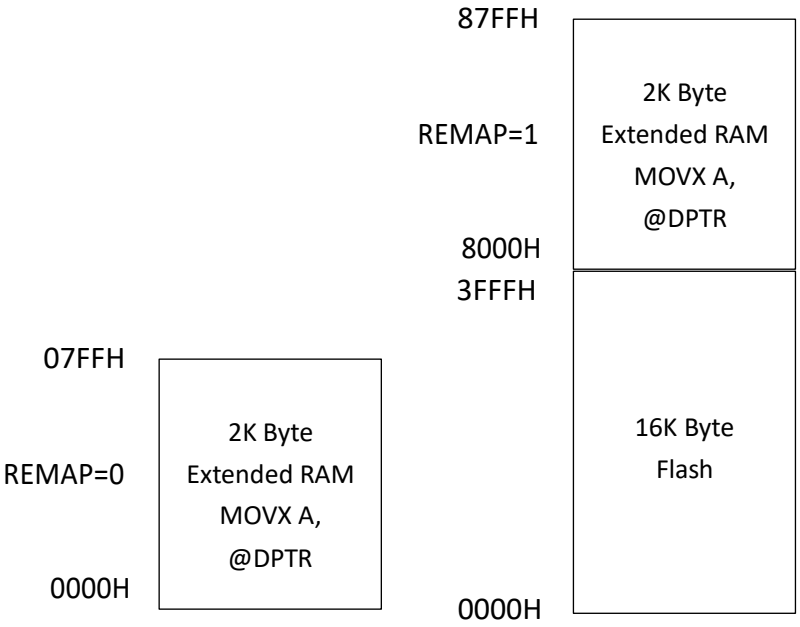


图 **7-4-2 XRAM** 地址映射图（**Flash** 容量为 **16K**）

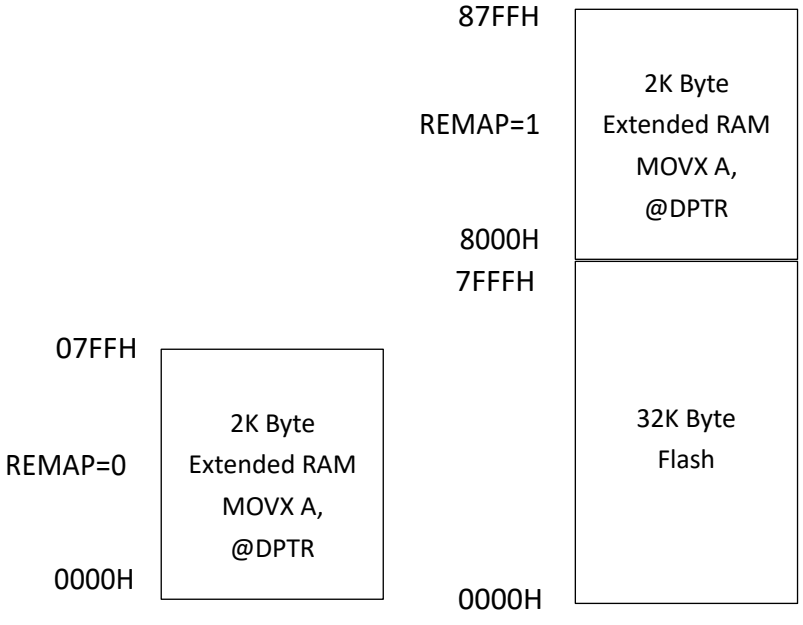


图 **7-4-3 XRAM** 地址映射图（**Flash** 容量为 **32K**）

8 中断系统

8.1 功能简介

MJ603 系列芯片有一个增强的中断控制系统，共有 15 个中断入口，每个中断入口有若干中断源，每个中断源有 2 级中断优先级。每个中断源都有独立的中断向量、优先级设置位、中断使能位、中断标志。CPU 在响应中断后，进入该中断对应的中断服务程序，接到 RETI 指令后将返回中断前状态。如果同时有多个有效中断产生中断请求，CPU 将根据设置的中断优先级依次响应；如果优先级相同，则根据它们的自然优先级（中断入口地址从低到高）依次响应。

8.2 中断逻辑

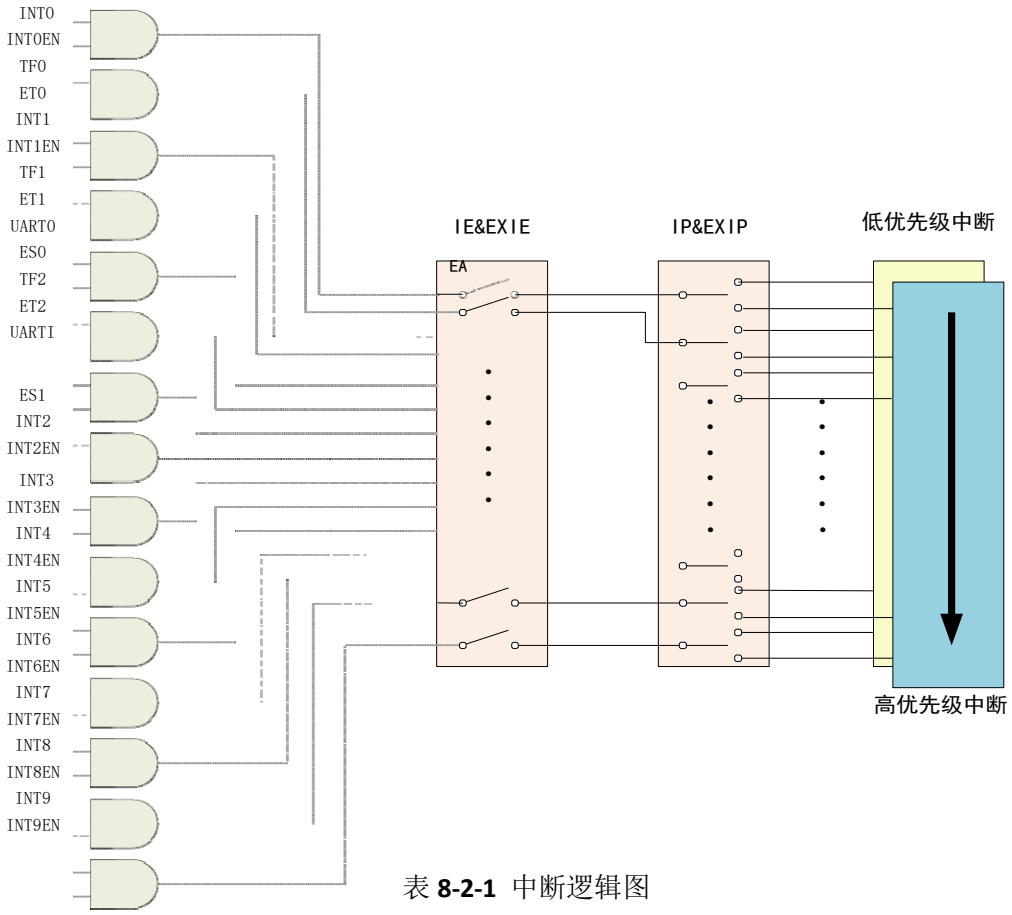


表 **8-2-1** 中断逻辑图

8.3 中断向量表

中断	中断源	向量	默认优先级
INT0	INT0	03H	0
TF0	定时器 0	0BH	1
INT1	INT1	13H	2
TF1	定时器 1	1BH	3
UART0	UART0	23H	4
TF2	定时器 2	2BH	5
UART1	UART1	33H	6
INT2	ADC/外部中断 2	3BH	7
INT3	TK/外部中断 3	43H	8
INT4	LVD/外部中断 4	4BH	9
INT5	时钟监控中断/外部中断 5	53H	10
INT6	I2C/外部中断 6	5BH	11
INT7	WDT/外部中断 7	63H	12
INT8	RTC/外部中断 8	6BH	13
INT9	PWM/外部中断 9	73H	14

表 **8-3-1** 中断向量表

8.4 中断控制寄存器

表 **8-4-1** 寄存器 **IE**

A8H	7	6	5	4	3	2	1	0
IE	EA	ES1	ET2	ES0	ET1	INT1EN	ET0	INT0EN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	EA	全局中断使能控制位 0：全局中断关闭 1：全局中断打开
6	ES1	UART1 中断使能控制位 0：UART1 中断关闭 1：UART1 中断打开
5	ET2	定时器 2 中断使能控制位 0：定时器 2 中断关闭 1：定时器 2 中断打开
4	ES0	UART0 中断使能控制位 0：UART0 中断关闭 1：UART0 中断打开

3	ET1	定时器 1 中断使能控制位 0：定时器 1 中断关闭 1：定时器 1 中断打开
2	EX1	外部中断 1 使能控制位 0：外部中断 1 中断关闭 1：外部中断 1 中断打开
1	ET0	定时器 0 中断使能控制位
0	EX0	外部中断 0 使能控制位 0：外部中断 0 中断关闭 1：外部中断 0 中断打开

表 **8-4-2** 寄存器 **EXIE**

E8H	7	6	5	4	3	2	1	0
EXIE	INT9EN	INT8EN	INT7EN	INT6EN	INT5EN	INT4EN	INT3EN	INT2EN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	INT9EN	中断 9 使能控制位（中断 9 用于 PWM/外部中断 9） 0：关闭 1：打开
6	INT8EN	中断 8 使能控制位（中断 8 用于 RTC/外部中断 8） 0：关闭 1：打开
5	INT7EN	中断 7 使能控制位（中断 7 用于 WDT/外部中断 7） 0：关闭 1：打开
4	INT6EN	中断 6 使能控制位（中断 6 用于 I2C/外部中断 6） 0：关闭 1：打开
3	INT5EN	中断 5 使能控制位（中断 5 用于时钟监控/外部中断 5） 0：关闭 1：打开
2	INT4EN	中断 4 使能控制位（中断 4 用于 LVD/外部中断 4） 0：关闭 1：打开
1	INT3EN	中断 3 使能控制位（中断 3 用于 TK/外部中断 3） 0：关闭 1：打开
0	INT2EN	中断 2 使能控制位（中断 2 用于 ADC/外部中断 2） 0：关闭 1：打开

备注：*EXIE* 的使能控制位是对应中断向量的，各中断源的中断开关也要另外打开。例如：要开启外部中 *2* 的中断，除了设置 *INT2EN* 为 *1*，*EPIE2*（外部中断 *2* 使能位）也要设为 *1*。

表 **8-4-3** 寄存器 **IP**

B8H	7	6	5	4	3	2	1	0
IP	-	PS1	PT2	PS0	PT1	PX1	PT0	PX0
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	-	-
6	PS1	UART1 优先级控制位 0：低优先级 1：高优先级
5	PT2	定时器 2 优先级控制位 0：低优先级 1：高优先级
4	PS0	UART0 优先级控制位 0：低优先级 1：高优先级
3	PT1	定时器 1 优先级控制位 0：低优先级 1：高优先级
2	PX1	外部中断 1 优先级控制位 0：低优先级 1：高优先级
1	PT0	定时器 0 优先级控制位 0：低优先级 1：高优先级
0	PX0	外部中断 0 优先级控制位 0：低优先级 1：高优先级

表 **8-4-4** 寄存器 **EXIP**

F8H	7	6	5	4	3	2	1	0
EXIP	PX9	PX8	PX7	PX6	PX5	PX4	PX3	PX2
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	PX9	中断 INT9 优先级控制位 0：低优先级 1：高优先级
6	PX8	中断 INT8 优先级控制位 0：低优先级 1：高优先级
5	PX7	中断 INT7 优先级控制位 0：低优先级 1：高优先级
4	PX6	中断 INT6 优先级控制位 0：低优先级 1：高优先级
3	PX5	中断 INT5 优先级控制位 0：低优先级 1：高优先级
2	PX4	中断 INT4 优先级控制位 0：低优先级 1：高优先级
1	PX3	中断 INT3 优先级控制位 0：低优先级 1：高优先级
0	PX2	中断 INT2 优先级控制位 0：低优先级 1：高优先级

8.5 外部中断

8.5.1 外部中断介绍

INT0 和 INT1 在标准 8051 的基础上，增加了可选择任意输入口作为中断触发源的功能。系统还扩展了 8 个中断入口 INT2~INT9 作为外部中断，每个中断入口也可选择任意输入口作为中断触发源，扩展外部中断可单独设置上升沿或下降沿触发中断。每个外部中断都可以用于 STOP 模式唤醒。EPIF 为 INT2~INT9 外部中断状态寄存器。INT2~INT9 对应的各个配置寄存器 EPCON0~EPCON7 可通过配置索引寄存器 INDEX 为 0~7 来访问。

备注：*INT0* 和 *INT1* 可选择上升沿或下降沿触发，选择位分别为 *IT0* 和 *IT1*，详见寄存器 *TCON* 相关描述。

8.5.2 外部中断寄存器

表 **8-5-2-1** 寄存器 **IT0CON**

8FH	7	6	5	4	3	2	1	0
IT0CON	-	-	IT0PS[5:0]					
R/W	-	-	R/W					
初始值	-	-	0	1	1	0	1	0

位编号	位符号	说明
7~6	-	-
5~0	IT0PS[5:0]	INT0 中断管脚选择位 编号和管脚对应表参考表 8-5-2-6

表 **8-5-2-2** 寄存器 **IT1CON**

8EH	7	6	5	4	3	2	1	0
IT1CON	--	-	-	IT1PS[5:0]				
R/W	-	-	-	R/W				
初始值	-	-	-	1	1	0	1	1

位编号	位符号	说明
7~5	-	-
4~0	IT1PS[5:0]	INT1 中断引脚选择位 编号和管脚对应表参考表 8-5-2-6

表 **8-5-2-3** 寄存器 **EPIE**

F9H	7	6	5	4	3	2	1	0
EPIE	EPIE9	EPIE8	EPIE7	EPIE6	EPIE5	EPIE4	EPIE3	EPIE2
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	EPIE9	外部中断 9 使能位
6	EPIE8	外部中断 8 使能位
5	EPIE7	外部中断 7 使能位
4	EPIE6	外部中断 6 使能位
3	EPIE5	外部中断 5 使能位
2	EPIE4	外部中断 4 使能位
1	EPIE3	外部中断 3 使能位
0	EPIE2	外部中断 2 使能位

表 **8-5-2-4** 寄存器 **EPIF**

FAH	7	6	5	4	3	2	1	0
EPIF	EPIF9	EPIF8	EPIF7	EPIF6	EPIF5	EPIF4	EPIF3	EPIF2
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	EPIF9	外部中断 9 中断标志位，写 1 清零
6	EPIF8	外部中断 8 中断标志位，写 1 清零
5	EPIF7	外部中断 7 中断标志位，写 1 清零
4	EPIF6	外部中断 6 中断标志位，写 1 清零
3	EPIF5	外部中断 5 中断标志位，写 1 清零
2	EPIF4	外部中断 4 中断标志位，写 1 清零
1	EPIF3	外部中断 3 中断标志位，写 1 清零
0	EPIF2	外部中断 2 中断标志位，写 1 清零

表 **8-5-2-5** 寄存器 **EPCON**

FBH	7	6	5	4	3	2	1	0
EPCON	EPPL	-	-	EPPS[5:0]				
R/W	R/W	-	-	R/W				
初始值	0	-	-	0	0	0	0	0
备注：EPCON 为带索引寄存器，设置 INDEX=0~7 分别对应 EPCON0~EPCON7								
位编号	位符号	说明						
7	EPPL	外部中断触发沿选择位 0：上升沿 1：下降沿						
6~5	-	-						
4~0	EPPS[4:0]	中断引脚选择位域 编号和引脚对应表参考表 8-5-2-6						

表 **8-5-2-6** 中断引脚编号索引表

引脚名称	编号
P00	0
P01	1
P02	2
P03	3
P04	4
P05	5
P06	6
P07	7
P10	8
P11	9
P12	10
P13	11
P14	12
P15	13
P16	14
P17	15
P20	16
P21	17
P30	24
P31	25
P32	26
P33	27

P34	28
P35	29
P36	30
P37	31

8.5.3 外部中断控制方法及例程

- 外部中断 **0/1** 控制例程

  例如，设置 P20 为外部中断 0 中断输入引脚并开启外部中断 0 (下降沿中断)，程序如下:

```
---------------------------------------------------------------------------------
void INT0_init(void)
{
    P20F = 1;           //设置 P20 为输入引脚
    IT0CON = 16;        //选择 P20 作为中断 0 引脚，16 为 P20 对应的索引编号
    EX0 = 1;            //INT0 中断使能
    IE0 = 1;            //外部中断 0 使能
    IT0 = 1;            //设置为下降沿中断
    PX0 = 1;            //设置 INT0 为高优先级
    EA = 1;             //总中断使能
}
void INT0_ISR (void) interrupt 0
{
    //外部中断 0 中断服务程序
}
---------------------------------------------------------------------------------
```

  例如，设置 P20 为外部中断 1 中断输入引脚并开启外部中断 1 (下降沿中断)，程序如下:

```
---------------------------------------------------------------------------------
void INT1_init(void)
{
    P20F = 1;           //设置 P20 为输入引脚
    IT1CON = 16;        //选择 P20 作为中断 1 引脚，16 为 P20 对应的索引编号
    EX1 = 1;            //INT1 中断使能
    IE1 = 1;            //外部中断 1 使能
    IT1 = 1;            //设置为下降沿中断
    PX1 = 1;            //设置 INT1 为高优先级
    EA = 1;             //总中断使能
}
void INT1_ISR (void) interrupt 2
{
    //外部中断 1 中断服务程序
}
---------------------------------------------------------------------------------
```

◆ 外部中断 **2~9** 控制例程

以外部中断 2 为例，设置 P20 为外部中断 2 中断输入引脚并开启外部中断 2，程序如下:

```
--------------------------------------------------------------------------------
void INT2_init(void)
{
    P20F = 1;                 //设置 P20 为输入引脚
    INDEX = 0;                //设置 EPCON 索引号，0~7 分别对应外部中断 2~9
    EPCON = (0<<7) | 16;      /*设置为上升沿触发并设置中断引脚索引编号，16 对应 P20，如果设置为下
                              降沿触发则设置为 EPCON = (1<<7) | 16;*/
    INT2EN = 1;               //INT2 中断使能
    EPIE |= 0x01;             //外部中断 2 中断使能
    EA = 1;                   //总中断使能
}
void INT2_ISR (void) interrupt 7
{
    if(EPIF & 0x01)       //判断外部中断 2 中断标志
    {
        EPIF = 0x01;    //中断标志写 1 清 0
        //外部中断 2 中断服务程序
        ......
    }
}
--------------------------------------------------------------------------------
```

9 时钟系统

9.1 时钟系统介绍

时钟系统负责整个系统时钟的生成、分频以及分配工作。MJ603 系列芯片共支持以下时钟源：

- 内置 2~4MHz RC 振荡器
- 内置 131KHz RC 振荡器
- 内置 4MHz RC 振荡器
- 支持外部 1~27MHz RC 谐振器
- 支持外部 32.768KHz 晶体谐振器
- 倍频倍数为 2~10 倍的 PLL

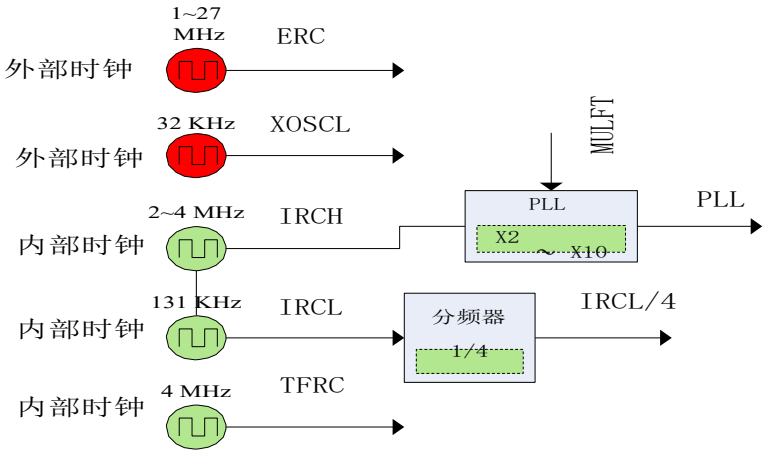


图 **9-1-1** 时钟源示意图

用户可独立的管理各个时钟源，每个时钟源都可以单独打开或关闭，从而可以灵活控制功耗。 所有时钟源都可设置为系统时钟，也可分配到各种外设中，作为外设的时钟源，详细请参考外设部分介绍。

9.1.1 时钟专用名称定义

名称缩写	描述	名称缩写	描述
IRCH	内置 2~4MHz RC 振荡器	ERC	外部 RC 振荡器
IRCL	内置 131KHz RC 振荡器	TFRC	内置 4MHz RC 振荡器
XOSCL	外部 32.768KHz 晶体振荡器	PLL	倍数为 2~10 倍的 PLL

9.1.2 内置 2 - 4MHz RC 振荡器（IRCH）

IRCH 是芯片上电后默认的系统时钟，可通过寄存器 CKCON 的 IHCKE 位打开或关闭。IRCH 的频率范围是 2~4MHz，可使用寄存器 IHCFGH、IHCFGL 设置不同频率，也可用内置 RC 校正模块以其他时钟源为参考时钟进行自动校正，校正精度可达 1%@25℃，出厂频率校正为 3.6864MHz±1%@3.3V/25℃。

备注：由于工艺的原因，*IHCFGH*、*IHCFGL* 设置相同的值每个芯片对应的 *IRCH* 频率不一定相同，*IRCL* 和 *TFRC* 也是同样情况。

9.1.3 外部 32.768KHz 晶体谐振器（XOSCL）

XOSCL 主要是作为 RTC 的时钟源，用于实时计时，实现产品的时钟功能。XOSCL 设置为系统时钟时主要应用在低功耗的场合，最低功耗可小于 13uA。XOSCL 通过寄存器 CKCON 的 XLCKE 位打开或关闭，要注意的是，XOSCL 起振时间比较长，大约需要 1 秒左右才能达到稳定，在应用时需要等待 XOSCL 时钟稳定后才可以使用，寄存器 CKCON 的 XLSTA 位是 XOSCL 时钟稳定标志。

9.1.4 内置 131KHz RC 振荡器（IRCL）

IRCL 可通过寄存器 CKCON 的 ILCKE 位打开或关闭。和 XOSCL 类似，IRCL 设为系统时钟也可实现系统低功耗。在没有外挂 32.768KHz 晶振的情况下，IRCL 也可设置为 RTC 模块时钟源，应用于对计时精度不高的场合。可使用寄存器 ILCFGH、ILCFGL 设置不同频率，也可用内置 RC 校正模块以其他时钟源为参考时钟进行自动校正，出厂频率校正为 131KHz±2%@3.3V/25℃。

9.1.5 内置 4MHz RC 振荡器（TFRC）

TFRC 可通过寄存器 CKCON 的 TFCKE 位打开或关闭。TFRC 主要是作为 Flash 工作时钟和触摸模块充放电时钟，可使用寄存器 TFCFG 设置不同频率，也可用内置 RC 校正模块以其他时钟源为参考时钟进行自动校正，出厂频率校正为 4MHz±2%@3.3V/25℃。

9.1.6 锁相环 PLL

内置倍频倍数为 2~10 倍的 PLL 以 IRCH 为参考时钟进行倍频，PLL 的最高频率可达 40MHz，在不需外挂高速晶振的情况下利用 PLL 也可实现芯片的高速运行。PLL 通过寄存器 PLLCON 可以设置 PLL 的开关及倍频倍数，要注意的是，PLL 打开后必须等待 PLL 时钟稳定才可以将其设置为系统时钟或其他外设时钟，PLL 时钟是否稳定可通过寄存器 PLLCON 的 PLSTA 位判断。PLL 打开后，大约需要约 50us 才能达到稳定状态。 重要提醒：由于 *CPU* 的最高频率为 *27MHz*，当 *PLL* 频率超过 *27MHz* 时不可设置为 *CPU* 时钟，但是可以设为其他外设的时钟。

9.1.7 外部高速 RC 振荡器（ERC）

ERC 可通过寄存器 CKCON 的 XHCKE 位打开或关闭。

9.2 时钟控制寄存器描述

表 **9-2-1** 寄存器 **CKCON**

A1H	7	6	5	4	3	2	1	0
CKCON	ILCKE	IHCKE	TFCKE	-	XLCKE	XLSTA	XHCKE	-
R/W	R/W	R/W	R/W	-	R/W	R	R/W	-
初始值	0	0	0	-	1	0	0	-

位编号	位符号	说明
7	ILCKE	IRCL 使能控制位 1：打开 0：关闭 备注： 该位为*1*时，时钟模块打开，但是该位为*0*时，如果系统或者其他模块选择了该时钟源，该时钟仍然会被打开。
6	IHCKE	IRCH 使能控制位 1：打开 0：关闭 备注： 该位为*1*时，时钟模块打开，但是该位为*0*时，如果系统或者其他模块选择了该时钟源，该时钟仍然会被打开。
5	TFCKE	TFRC 使能控制位 1：打开 0：关闭 备注： 该位为*1*时，时钟模块打开，但是该位为*0*时，如果其他模块选择了该时钟源，该时钟仍然会被打开。
4	-	-
3	XLCKE	XOSCL 使能控制位 1：打开 0：关闭 备注： *1* 该位为*1*时，时钟模块打开，但是该位为*0*时，如果其他模块选择了该时钟源，该时钟仍然会被打开。 *2* 由于 *XOSCL* 为外部时钟，所以启动 *XOSCL* 还需要把对应的引脚功能配置为 *XOSCL* 的功能。
2	XLSTA	XOSCL 时钟稳定信号标志，1 有效
1	XHCKE	ERC 使能控制位 1：打开 0：关闭 备注：

		1 位有效时，时钟模块开启，但是该位为*0*时，如果系统或者其他模块选择了该时钟源，该时钟仍然会被打开。 *2* 由于 *ERC* 为外部时钟，所以若想所以启动 *ERC* 还需要把对应的引脚功能配置为 *ERC* 的功能。
0	-	-

表 **9-2-2** 寄存器 **PLLCON**

AEH	7	6	5	4	3	2	1	0
PLLCON	PLLON	MULFT[3:0]				-	-	PLSTA
R/W	R/W	R/W	R/W	R/W	R/W	-	-	R
初始值	0	0	0	0	0	-	-	0

位编号	位符号	说明
7	PLLON	PLL 使能控制位 0：关闭 1：打开
6~3	MULFT	PLL 倍频倍数设置位 0000:2 倍 0001:3 倍 0010:4 倍 0011:5 倍 0100:6 倍 0101:7 倍 0110:8 倍 0111:9 倍 1000:10 倍 其他值：无效
2~1	-	-
0	PLSTA	PLL 时钟稳定标志位，1 表示已稳定

表 **9-2-3** 寄存器 **IHCFGL**、**IHCFGH**

A4H	7	6	5	4	3	2	1	0
IHCFGL	IHCFG[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	1	1	1	1	1	1	1
A5H	7	6	5	4	3	2	1	0
IHCFGH	IHCFG[15:8]							
R/W	-	-	-	-	R/W	R/W	R/W	R/W
初始值	-	-	-	-	1	0	0	0

位编号	位符号	说明
11~0	IHCFG	IRCH 频率校正寄存器 备注：此寄存器上电后自动加载值对应频率 *3.6864MHz*，除特殊应用外，不建议修改此数值。

表 **9-2-4** 寄存器 **ILCFGL**、**ILCFGH**

A6H	7	6	5	4	3	2	1	0
ILCFGL	ILCFG[7:0]							
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
A7H	7	6	5	4	3	2	1	0
ILCFGH	-	-	-	-	-	-	-	ILCFG[8]
R/W	-	-	-	-	-	-	-	R/W
初始值	-	-	-	-	-	-	-	1

位编号	位符号	说明
8~0	ILCFG	IRCL 频率校正寄存器 备注：此寄存器上电后自动加载值对应频率 *131KHz*，除特殊应用外，不建议修改此数值。

表 **9-2-5** 寄存器 **TFCFG**

AFH	7	6	5	4	3	2	1	0
TFCFG	-	-	TFCFG[7:0]					
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
初始值	-	-	1	0	0	0	0	0

位编号	位符号	说明
7~0	TFCFG	TFRC 频率校正寄存器 备注：此寄存器上电后自动加载值对应频率 *4MHz*，除特殊应用外，不建议修改此数值。

9.3 系统时钟

MJ603 系列芯片的所有时钟源都可以设置为系统时钟。系统时钟控制由寄存器 CKCON、CKSEL、CKDIV 完成。通过这些寄存器组，可以单独设置各时钟源的开关、系统时钟的切换和分频等操作。

9.3.1 系统时钟结构图

系统时钟结构图见图 9-3-1。

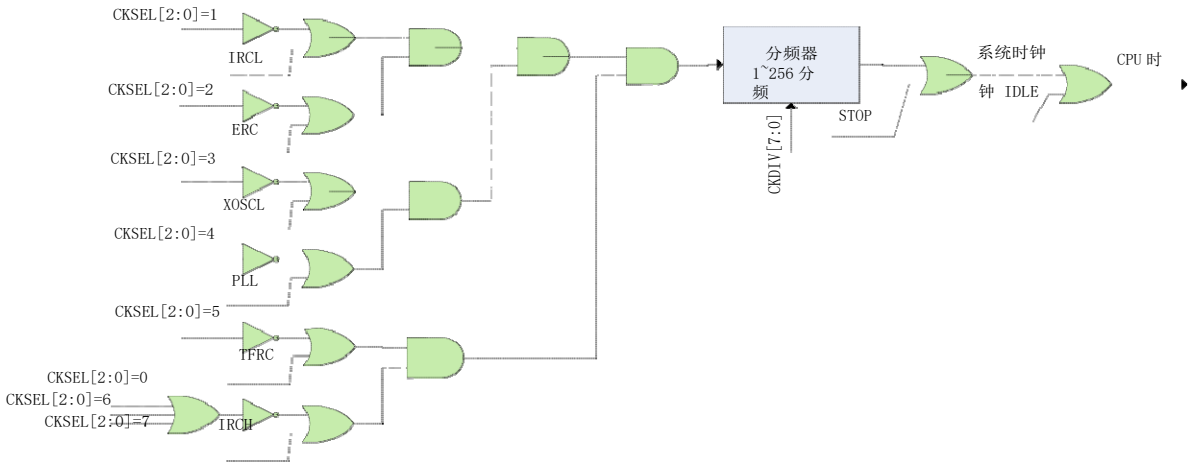


图 **9-3-1-1** 系统时钟结构图

9.3.2 系统时钟控制寄存器描述

表 **9-3-2-1** 寄存器 **CKSEL**

A2H	7	6	5	4	3	2	1	0
CKSEL	RTCKS	-	-	-	-	CKSEL[2:0]		
R/W	R/W	-	-	-	-	R/W		
初始值	0	-	-	-	-	0	0	0

位编号	位符号	说明
7	RTCKS	RTC 时钟源选择位 0：XOSCL 1：IRCL 备注：当设置为 IRCL 时，RTC 的时钟为 IRCL 的 4 分频
6~3	-	-
2~0	CKSEL	系统时钟选择位 000：IRCH 001：IRCL

		010：ERC 011：XOSCL 100：PLL 101：TFRC 其他：IRCH

表 **9-3-2-2** 寄存器 **CKDIV**

A3H	7	6	5	4	3	2	1	0
CKDIV	CKDIV[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	CKDIV	系统时钟分频： 00H：不分频 01H：2 分频 02H：3 分频 03H：4 分频 …… FFH：256 分频						

9.3.3 系统时钟控制方法及例程

- 设置系统时钟为 **IRCH**

  设置系统时钟为 IRCH，程序如下：

```
---------------------------------------------------------------------------------
#define IHCKE               (1<<6)
#define CKSEL_IRCH      0
void Sys_Clk_Set_IRCH(void)
{
    CKCON |= IHCKE;    //打开 IRCH 时钟
    CKSEL = (CKSEL&0xF8) | CKSEL_IRCH;          //设置时钟为 IRCH
}
---------------------------------------------------------------------------------
```

- 设置系统时钟为 **XOSCL**

  设置系统时钟为 XOSCL，程序如下：

```
---------------------------------------------------------------------------------
#define XLCKE               (1<<3)
#define XLSTA               (1<<2)
#define CKSEL_XOSCL     3
void Sys_Clk_Set_XOSCL(void)
{
    P32F = 3; //设置 P32,P33 为晶振引脚功能
    P33F = 3;
    CKCON |= XLCKE;                 //打开 XOSCL 时钟
    while(!(CKCON & XLSTA));      //等待 XOSCL 时钟稳定
    CKSEL = (CKSEL&0xF8) | CKSEL_XOSCL; //设置时钟为 XOSCL
}
---------------------------------------------------------------------------------
```

- 设置系统时钟为 **IRCL**

  设置系统时钟为 IRCL，程序如下：

```
---------------------------------------------------------------------------------
#define ILCKE               (1<<7)
#define CKSEL_IRCL      1
void Sys_Clk_Set_IRCL(void)
{
    CKCON |= ILCKE;          //打开 IRCL 时钟
    CKSEL = (CKSEL&0xF8) | CKSEL_IRCL;    //设置时钟为 IRCL
}
---------------------------------------------------------------------------------
```

◆ 设置系统时钟为 **PLL**

设置系统时钟为 PLL，程序如下：

```
---------------------------------------------------------------------------------------
#define IHCKE        (1<<6)

//寄存器 RCCON 定义
#define PLLON(N)          (N<<7)          //N=0~1
#define MULFT(N)          (N<<3)          //N=0~8
#define PLSTA             (1<<0)
#define CKSEL_PLL         4

void Sys_Clk_Set_PLL(unsigned char Multiple)      //Multiple: 倍频倍数
{
      if(Multiple < 2 || Multiple > 8) return;
      CKCON |= IHCKE;
      PLLCON = PLLON(1) | MULFT(Multiple-2);
      while(!(PLLCON & PLSTA));
      CKSEL = (CKSEL&0xF8) | CKSEL_PLL;
}
---------------------------------------------------------------------------------------
```

◆ 设置系统时钟为 **TFRC**

设置系统时钟为 TFRC，程序如下：

```
---------------------------------------------------------------------------------------
#define TFCKE             (1<<5)
#define CKSEL_TFRC        5
void Sys_Clk_Set_TFRC(void)
{
      CKCON |= TFCKE;
      CKSEL = (CKSEL&0xF8) | CKSEL_TFRC;
}
---------------------------------------------------------------------------------------
```

9.4 内部 RC 振荡器校正

9.4.1 校正模块介绍

由于工艺的偏差，每个芯片的 RC 振荡器的频率出厂时都不一定相同，所以芯片出厂后有必要对 RC 振荡器进行校正。MJ603 系列芯片内置自动校正模块，可以对内部 RC 振荡器进行校正。校正模块共有 5 个参考时钟源选择：IRCH、IRCL、XOSCL、ERC、TFRC，有 3 个目标时钟选择：IRCH、IRCL、TFRC。选择参考时钟源时必须保证参考时钟源的频率是准确的，否则校正后的目标时钟也相应的不准确。图 9-4-1 为校正模块电路示意图，其中 VCLK 为参考时钟，TCLK 为目标时钟。

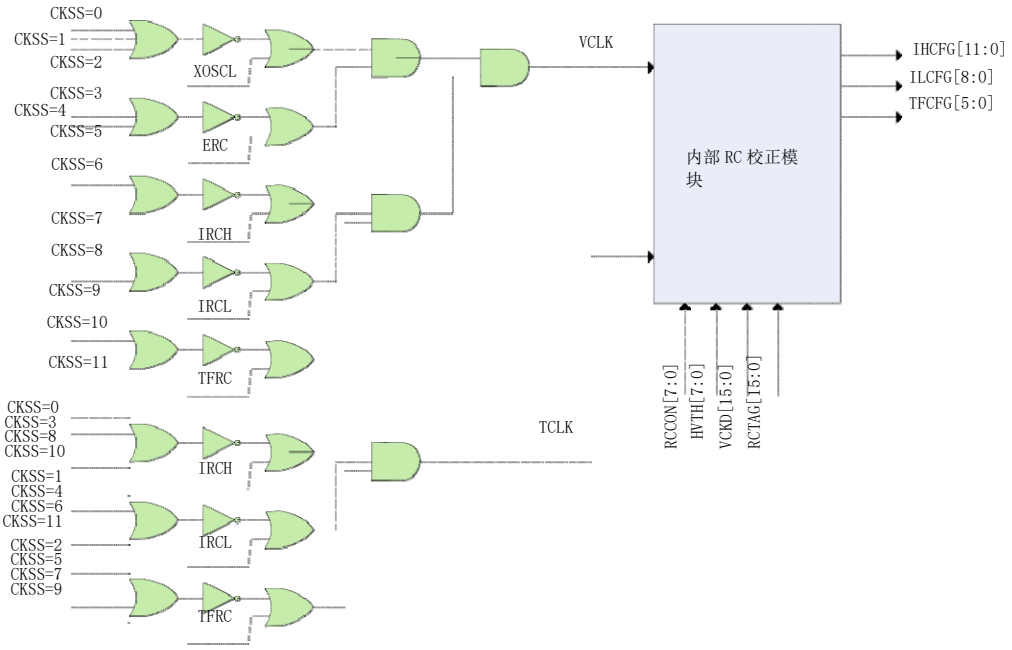


图 **9-4-1** 校正电路示意图

RC 校正模块共有 **3** 种工作模式：

- 计数模式

计数模式用于手动测量 TCLK，设置 MODE（RCCON[7:6]）为 1 后启动 TCLK 计数，MODE 设置为 0 后计数停止，停止计数后计数值存入寄存器 RCMS（RCMSHH/RCMSHL/RCMSLH/RCMSLL）中。在应用中，用户可以在确定的时间段内启动 TCLK 计数，通过对计数值 RCMS 简单的计算，可以得到 TCLK 的频率。

- 测量模式

测量模式在若干个 VCLK 时钟周期内对 TCLK 进行计数，通过计数值来推算出 TCLK 的频率。设置 MODE 为 2 开始测量，测量完成后计数值存入 RCMS 寄存器，MODE 自动清 0。为了提高测量精度，应尽量延长测量周期时间长度，可以通过配置寄存器 VCKD（VCKDH/VCKDL），设置一个测量周期为 VCLK 的 VCKD 倍。这样，测量之后经过对计数值的简单计算，可以得到 TCLK 的频率。计算公式如下：

TCLK 的周期＝ (VCLK 的周期 x VCKD) ÷RCMS

- 校正模式

校正模式通过二分法，把若干个 VCLK 时钟周期内 TCLK 的计数值和设定值（对应目标频率）不断进行比对，比对达到用户设置的差值的阈值 HVTH 时产生 HMSK 标志。用户可以通过设置 MSE 位来选择 HMSK 位置 1 时即停止校正或二分法拆分到最后才停止。用户设置寄存器 HVTH 的原则是根据用户对 TCLK 时钟频率误差范围接受程度来设置，HVTH 越小，校正精度越高，但校正时间可能会更长。设置 MODE 为 3 启动校正，校正结束后，MODE 自动清 0。和测量模式一样，单个测量周期应尽量延长，以提高校正精度，但是校正时间也会相应延长。VCLK 和 TCLK 之间的关系是：

VCLK 的周期 x VCKD ＝ TCLK 的目标周期 x RCTAG

9.4.2 校正模块控制寄存器

表 **9-4-2-1** 寄存器 **RCCON**

91H	7	6	5	4	3	2	1	0
RCCON	MODE[1:0]		MSE	HMSK	CKSS[3:0]			
R/W	R/W		R/	R	R/W			
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~6	MODE	工作模式选择位 01：计数模式，设置 MODE 为 0 则退出计数模式 10：测量模式，完成后 MODE 自动清 0 11：校正模式，完成后 MODE 自动清 0
5	MSE	校正模式下退出方式选择位 0：即使检测到 HMSK 被置 1，也等到校正次数足够才退出 1：检测到 HMSK 被置 1 后直接退出
4	HMSK	在校正模式下，校正值与目标值差值小于等于 HVTH，HMSK 将被置 1，否则清 0
3~0	CKSS	时钟配对选择位 0000：目标时钟 IRCH，参考时钟 XOSCL 0001：目标时钟 IRCL，参考时钟 XOSCL 0010：目标时钟 TFRC，参考时钟 XOSCL 0011：目标时钟 IRCH，参考时钟 ERC 0100：目标时钟 IRCL，参考时钟 ERC 0101：目标时钟 TFRC，参考时钟 ERC 0110：目标时钟 IRCL，参考时钟 IRCH 0111：目标时钟 TFRC，参考时钟 IRCH 1000：目标时钟 IRCH，参考时钟 IRCL 1001：目标时钟 TFRC，参考时钟 IRCL 1010：目标时钟 IRCH，参考时钟 TFRC 1011：目标时钟 IRCL，参考时钟 TFRC 其他值：无效

表 **9-4-2-2** 寄存器 **HVTH**

AFH	7	6	5	4	3	2	1	0
HVTH	HVTH[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	HVTH	校正模式下，校正值与目标值差值阈值寄存器						

表 **9-4-2-3** 寄存器 **VCKDL**、**VCKDH**

92H	7	6	5	4	3	2	1	0
VCKDL	VCKD[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
93H	7	6	5	4	3	2	1	0
VCKDH	VCKD[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
15~0	VCKD	测量和校正模式下，参考时钟分频倍数，为 VCKD 倍分频(VCKD>1)						

表 **9-4-2-4** 寄存器 **RCTAGL**、**RCTAGH**

94H	7	6	5	4	3	2	1	0
RCTAGL	RCTAGL[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
95H	7	6	5	4	3	2	1	0
RCTAGH	RCTAGH[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
15~0	RCTAG	校正模式下，目标时钟分频倍数，为 RCTAG 倍分频(RCTAG>=1)						

表 **9-4-2-5** 寄存器 **RCMSLL**、**RCMSLH**、**RCMSHL**、**RCMSHH**

96H	7	6	5	4	3	2	1	0
RCMSLL	RCMS[7:0]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
97H	7	6	5	4	3	2	1	0
RCMSLH	RCMS[15:8]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
9EH	7	6	5	4	3	2	1	0
RCMSHL	RCMS[23:16]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
9FH	7	6	5	4	3	2	1	0
RCMSHH	RCMS[31:24]							
R/W	R							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
31~0	RCMS	计数模式完成后，存放计数结果 测量模式完成后，存放测量结果 校正模式下， RCMS 的值无意义

9.4.3 校正模块控制例程

◆ 校正 **IRCH** 例程

参考时钟设为 XOSCL，目标时钟设为 IRCH，校正目标频率为 3.6864MHz,程序如下：

```
--------------------------------------------------------------------------------
#define IHCKE           (1<<6)
#define XLCKE           (1<<3)
#define XLSTA           (1<<2)

#define MODE(N)         (N<<6)      //N=0~3
#define MSEX(N)         (N<<5)      //N=0~1
#define CKSS(N)         N           //N=0~11

CKCON |= IHCKE;                     //打开 IRCH 时钟
CKCON |= XLCKE;                     //打开 XOSCL 时钟
while(!(CKCON & XLSTA));            //等待 XOSCL 时钟稳定
RCTAGH =    ((3686400*400)/32768)/256;  //设置目标频率
RCTAGL =    ((3686400*400)/32768)%256;
VCKDH = 400/256;                    //设置参考时钟分频，分频后的频率为 81.92HZ
```

```
VCKDL = 400%256;
RCCON = MODE(3) | MSEX(0) | CKSS(0);      //设置目标时钟为 IRCH，参考时钟为 XOSCL，设置校正方式，
                                          //启动校正模式
while(RCCON&0xC0);                        //等待校正完成
-------------------------------------------------------------------------------
```

◆ 校正 **IRCL** 例程

参考时钟设为 XOSCL，目标时钟设为 IRCL，校正目标频率为 131KHz,程序如下：

```
-------------------------------------------------------------------------------
#define ILCKE                 (1<<7)
#define XLCKE                 (1<<3)
#define XLSTA                 (1<<2)

#define MODE(N)               (N<<6)        //N=0~3
#define MSEX(N)               (N<<5)        //N=0~1
#define CKSS(N)               N             //N=0~11

CKCON |= ILCKE;                   //打开 IRCL 时钟
CKCON |= XLCKE;                   //打开 XOSCL 时钟
while(!(CKCON & XLSTA));          //等待 XOSCL 时钟稳定
RCTAGH =     ((131000*400)/32768)/256;    //设置目标频率
RCTAGL =     ((131000*400)/32768)%256;
VCKDH = 400/256;                          //设置参考时钟分频，分频后的频率为 81.92HZ
VCKDL = 400%256;
RCCON = MODE(3) | MSEX(0) | CKSS(1);      /*设置目标时钟为 IRCL，参考时钟为 XOSCL，设置校正方式，
                                          启动校正模式*/
while(RCCON&0xC0);                        //等待校正完成
-------------------------------------------------------------------------------
```

◆ 校正 **TFRC** 例程

参考时钟设为 XOSCL，目标时钟设为 TFRC，校正目标频率为 4MHz,程序如下：

```
-------------------------------------------------------------------------------
#define TFCKE             (1<<5)
#define XLCKE             (1<<3)
#define XLSTA             (1<<2)

#define MODE(N)           (N<<6)        //N=0~3
#define MSEX(N)           (N<<5)        //N=0~1
#define CKSS(N)           N             //N=0~11

CKCON |= TFCKE;                           //打开 TFRC 时钟
CKCON |= XLCKE;                           //打开 XOSCL 时钟
while(!(CKCON & XLSTA));                  //等待 XOSCL 时钟稳定
RCTAGH =     ((4000000*400)/32768)/256;   //设置目标频率
```

```
RCTAGL =     ((4000000*400)/32768)%256;
VCKDH = 400/256;                        //设置参考时钟分频，分频后的频率为 81.92HZ
VCKDL = 400%256;
RCCON = MODE(3) | MSEX(0) | CKSS(2);    /*设置目标时钟为 TFRC，参考时钟为 XOSCL，设置校正方式，
                                        启动校正模式*/
while(RCCON&0xC0);                      //等待校正完成
```

9.5 外部时钟监控

9.5.1 功能描述

时钟监控模块用于监控外部时钟的异常，并且作出处理，增强了系统的可靠性。开启时钟监控模块后，当外部时钟作为系统时钟时，如果外部时钟停振，系统时钟将切换到 IRCL。当外部时钟作为 RTC 时钟时，如果外部时钟停振，RTC 时钟将切换到 IRCL 四分频时钟。当外部时钟作为 WDT 时钟时，如果外部时钟停振，WDT 时钟将切换到 IRCL 四分频时钟。

外部高速时钟（ERC）监控通过 MHE 位使能，而中断使能通过 IHE 位设置。当 ERC 时钟出现异常后，时钟异常标志位 XHFD 置 1，如果设置 ATH=1，当 ERC 时钟恢复正常后，时钟源会自动恢复回 ERC；当 ATH=0，清除中断标志位 XHFD 就会恢复时钟源为 ERC。

外部低速时钟（XOSCL）监控通过 MLE 位使能，而中断使能通过 ILE 位设置。当 XOSCL 时钟出现异常后，时钟异常标志位 XLFD 置 1，如果设置 ATL=1，当 XOSCL 时钟恢复正常后，时钟源会自动恢复回 XOSCL；当 ATH=0，清除中断标志位 XLFD 就会恢复时钟源为 XOSCL。

标志位 HSP 和 LSP 分别指示 ERC 和 XOSCL 的当前状态，当 HSP=1 或 LSP=1，分别表示 ERC 和 XOSCL 出现异常，时钟已切换到内部 RC 时钟。

系统在 STOP 或 IDLE 模式也可通过时钟监控中断唤醒。

9.5.2 外部时钟监控控制寄存器

表 **9-5-2-1** 寄存器 **CKMON**

BEH	7	6	5	4	3	2	1	0
CKMON	MHE	IHE	ATH	HSW	MLE	ILE	ATL	LSW
R/W	R/W	R/W	R/W	W	R/W	R/W	R/W	W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	MHE	时钟监控模块监控 ERC 时钟控制位，1 有效
6	IHE	时钟监控 ERC 中断使能位，1 有效
5	ATH	时钟监控模块自动恢复为 ERC 使能位，1 有效 备 注： 模块检测到 ERC 异常后，将会把对应电路的工作时钟切换为 IRCL，如果 ATH 位为 1，那么在 ERC 恢复正常之后，模块将会自动把对应电路的工作时钟切换回 ERC。如果 ATH 位为 0，则必须用户对 HSW 写 1 才能切换回 ERC。
4	HSW	只写寄存器，对其写 1 将清除 HSP（CKMIF[7]）
3	MLE	时钟监控模块监控 XOSCL 时钟控制位，1 有效
2	ILE	时钟监控 XOSCL 中断使能位，1 有效
1	ATL	时钟监控模块自动恢复为 XOSCL 使能位，1 有效 备注：

		模块检测到 XOSCL 异常后，将会把对应电路的工作时钟切换为 IRCL，如果 ATL 位为 1，那么在 XOSCL 恢复正常之后，模块将会自动把对应电路的工作时钟切换回 XOSCL。如果 ATL 位为 0，则必须用户对 LSW 写 1 才能切换回 XOSCL。
0	LSW	只写寄存器，对其写 1 将清除 LSP（CKMIF[6]）

表 **9-5-2-2** 寄存器 **CKMIF**

BFH	7	6	5	4	3	2	1	0
CKMIF	HSP	LSP	-	-	-	-	XHFD	XLFD
R/W	R	R	-	-	-	-	R	R
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	HSP	ERC 异常，时钟切换到内部时钟的状态标志位，1 表示工作于内部时钟
6	LSP	XOSCL 异常，时钟切换到内部时钟的状态标志位，1 表示工作于内部时钟
5~2	-	-
1	XHFD	监控 ERC 异常中断标志，1 有效，写 1 清 0
0	XLFD	监控 XOSCL 异常中断标志，1 有效，写 1 清 0

10 供电和复位系统

10.1 供电系统

在 MJ603 系列芯片 VDD 和 VSS 引脚间接入 1.8V~5.5V 的电源，用于供电。模拟系统由 VDD 以及 LDO 供电，数字系统只需 LDO 供电。

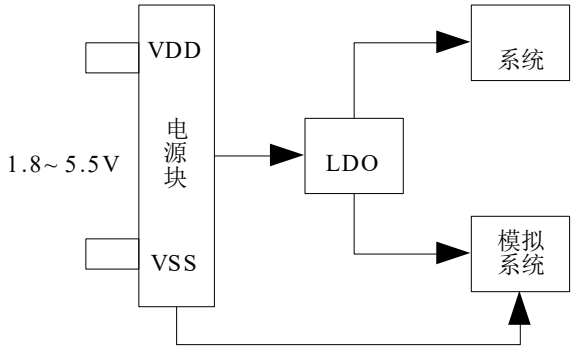


图 **10-1-1** 供电系统示意图

图 **10-1-2** 为芯片供电典型电路图。

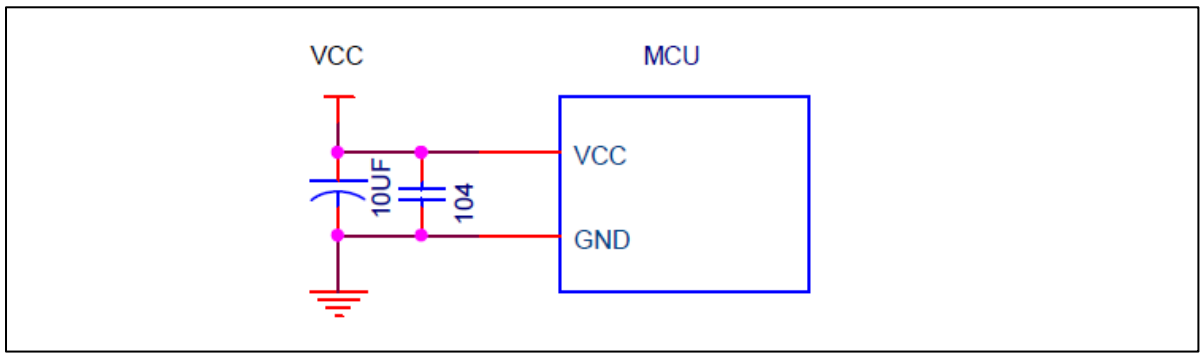


图 **10-1-2** 芯片供电典型电路图

重要提醒：*1.* 以上电路中，滤波电容 *10uF* 和 *104* 为芯片供电电路标配，不可省略，此电容须靠近芯片电源引脚摆放，否则有可能会导致芯片工作异常。

*2.*以上电路及元件参数仅供参考，根据外围工作环境及不同电压供电参数可能需要修改。

10.1.1 LDO 功能简介

MJ603 系列芯片有一个内置的低压差线性稳压器（LDO）。LDO 模块为芯片提供内核电压。LDO 的输出电压通过 VLEVEL 位（PWCON[2:0]）设置，VLEVEL 默认值为 5，对应的输出电压为 1.61V。当 VDD/VSS 小于 VLEVEL 位设定的输出电压时，LDO 直接输出 VDD；当 VDD/VSS 大于设定电压时，LDO 输出设定的电压。LDO 设置高电压有助于时钟模块快速启动，而设置为较低电压时有助于降低芯片功耗。LDO 有两种不同的工作模式：高功率模式和低功率模式，通过 VHL 位（PWCON[3]）设置。两种模式下，LDO 的负载能力不相同。在高功率模式，LDO 能提供的电流更大，但自身功耗也更大，低功率模式则反之。当系统处于正常工作状态时，LDO 一般都设置为高功率模式，而低功率模式一般应用于省电模式，例如 STOP、IDLE、低速运行模式等。

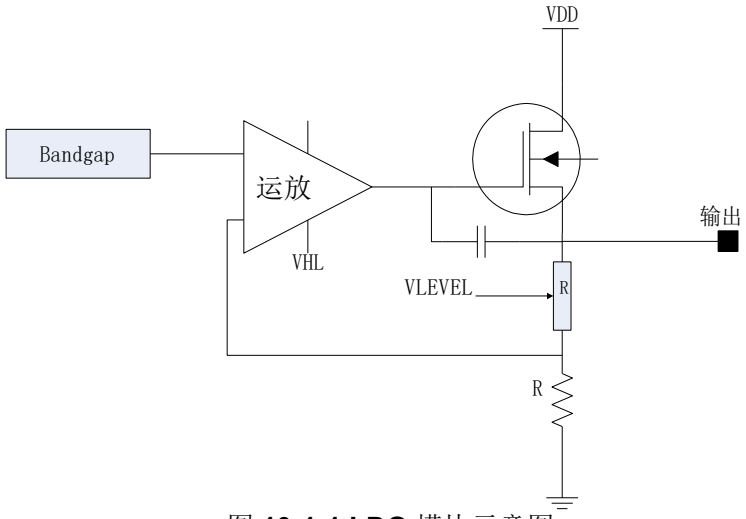


图 **10-1-1 LDO** 模块示意图

10.1.2 LDO 控制寄存器

表 **10-1-2-1** 寄存器 **PWCON**

86H	7	6	5	4	3	2	1	0
PWCON	FLEVEL[3:0]				VHL	VLEVEL[2:0]		
R/W	R/W				R/W	R/W		
初始值	0	1	1	1	1	1	0	1

位编号	位符号	说明
7~4	FLEVEL	内部基准电压（Bandgap）输出调整位域 0000：0.825V 0001：0.850V 0010：0.875V 0011：0.900V 0100：0.925V 0101：0.950V 0110：0.975V 0111：1.000V

		1000：1.025V 1001：1.050V 1010：1.075V 1011：1.100V 1100：1.125V 1101：1.150V 1110：1.175V 1111：1.200V 备注：内部基准电压上电时由系统自动加载，用户不允许修改。
3	VHL	LDO 工作模式控制位 1：高功率模式 0：低功率模式
2~0	VLEVEL	LDO 输出电压设置位域 000：1.31V 001：1.37V 010：1.43V 011：1.49V 100：1.55V 101：1.61V 110：1.67V 111：1.73V 备注： *1.* 内部时钟电路由 *LDO* 供电，改变 *LDO* 输出电压会引起内部时钟频率的变化，一般情况下， *LDO* 电压保持默认值即可，不建议修改。 *2.* 不允许设置 *LDO* 输出电压小于 *1.5V*，否则有可能引起异常。

10.2 复位系统

MJ603 系列芯片有多个内部和外部复位源，如图 10-2-1 所示。

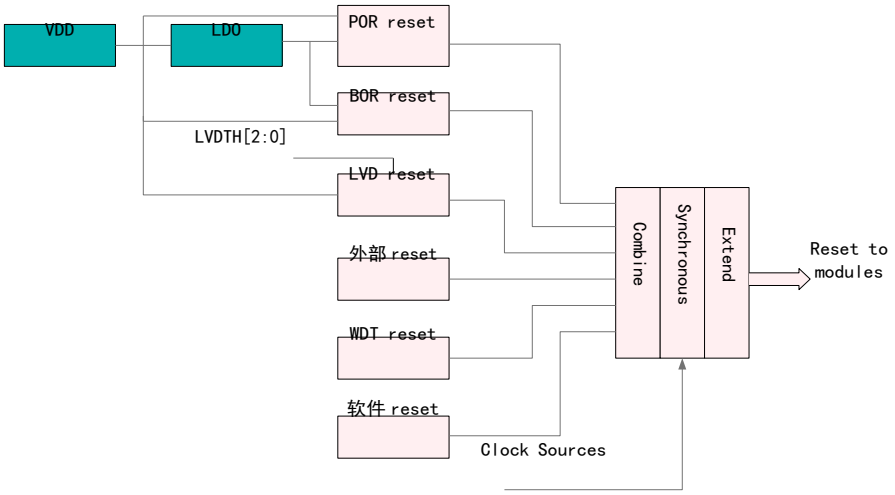
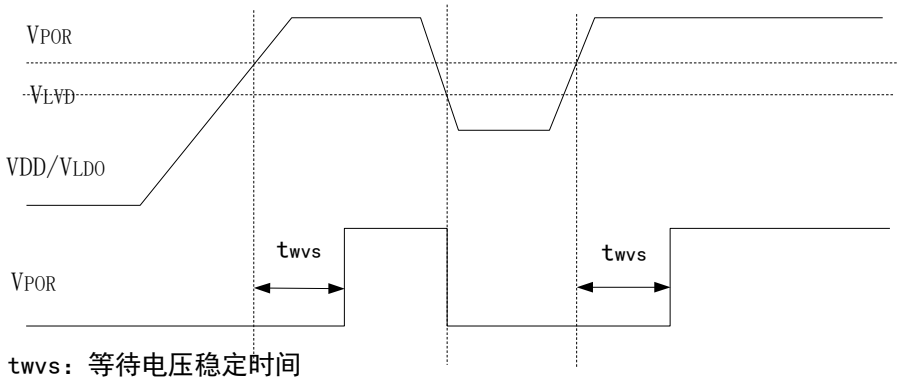


图 **10-2-1** 复位系统结构图

- 上电复位（**POR**）

系统上电呈现逐渐上升的曲线形式，需要一定时间才能达到正常的工作电压。上电复位是基于电源电压 VDD 和内部 LDO 的输出电压，当电压低于检测阈值时，上电复位信号有效。 上电复位电路能够保证芯片在上电过程中处于复位状态，芯片上电后能够从一个已知的稳定的状态开始运行。上
电复位信号也会被芯片内部的计数器展宽，以保证上电后内部的各种模拟模块能够进入稳定的工作状态。



twvs：等待电压稳定时间

图 **10-2-2** 上电复位电路示例及上电过程

- 掉电复位（**BOR**） 利用掉电复位，可以为芯片提供电源跌落(例如受到干扰或者负载变化)的预警信号。一旦发现电源电压 VDD
或内部 LDO 的输出电压下降到某一个阈值时，就使芯片及时复位以免系统工作状态不正常或者程序执行错误。

- 低电压复位

低电压检测（LVD）可以在多种工作模式下持续监控电源电压 VDD。当 VDD 低于 LVD 设定的域值电压超过 20us 就可以产生复位信号（前提是 LVD 设置为复位模式）。

- 外部复位

通过拉低复位引脚(RESET)，可以从外部源复位器件。在正常工作情况下，RESET 可以复位整个芯片，在 STOP 状态，硬复位会唤醒芯片后再复位。一般情况下，RESET 被内部上拉拉高，不会影响内部的复位电路。

特别提醒：复位引脚*(P20)*是多功能引脚*(P20* 默认为复位功能*)*，*P20* 只能从复位功能切换到其他功能*(*例如输入、输出、高阻*)*，*P20* 切换为其他功能引脚后，不能再切换为复位功能，否则会引起异常复位。

- 看门狗复位 看门狗定时器负责监控处理器执行指令的情况，通过合适的配置，如果看门狗定时器在特定时间段内未被刷

新，则可以产生复位信号。上电复位后，看门狗定时器是关闭的，用户需要时，再配置开启。

- 软复位

芯片可以在程序控制下执行软复位。通过对 PCON 寄存器中的 SWRST 位写 1，CPU 可以发出复位指令。软复位后，程序将从 BOOT 配置指向的位置开始运行。所有复位产生之后，PC 都将指向地址 0。

11 功耗管理

MJ603 系列芯片有三种不同的低功耗模式： IDLE 模式、STOP 模式、低速运行模式。STOP 模式时系统功耗小于 7uA，IDLE 模式时系统功耗小于 12uA，低速运行时功耗小于 20uA。

11.1 IDLE 模式

在 IDLE 模式下，CPU 将停止工作。进入 IDLE 模式前，除了主时钟，其他的时钟源根据需要都可选择关闭，以便节省功耗。同样地，进入 IDLE 模式前，可根据需要设定芯片某些外设的开关。打开的外设在 IDLE 状态下仍然可以正常工作。

设置进入 IDLE 模式前，需要先查看一下寄存器 IDLST（IDLSTH 和 IDLSTL），如果所有位都为 0，则设置进入 IDLE 模式后，CPU 将正常进入 IDLE 模式。如果 IDLST 的位不全为 0，即使有设置进入 IDLE 模式的操作，CPU 也不会进入 IDLE 模式，而是继续停留在正常工作模式。此时用户需先把 IDLST 对应位的中断处理完成，再重新设置进入 IDLE 模式的动作。

所有复位事件和任何中断事件都将唤醒芯片。中断唤醒 CPU 后，芯片首先将恢复时钟，然后响应该中断，进入该中断的服务程序。退出中断服务程序后，芯片将执行置位 IDLE 指令后面的指令。退出 IDLE 模式时，IDLE 位将自动清零。

需要注意的是，在置位 IDLE 的指令后面需要紧接两条 nop 指令，防止程序出错。

11.2 STOP 模式

STOP 模式是比 IDLE 更深层次的低功耗模式。STOP 模式可以停止所有时钟（包括主时钟）和时钟产生电路。如果 WDT 和 RTC 处于打开状态，则它们使用的时钟模块将处于工作状态，可以有选择地关闭 WDT 和 RTC 以节省功耗。

类似于 IDLE 模式，进入 STOP 模式前，需要先查看 STPST（STPSTH 和 STPSTL）寄存器，若有置 1 的位存在，需要先行处理，以确保能顺利进入 STOP 模式。

STOP 模式可以通过外部中断、LVD 中断或复位、硬复位、RTC 中断、WDT 中断或复位、时钟监控中断、触摸中断来唤醒。如果是中断唤醒，那么唤醒 MCU 后，芯片首先将恢复时钟，然后响应该中断，进入该中断的服务程序。退出中断服务程序后，芯片将执行置位 STOP 指令后面的指令。退出 STOP 模式时，STOP 位将自动清零。

为了更好的唤醒芯片，推荐在进入 STOP 模式前切换系统时钟到内部时钟，因为唤醒时，外部时钟需要更多时间去等待稳定。

在进入 STOP 模式时，最后一个时钟沿将关闭系统时钟，然后芯片完全进入 STOP 模式。需要注意的是，在置位 STOP 的指令后面需要紧接三条 nop 指令，防止程序出错。

重要提醒：进入 *STOP/IDLE* 模式时，设置 *LDO* 为低功率模式可有效降低待机功耗，但是退出 *STOP/IDLE* 模式时，一定要把 *LDO* 设置回高功率模式，否则会导致芯片工作异常。

11.3 低速运行模式

由于芯片的功耗与运行速度直接相关，所以把主时钟切换到低速时钟运行也可以显著降低功耗。系统支持两个低速时钟源：IRCL、XOSCL。系统时钟设为 XOSCL (32.768KHz)时的电流小于 15uA，而设为 IRCL (131KHz)时的电流小于 25uA.

关于 IRCL、XOSCL 更多介绍请参考时钟系统章节。

11.4 低功耗相关寄存器描述

表 **11-4-1** 寄存器 **PCON**

87H	7	6	5	4	3	2	1	0
PCON	SMOD	-	SWRST	-	-	TSMODE	STOP	IDLE
R/W	R/W	-	W	-	-	R	W	W
初始值	0	-	0	-	-	0	0	0

位编号	位符号	说明
7	SMOD	UART0 波特率倍频控制位 在 UART0 工作于模式 1,2,3 时，设置 SMOD=1 会使波特率倍频，与标准 8051 相同。
6	-	-
5	SWRST	软复位控制位，1 有效 设置 SWRST=1 产生软复位，复位产生后自动清 0。
4~3	-	-
2	TSMODE	调试模式标志位，为 1 表示芯片正工作于调试模式
1	STOP	STOP 模式控制位，1 有效 当设置 STOP=1 且 STPST 为 0 时，芯片进入 STOP 模式，退出 STOP 模式后自动清 0
0	IDLE	IDLE 模式控制位，1 有效 当设置 IDLE=1 且 IDLST 为 0 时，芯片进入 IDLE 模式，退出 IDLE 模式后自动清 0

表 **11-4-2** 寄存器 **IDLSTL**、**IDLSTH**

FCH	7	6	5	4	3	2	1	0
IDLSTL	IDLST[7:0]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
FDH	7	6	5	4	3	2	1	0
IDLSTH	-	IDLST[14:8]						
R/W	-	R						
初始值	-	0	0	0	0	0	0	0

位编号	位符号	说明
15	-	-
14	PWMINT/EPIF[7]	IDLE 模式时，PWM/外部中断 9 的中断状态
13	RTCINT/EPIF[6]	IDLE 模式时，RTC/外部中断 8 的中断状态
12	WDFLG[1]/EPIF[5]	IDLE 模式时，WDT/外部中断 7 的中断状态
11	I2CINT/EPIF[4]	IDLE 模式时，I2C/外部中断 6 的中断状态
10	CKMINT/EPIF[3]	IDLE 模式时，时钟监控/外部中断 5 的中断状态
9	LVDINT/EPIF[2]	IDLE 模式时，LVD/外部中断 4 的中断状态
8	TKINT/EPIF[1]	IDLE 模式时，TK/外部中断 3 的中断状态
7	ADCINT/EPIF[0]	IDLE 模式时，ADC/外部中断 2 的中断状态
6	U1INT	IDLE 模式时，UART1 中断的中断状态
5	T2INT	IDLE 模式时，定时器 2 的中断状态
4	U0INT	IDLE 模式时，UART0 的中断状态
3	TCON[7]	IDLE 模式时，定时器 1 的中断状态
2	PIF[1]	IDLE 模式时，外部中断 1 的中断状态
1	TCON[5]	IDLE 模式时，定时器 0 的中断状态
0	PIF[0]	IDLE 模式时，外部中断 0 的中断状态

表 **11-4-3** 寄存器 **STPSTL**、**STPSTH**

FEH	7	6	5	4	3	2	1	0
STPSTL	STPST[7:0]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
FFH	7	6	5	4	3	2	1	0
STPSTH	STPST[15:8]							
R/W	R							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
15	RTCWKF	STOP 模式时，RTC 中断状态
14	WDTWKF	STOP 模式时，WDT 中断状态
13	I2CWKF	STOP 模式时，I2C 的中断状态
12	CKMWKF	STOP 模式时，时钟监控的中断状态
11	LVDWKF	STOP 模式时，LVD 中断状态

10	TKWKF	STOP 模式时，触摸按键的中断状态
9	EPWKF[7]	STOP 模式时，外部中断 9 的中断状态
8	EPWKF[6]	STOP 模式时，外部中断 8 的中断状态
7	EPWKF[5]	STOP 模式时，外部中断 7 的中断状态
6	EPWKF[4]	STOP 模式时，外部中断 6 的中断状态
5	EPWKF[3]	STOP 模式时，外部中断 5 的中断状态
4	EPWKF[2]	STOP 模式时，外部中断 4 的中断状态
3	EPWKF[1]	STOP 模式时，外部中断 3 的中断状态
2	EPWKF[0]	STOP 模式时，外部中断 2 的中断状态
1	PWKF[1]	STOP 模式时，外部中断 1 的中断状态
0	PWKF[0]	STOP 模式时，外部中断 0 的中断状态

11.5 低功耗模式控制例程

◆ **STOP** 模式例程

STOP 模式程序如下：

```
-------------------------------------------------------------------------------------
void Stop(void)
{
    I2CCON = 0;             //关闭 I2C 功能，因为 I2C 默认是使能的，如果 I2C 不关闭将无法关闭 IRCH 时钟
    CKCON = 0;              //关闭所有时钟
    PWCON &=0xF7;           //设置 LDO 进入低功率模式
    MECON |= (1<<6);        //设置 FLASH 进入深度睡眠状态
    while(STPSTH|STPSTL);   //如果有中断未响应,等待中断被响应
    PCON |= 0x02;           //进入 STOP 模式
    _nop_();
    _nop_();
    PWCON  |=0x08;          //退出 STOP 后，必须把 LDO 设置回高功率模式
}
-------------------------------------------------------------------------------------
```

◆ **IDLE** 模式例程

IDLE 模式程序如下：

```
-------------------------------------------------------------------------------------
void Idle(void)
{
    I2CCON = 0;             //关闭 I2C 模块，因为 I2C 默认是使能的，如果 I2C 不关闭将无法关闭 IRCH 时钟
    CKCON = 0;              //关闭除主时钟外的所有时钟

    Sys_Clk_Set_XOSCL();    //切换主时钟到 XOSCL，见备注说明
    //Sys_Clk_Set_IRCL();   //切换主时钟到 IRCL，见备注说明

    PWCON &=0xF7;           //设置 LDO 进入低功率模式
    MECON |= (1<<6);        //设置 FLASH 进入深度睡眠状态
    while(IDLSTH|IDLSTL);   //如果有中断未响应,等待中断被响应
    PCON |= 0x01;           //进入 IDLE 模式
    _nop_();
    _nop_();
    PWCON  |=0x08;          //退出 IDLE 后，必须把 LDO 设置回高功率模式
}
```

备注：由于进入 *IDLE* 后，主时钟仍是打开的，如果进入 *IDLE* 前主时钟是高速时钟，进入 *IDLE* 模式后功耗仍会很大，所以进入 *IDLE* 之前需要把主时钟切换到低速时钟。

```
-------------------------------------------------------------------------------------
```

◆ 低速运行模式例程 低速
运行模式程序如下：

```
--------------------------------------------------------------------------------
void LowSpeedMode(void)
{
    I2CCON = 0;             //关闭 I2C 模块，因为 I2C 默认是使能的，如果 I2C 不关闭将无法关闭 IRCH 时钟
    Sys_Clk_Set_XOSCL();    //切换主时钟到 XOSCL，见备注说明
    //Sys_Clk_Set_IRCL();   //切换主时钟到 IRCL，见备注说明
    CKCON = 0;              //关闭除主时钟外的所有时钟
    PWCON &=0xF7;           //设置 LDO 进入低功率模式
}
备注：退出低速运行模式后，必须把 LDO 设置回高功率模式，参考 STOP/IDLE 例程。
--------------------------------------------------------------------------------
```

12 通用定时器（定时器 0,定时器 1,定时器 2）

12.1 定时器 0

12.1.1 定时器 0 介绍

定时器或计数器功能通过 CT0 位（TMOD[2]）来选择，CT0=0 选择为定时器，CT0=1 选择为计数器。作为定时器时，时钟是系统时钟的 12 分频。作为计数器时，时钟是 T0 的输入时钟。由于检测 T0 输入边沿变化需要 2 个时钟周期，所以作为计数器时最大的输入波特率是内部系统时钟频率的 1/2。T0 输入信号在占空比上没有限制，然而为了完全识别 0 或 1 的状态，信号至少需要保持 1 个内部系统时钟周期时间。定时器 0 有 4 个工作模式，通过 T0M0、T0M1 位(TMOD[1:0])来选择。

- 模式 **0**

在此模式下，定时器 0 作为 13 位定时器/计数器，TH0 存放 13 位定时器/计数器的高 8 位，TL0[4:0]存放低 5 位，而 TL0[7:5]是无效的，在读取时应被忽略。当定时器 0 溢出，中断标志位 TF0（TCON[5]）会被置 1。中断被响应后，TF0 位会自动清 0。当 GATE0（TCON[3]）=0 时，定时器/计数器由 TR0（TCON[4]）位使能计数，当 GATE0=1 时，定时器/计数器由引脚 INT0 控制使能，INT0 为高电平时计数，INT0 为低电平则停止计数。

- 模式 **1**

此模式下，定时器 0 作为 16 位定时器/计数器，除此之外，功能与模式 0 完全相同。

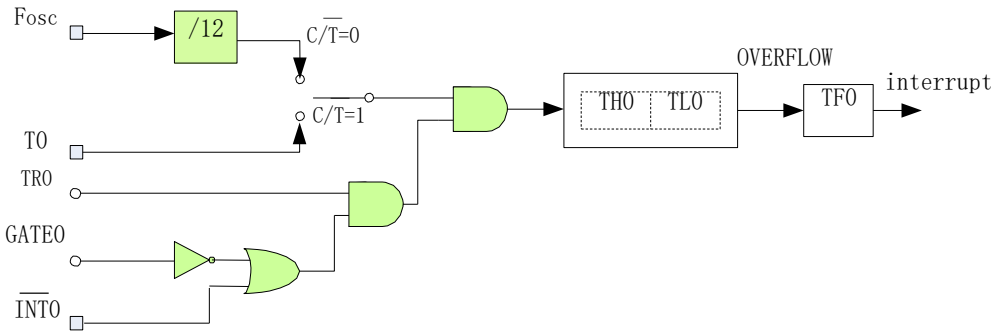


图 **12-1-1-1** 定时器 **0** 的模式 **0** 与 **1**

- 模式 **2**

在此模式中，定时器 0 作为 8 位自动重载定时器/计数器，只有 TL0 自动累加。当 TL0 计数溢出时，不但产生中断标志 TF0，而且从 TH0 中自动装载计数初始值到 TL0。其他设置方法和模式 0、1 相同。

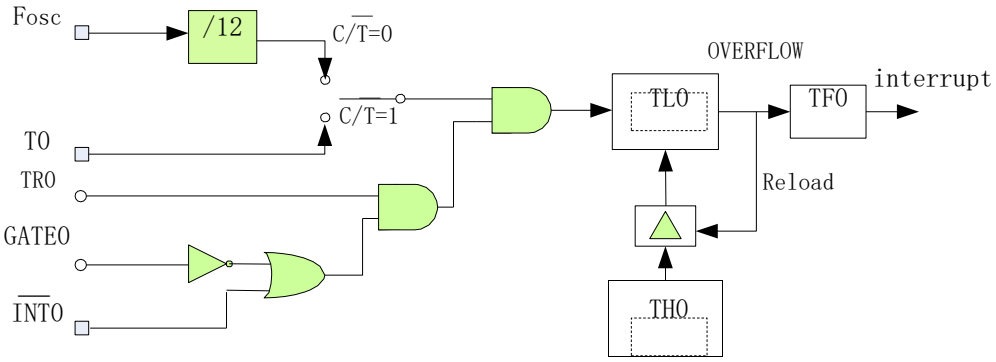


图 **12-1-1-2** 定时器 **0** 的模式 **2**

- 模式 **3**

在此模式中，TL0 和 TH0 作为两个独立的 8 位定时器/计数器。TL0 可以作为定时器或计数器，而 TH0 只能作为定时器。其中 TL0 占用定时器 0 的控制位 CT0、GATE0、TR0、TF0、INT0，而 TH0 只能占用定时器 1 的控制位 TR1、TF1。其他控制方法和模式 0、1 相同。当定时器 0 工作于模式 3 时，定时器 1 和 TH0 共用控制位 TR1，但定时器 1 由于 TF1 已被 TH0 占用，所以只能工作于不需要产生中断的场合，例如作为 UART 的波特率产生器。

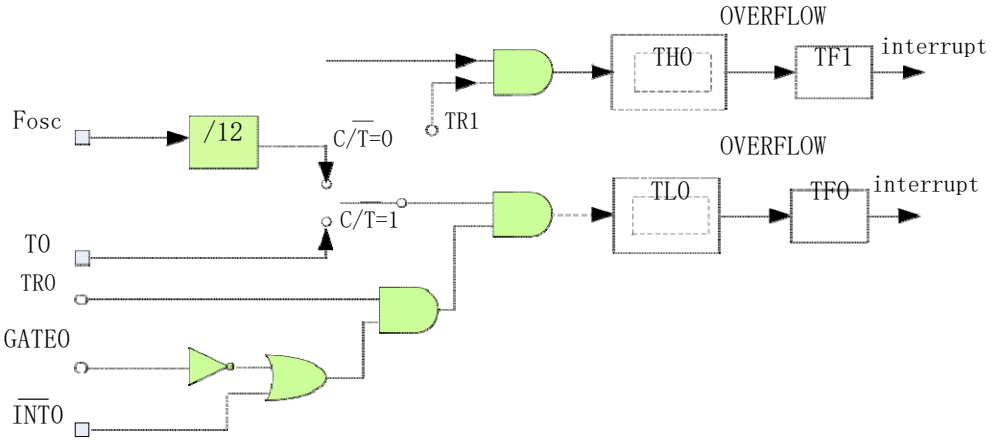


图 **12-1-1-3** 定时器 **0** 的模式 **3**

12.1.2 定时器 0 寄存器描述

表 **12-1-2-1** 寄存器 **TCON**

88H	7	6	5	4	3	2	1	0
TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7	TF1	定时器 0 模式 3 的 TH0 溢出/定时器 1 溢出标志位，中断被响应后自动清 0.						

6	TR1	定时器 1 运行控制位，1 有效
5	TF0	定时器 0 溢出标志位，中断被响应后自动清 0.
4	TR0	定时器 0 运行控制位，1 有效
3	IE1	外部中断 1 使能位，1 有效
2	IT1	外部中断 1 触发类型控制位 0：外部中断 1 在输入管脚上升沿时触发 1：外部中断 1 在输入管脚下降沿时触发
1	IE0	外部中断 0 使能位，1 有效
0	IT0	外部中断 0 触发类型控制位 0：外部中断 0 在输入管脚上升沿时触发 1：外部中断 0 在输入管脚下降沿时触发

表 **12-1-2-2** 寄存器 **TMOD**

89H	7	6	5	4	3	2	1	0
TMOD	GATE1	CT1	T1M1	T1M0	GATE0	CT0	T0M1	T0M0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	GATE1	定时器 1 门控控制位，1 有效。有效时定时器 1 由 INT1 控制开关
6	CT1	定时器 1 计数器/定时器选择位 0：定时器，时钟为系统时钟 12 分频 1：计数器，时钟为 T1 输入时钟
5	T1M1	[T1M1,T1M0]为定时器 1 模式选择位
4	T1M0	00：模式 0，TL1 和 TH1 组成 13 位定时器/计数器 01：模式 1，TL1 和 TH1 组成 16 位定时器/计数器 10：模式 2，TL1 作为 8 位定时器/计数器，TH1 作为自动重载寄存器 11：模式 3，此模式会锁住 TH1/TL1，等效于 TR1=0
3	GATE0	定时器 0 门控控制位，1 有效。有效时定时器 0 由 INT0 控制开关
2	CT0	定时器 0 计数器/定时器选择位 0：定时器，时钟为系统时钟 12 分频 1：计数器，时钟为 T0 输入时钟
1	T0M1	[T0M1,T0M0]为定时器 0 模式选择位
0	T0M0	00：模式 0，TL0 和 TH0 组成 13 位定时器/计数器 01：模式 1，TL0 和 TH0 组成 16 位定时器/计数器 10：模式 2，TL0 作为 8 位定时器/计数器，TH0 作为自动重载寄存器 11：模式 3，TL0 和 TH0 作为两个完全独立的 8 位定时器/计数器

表 **12-1-2-3** 寄存器 **TL0**

8AH	7	6	5	4	3	2	1	0
TL0	TL0							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	TL0	定时器 0 模式 0/1 计数值的低字节，模式 2/3 计数值						

表 **12-1-2-4** 寄存器 **TH0**

8CH	7	6	5	4	3	2	1	0
TH0	TH0							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	TH0	定时器 0 模式 0/1 计数值的高字节，模式 2 重载值，模式 3 计数值						

12.2 定时器 1

12.2.1 定时器 1 介绍

定时器或计数器功能通过 CT1 位（TMOD[6]）来选择，CT1=0 选择为定时器，CT1=1 选择为计数器。作为定时器时，时钟是系统时钟的 12 分频。作为计数器时，时钟是 T1 的输入时钟。由于检测 T1 输入边沿变化需要 2 个时钟周期，所以作为计数器时最大的输入波特率是内部系统时钟频率的 1/2。T1 输入信号在占空比上没有限制，然而为了完全识别 0 或 1 的状态，信号至少需要保持 1 个内部系统时钟周期时间。定时器 1 有 4 个工作模式，通过 T1M0、T1M1 位(TMOD[5:4])来选择。

- 模式 **0**
  在此模式下，定时器 1 作为 13 位定时器/计数器，TH1 存放 13 位定时器/计数器的高 8 位，TL1[4:0]存放低 5 位，而 TL1[7:5]是无效的，在读取时应被忽略。当定时器 1 溢出，中断标志位 TF1（TCON[7]）会被置 1。中断被响应后，TF1 位会自动清 0。当 GATE1（TCON[7]）=0 时，定时器/计数器由 TR1（TCON[6]）位使能计数，当 GATE1=1 时，定时器/计数器由引脚 INT1 控制使能，INT1 为高电平时计数，INT1 为低电平则停止计数。

- 模式 **1**
  在此模式下，定时器 1 作为 16 位定时器/计数器，TH1 存放 16 位定时器/计数器的高 8 位，TL1 存放低 8 位。当定时器 1 溢出，中断标志位 TF1（TCON[7]）会被置 1。中断被响应后，TF1 位会自动清 0。当 GATE1（TCON[7]）=0 时，定时器/计数器由 TR1（TCON[6]）位使能计数，当 GATE1=1 时，定时器/计数器由引

脚 INT1 控制使能，INT1 为高电平时计数，INT1 为低电平则停止计数。

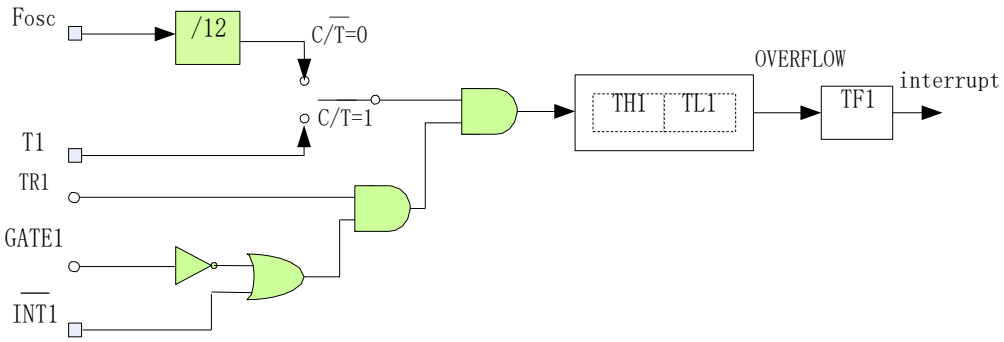


图 **12-2-1-1** 定时器 **1** 的模式 **1**

- 模式 **2**

在此模式中，定时器 1 作为 8 位自动重载定时器/计数器，只有 TL1 自动累加。当 TL1 计数溢出时，不但产生中断标志 TF1，而且从 TH1 中自动装载计数初始值到 TL1。其他设置方法和模式 0、1 相同。

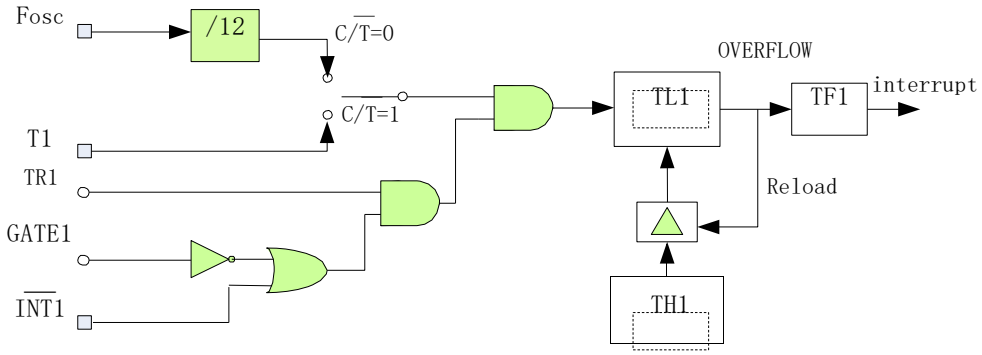


图 **12-2-1-2** 定时器 **1** 的模式 **2**

- 模式 **3**

此模式下，TH1、TL1 会被锁住，等效于 TR1=0。

12.2.2 定时器 1 寄存器描述

寄存器 TCON 和 TMOD 见表 12-1-2-1 和表 12-1-2-2。

表 **12-2-2-1** 寄存器 **TL1**

8BH	7	6	5	4	3	2	1	0
TL1	TL1							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	TL1	定时器 1 模式 0/1 计数值的低字节，模式 2/3 计数值						

表 **12-2-2-2** 寄存器 **TH1**

8DH	7	6	5	4	3	2	1	0
TH1	TH1							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	TH1	定时器 1 模式 0/1 计数值的高字节，模式 2 重载值，模式 3 计数值						

12.3 定时器 2

12.3.1 功能简介

定时器 2 是一个 16 位（TH2、TL2）的定时器/计数器。T2P0、T2P1 位可选择不同的控制方式或时钟源。当 T2P=0、3 时，选择系统时钟作为定时器 2 时钟（注意：和定时器 0、1 不同的是，时钟没有经过 12 分频）；当 T2P=0 时，定时器 2 由 TR2 位使能；当 T2P=2 时，由 T2 电平门控，T2 为高时，计数使能，T2 为低时，计数停止。当 T2P=1、2 时，选择 T2 的输入信号作为计数时钟，当 T2P=1 时，检测 T2 的下降沿计数，当 T2P=2 时，检测 T2 的上升沿。

定时器 2 可通过 T2M0、T2M1 位设置不同的工作模式。当 T2M=0 时，定时器 2 工作于定时器/计数器模式，TH2、TL2 作为 16 位计数器自动累加；在此模式下，通过设置 T2R0、T2R1 位可选择两种不同的重载模式或关闭重载功能，在重载模式下，T2CH、T2CL 存放重载值，当 T2R=2 时，定时器 2 溢出会从 T2CH、T2CL 装载计数初值到 TH2、TL2，而当 T2R=3 时，在引脚 T2EX 下降沿进行重载。当重载事件发生后，重载中断标志 RF2 置 1，如果定时器 2 中断使能会触发重载中断，RF2 通过写 1 清 0。

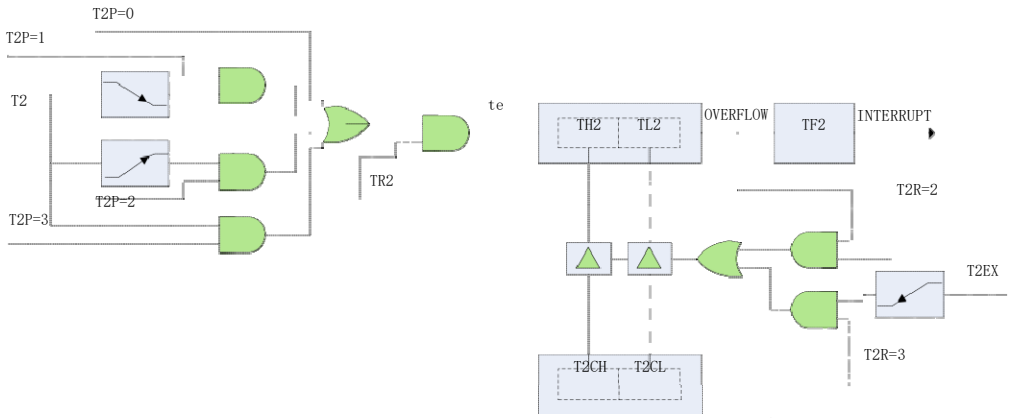


图 **12-3-1-1** 定时器 2 的重载模式

当 T2M=1 时，定时器 2 工作于比较模式，当计数值 TH2、TL2 大于 T2CH、T2CL 时，引脚 T2CP 输出高，否则 T2CP 输出低。

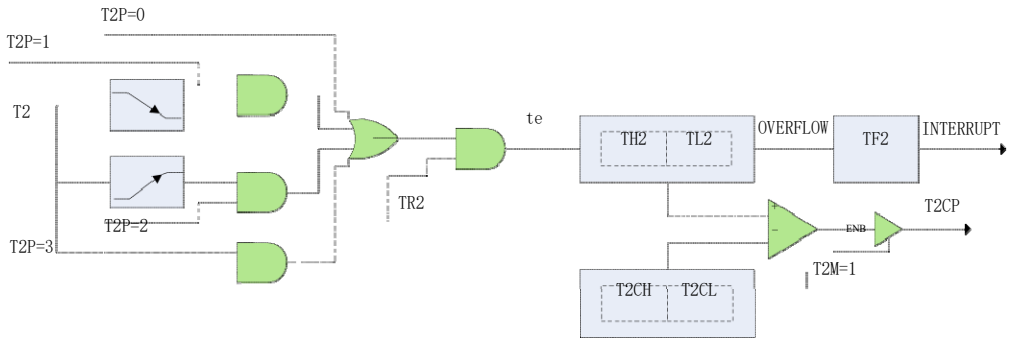


图 **12-3-1-2** 定时器 2 的比较模式

当 T2M=2 或 3 时，定时器 2 工作于抓取模式。当 T2M=2 时，当引脚 T2CP 触发沿发生时，定时器 2 的计数值 TH2、TL2 被锁存到 T2CH、T2CL，触发沿可通过 CCFG 位设置，当抓取事件产生后，抓取中断标志 CF2 置 1，如果定时器 2 中断使能会触发抓取中断，CF2 通过写 1 清 0。当 T2M=3 时，写寄存器 T2CL 将产生锁存的触发事件，而写 T2CL 的值不保存，在此模式下，抓取事件不会置位 CF2。

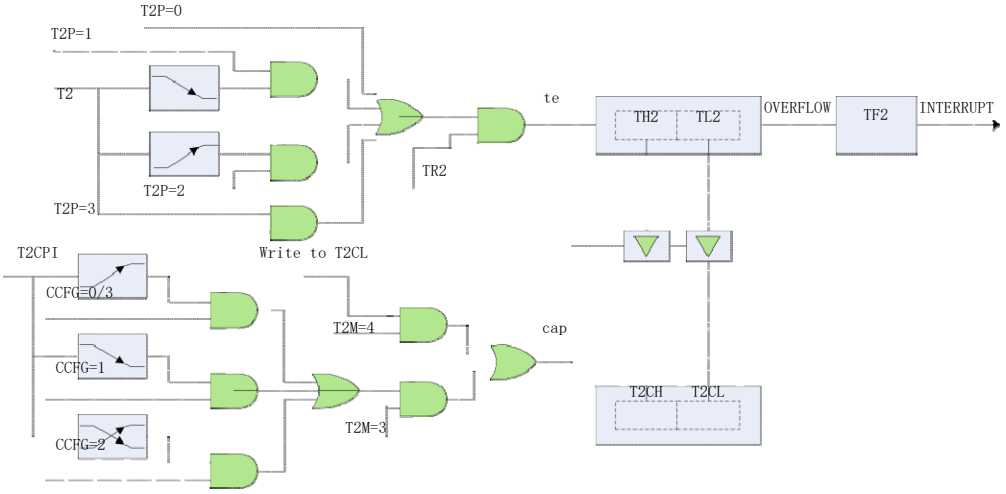


图 **12-3-1-3** 定时器 2 的抓取模式

2.3.2 定时器 2 寄存器描述

表 **12-3-2-1** 寄存器 **T2CON**

C8H	7	6	5	4	3	2	1	0
T2CON	-	TR2	T2R1	T2R0	T2IE	UCKS	T2P1	T2P0
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W

初始值	-	0	0	0	0	0	0	0

位编号	位符号	说明
7	-	-
6	TR2	定时器 2 运行控制位，1 有效
5	T2R1	[T2R1,T2R0]是定时器 2 重载模式选择位 10：模式 0 11：模式 1 其他：重载功能关闭
4	T2R0	
3	T2IE	定时器 2 中断使能位，1 有效
2	UCKS	UART0 时钟选择位 0：UART0 使用定时器 1 溢出脉冲 1：UART0 使用定时器 2 溢出脉冲
1	T2P1	[T2P1,T2P0]是定时器 2 引脚 T2 功能选择位 00：定时器 2 使用内部系统时钟计数，没有使用 T2 01：定时器 2 检测 T2 下降沿计数 10：定时器 2 检测 T2 上升沿计数 11：定时器 2 使用内部系统时钟计数，通过 T2 门控
0	T2P0	

表 **12-3-2-2** 寄存器 **T2MOD**

C9H	7	6	5	4	3	2	1	0
T2MOD	TF2	CF2	RF2	CCFG1	CCFG0	-	T2M1	T2M0
R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W
初始值	0	0	0	0	0	-	0	0

位编号	位符号	说明
7	TF2	Timer2 计数器溢出中断标志，写 1 清 0
6	CF2	抓取中断标志，写 1 清 0
5	RF2	自动重载中断标志，写 1 清 0
4	CCFG1	[CCFG1,CCFG0]抓取模式触发沿选择位，在 T2M＝3 或 T2M＝4 时有效 01：下降沿 10：上升或下降沿 其它值：上升沿
3	CCFG0	
2	-	-
1	T2M1	工作模式选择位 00：定时器/计数器模式 01：比较模式 10：抓取模式 0 11：抓取模式 1
0	T2M0	

表 **12-3-2-3** 寄存器 **T2CL**

CAH	7	6	5	4	3	2	1	0
T2CL	T2CL							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	T2CL	在重载模式，T2CL 是重载值的低字节 在比较模式，T2CL 是比较值的低字节 在抓取模式， T2CL 保存捕获值的低字节

表 **12-3-2-4** 寄存器 **T2CH**

CBH	7	6	5	4	3	2	1	0
T2CH	T2CH							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	T2CH	在重载模式，T2CH 是重载值的高字节 在比较模式，T2CH 是比较值的高字节 在捕获模式， T2CH 保存捕获值的高字节

表 **12-3-2-5** 寄存器 **TL2**

CCH	7	6	5	4	3	2	1	0
TL2	TL2							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	TL2	定时器 2 计数值的低字节

表 **12-3-2-6** 寄存器 **TH2**

CDH	7	6	5	4	3	2	1	0
TH2	TH2							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	TH2	定时器 2 计数值的高字节

13 看门狗定时器（WDT）

13.1 看门狗定时器(WDT)功能简介

看门狗定时器是一个可选时钟源的 27 位减法计数器，时钟为时钟为 3.6864MHz 下计数时间范围为 0.56ms - 36.4s，有 16 位调节精度。看门狗主要用于监控系统，避免 CPU 因为外界干扰出现死机。如果软件不能在溢出前刷新看门狗定时器，看门狗将产生内部复位或者中断。写 A5H 到寄存器 WDFLG 将刷新看门狗，读 WDFLG 可得到看门狗状态。在 STOP 模式下，如果看门狗处于使能状态，则看门狗所选的时钟源正常工作，此时如果看门狗设为中断，看门狗中断可唤醒 CPU。

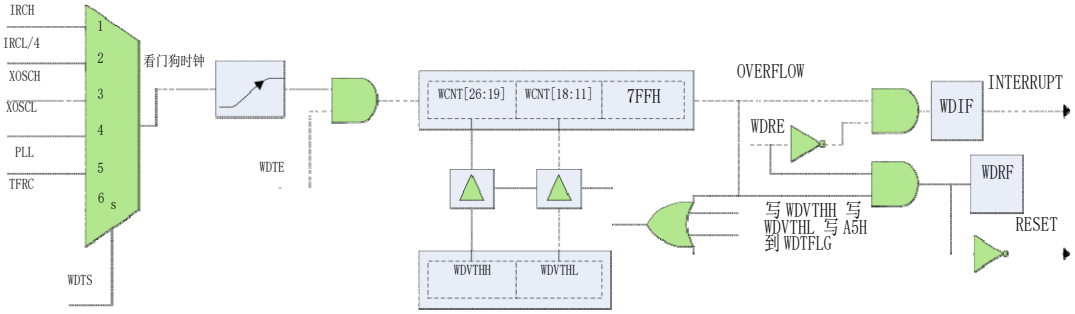


图 **13-1-1** 看门狗模块结构图

13.2 看门狗定时器(WDT)寄存器描述

表 **13-2-1** 寄存器 **WDCON**

AAH	7	6	5	4	3	2	1	0
WDCON	WDTS[2:0]			-	-	-	-	WDRE
R/W	R/W	R/W	R/W	-	-	-	-	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~5	WDTS	WDT 时钟选择位 001：选择 IRCH 010：选择 IRCL 四分频 011：选择 ERC 100：选择 XOSCL 101：选择 PLL 110：选择 TFRC 其他：WDT 关闭
4~1	-	
0	WDRE	WDT 功能选择位 0：WDT 溢出后产生中断 1：WDT 溢出后产生复位

表 **13-2-2** 寄存器 **WDFLG**

ABH	7	6	5	4	3	2	1	0
WDFLG							WDIF	WDRF
R/W	-						R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~2	-	-
1	WDIF	WDT 中断标志，写 A5H 时将清除该标志
0	WDRF	WDT 复位标志，写 A5H 时将清除该标志

表 **13-2-3** 寄存器 **WDVTHL**、**WDVTHH**

ACH	7	6	5	4	3	2	1	0
WDVTHL	WDVTH[7:0]							
R/W	R/W							
初始值	1	1	1	1	1	1	1	1
ADH	7	6	5	4	3	2	1	0
WDVTHH	WDVTH[15:8]							
R/W	R/W							
初始值	1	1	1	1	1	1	1	1

位编号	位符号	说明
15~0	WDVTH	WDT 阈值设置寄存器，计算公式如下： WDT 触发时间＝ (WDVTH * 800H+7FFH) * clock cycle 当看门狗时钟为 3.6864M 时，覆盖范围为 0.56ms~ 36s

13.3 看门狗定时器控制例程

◆ 看门狗中断模式例程

例如，看门狗时钟设置为 IRCH，IRCH 的频率为 3.6864MHz，看门狗设置为中断模式，溢出时间为 1 秒，程序如下：

```
---------------------------------------------------------------------------------
#define WDTS_IRCH           (1<<5)
#define WDRE_reset          (1<<0)
#define WDRE_int            (0<<0)
void WDT_init(void)
{
    WDCON = WDTS_IRCH | WDRE_int;       //设置看门时钟为 IRCH, 看门狗中断模式
    WDVTHH = 0x07;                      //设置看门狗时间为 1 秒
    WDVTHL = 0x08;
    WDFLG = 0xA5;                       //刷新看门狗
}
void WDT_ISR (void) interrupt 12
{
    if(WDFLG & 0x02)
    {
        //看门狗中断服务程序
        WDFLG = 0xA5;//刷新看门狗
    }
}
---------------------------------------------------------------------------------
```

◆ 看门狗复位模式例程

例如，看门狗时钟设置为 IRCH，IRCH 的频率为 3.6864MHz，看门狗设置为复位模式，溢出时间为 1 秒，程序如下：

```
---------------------------------------------------------------------------------
#define WDTS_IRCH           (1<<5)
#define WDRE_reset          (1<<0)
#define WDRE_int        (0<<0)
void WDT_init(void)
{
    WDCON = WDTS_IRCH | WDRE_reset;     //设置看门时钟为 IRCH, 看门狗复位模式
    WDVTHH = 0x07;                      //设置看门狗时间为 1 秒
    WDVTHL = 0x08;
    WDFLG = 0xA5;                       //刷新看门狗
}
---------------------------------------------------------------------------------
```

14 实时定时器（RTC）

14.1 RTC 功能简介

内置 RTC 是一个实时时钟模块，主要时钟源是外部 32.768KHz 晶体振荡器，它包含毫秒、秒、分、时、天和星期寄存器，同时，还内置了闹钟功能，当 RTC 时间和设定的闹钟时间匹配时，会产生中断，这对于包含时钟和闹钟功能的产品来说特别方便。另外，RTC 还可以设置毫秒级中断、半秒中断，其中，毫秒中断中断时间可设置。RTC 在不外挂 32.768KHz 晶振的情况下，也可以设置 IRCL 的四分频作为 RTC 的时钟源，应用于对计时精度要求不高的场合。在 STOP/IDLE 模式，RTC 也可以开启并作为 STOP/IDLE 模式唤醒的触发源。RTC 结构图如图 14-1-1 所示。

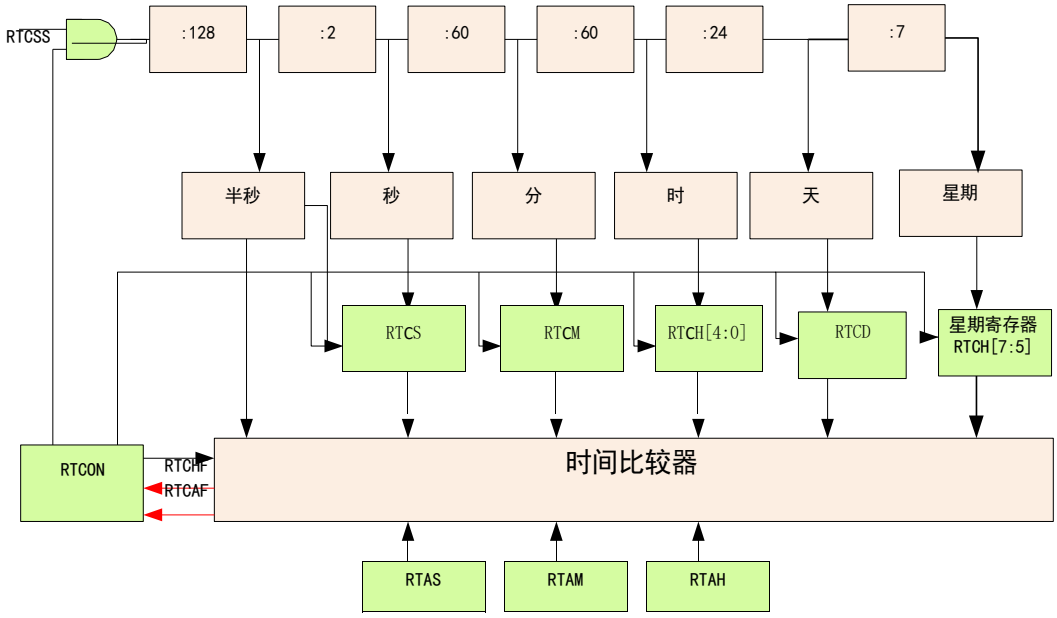


图 **14-1-1　RTC** 结构图

- **RTC** 使能和关闭

RTC 使能和关闭由 RTCE 位（RTCON[7]）控制。设置 RTCE=1 后，RTC 开始计时；设置 RTCE=0 后，RTC 模块所有寄存器的状态都会被锁存。RTC 使能后，需要等待 300us 才能写 RTC 时间寄存器，否则写入值无效。值得注意的是，由于 RTC 的时钟源主要是外挂 32.768KHz 晶振，必须要等待 32.768KHz 晶振正常起振后再使能 RTC 模式，否则可能会操作无效。

- **RTC** 寄存器读写

RTC 寄存器（RTCS、RTCM、RTCH、RTCDL、RTCDH）写入由 RTCWE 位（RTCON[1]）控制。RTCWE 置 1 后，需要等待 50us 才能改写 RTC 寄存器，改写后等待 50us 后 RTCWE 才变为 0。写一个非法时间（超过了有效秒、分或时范围的值）到 RTC 寄存器将会被当作该寄存器的最大有效值写入。在写秒、分、时和星期时，微秒寄存器 RTCSS 会被清零，从而能精确的计时。RTC 寄存器可直接进行读取。

- **RTC** 闹钟功能

当 RTC 时间和闹钟时间匹配时，将产生闹钟中断，标志位为 RTCAF。用户可以通过寄存器 RTAS、RTAM、RTAH 来设置闹铃时间，不需要设置 RTCWE 位。写一个非法的值(超过了有效秒、分或时范围的值)将会当作该寄存器最大有效值来写入。用户可以设置相应的比较使能位（HCE、MCE、SCE）去比较 RTAS、RTAM 和 RTAH 寄存器的一个或多个值。如果相应比较使能位设为 0，相应的时间项比较会被忽略（例如设置 HCE=1、MCE=0、SCE=1，只会比较小时、秒寄存器，分寄存器会认为已匹配）。这样就允许闹铃可以在一天的特定时间发生一次（所有的比较位使能），或者周期性的一每秒一次、每分钟一次、每小时一次（通过比较使能位的组合设置来实现）。

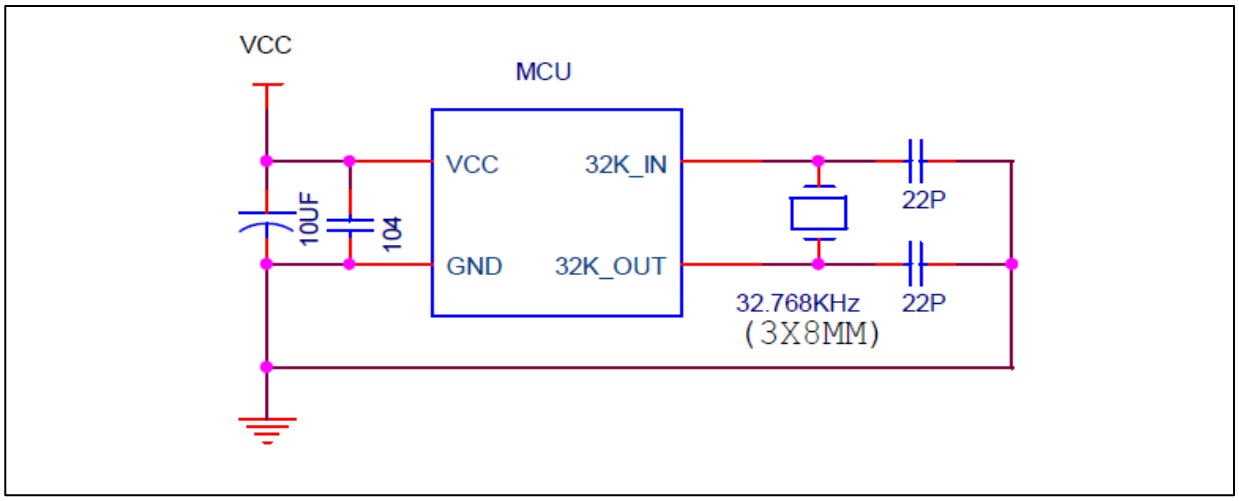


图 **14-1-2 XOSCL** 典型电路图

重要提醒：*1.* 硬件设计时晶振负载电容地必须和芯片地连接，晶振补偿电容尽量靠近芯片 *GND* 引脚。*32.768KHz* 石英晶振要求使用直径 *3mmx8mm* 的晶振规格。

*2.*以上电路及元件参数仅供参考，使用不同厂家晶振在电路使用中参数可能需要修改。

14.2 RTC 寄存器描述

表 **14-2-1** 寄存器 **RTCON**

F1H	7	6	5	4	3	2	1	0
RTCON	RTCE	MSE	HSE	SCE	MCE	HCE	RTCWE	-
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-
初始值	0	0	0	0	0	0	0	-

位编号	位符号	说明
7	RTCE	RTC 时钟使能，1 有效
6	MSE	毫秒中断使能信号，1 有效
5	HSE	半秒中断使能信号，1 有效

4	SCE	闹钟秒比较使能，1 有效
3	MCE	闹钟分钟比较使能，1 有效
2	HCE	闹钟小时比较使能，1 有效
1	RTCWE	时钟写使能，1 有效
0	-	-

表 **14-2-2** 寄存器 **RTCSS**

E9H	7	6	5	4	3	2	1	0
RTCSS	RTCSS[7:0]							
R/W	R							
初始值	0	0	0	0	0	0	0	-
位编号	位符号	说明						
7~0	RTCE	RTC 微秒计数器，每 1/256 秒加 1						

表 **14-2-3** 寄存器 **RTCS**

F2H	7	6	5	4	3	2	1	0
RTCS	-	-	RTCS[5:0]					
R/W	-	-	R/W					
初始值	-	-	0	0	0	0	0	0
位编号	位符号	说明						
7~6	-	-						
5~0	RTCS	秒计数器，每 1 秒加 1，计数范围为 0~59						

表 **14-2-4** 寄存器 **RTCM**

F3H	7	6	5	4	3	2	1	0
RTCM	-	-	RTCM[5:0]					
R/W	-	-	R/W					
初始值	-	-	0	0	0	0	0	0
位编号	位符号	说明						
7~6	-	-						
5~0	RTCM	分计数器，每分钟加 1，计数范围为 0~59						

表 **14-2-5** 寄存器 **RTCH**

F4H	7	6	5	4	3	2	1	0
RTCH	RTCW[2:0]			RTCH[4:0]				
R/W	R/W			R/W				
初始值	x	x	x	0	0	0	0	0

位编号	位符号	说明
7~5	RTCW	星期计数器，计数范围为 1~7，代表星期一到星期日，当设置为 0 时，星期计数功能关闭 备注：上电复位后 *RTCW* 为不确定值，在应用之前必须先写入初值。
4~0	RTCH	小时计数器，每小时该加 1，计数范围为 0~23

表 **14-2-6** 寄存器 **RTCDL**、**RTCDH**

F5H	7	6	5	4	3	2	1	0
RTCDL	RTCD[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
F6H	7	6	5	4	3	2	1	0
RTCDH	RTCD[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
15~0	RTCD	天数计数器，每天计数加 1

表 **14-2-7** 寄存器 **RTAS**

EAH	7	6	5	4	3	2	1	0
RTAS	-	-	RTAS[5:0]					
R/W	-	-	R/W					
初始值	-	-	0	0	0	0	0	0

位编号	位符号	说明
7~6	-	-
5~0	RTAS	闹钟秒值设定，取值范围为 0~59

表 **14-2-8** 寄存器 **RTAM**

EBH	7	6	5	4	3	2	1	0
RTAM	-	-	RTAM[5:0]					
R/W	-	-	R/W					
初始值	-	-	0	0	0	0	0	0
位编号	位符号	说明						
7~6	-	-						
5~0	RTAM	闹钟分钟值设定，取值范围为 0~59						

表 **14-2-9** 寄存器 **RTAH**

ECH	7	6	5	4	3	2	1	0
RTAH	-	-	-	RTAH[4:0]				
R/W	-	-	-	R/W				
初始值	-	-	-	0	0	0	0	0
位编号	位符号	说明						
7~6	-	-						
5~0	RTAH	闹钟小时值设定，取值范围为 0~23						

表 **14-2-10** 寄存器 **RTMSS**

EDH	7	6	5	4	3	2	1	0
RTMSS	RTMSS[7:0]							
R/W	R/W							
初始值	0	0	0-	0	0	0	0	0
位编号	位符号	说明						
7~0	RTMSS	RTC 毫秒中断阈值寄存器，毫秒中断时间=（RTMSS+1）*128*RTC 时钟周期。如果 RTC 时钟是 32.768KHz，那设置的时间单位是 128*（1/32.768）=3.90625ms。						

表 **14-2-11** 寄存器 **RTCIF**

EEH	7	6	5	4	3	2	1	0
RTCIF	-	-	-	-	-	RTCMF	RTCHF	RTCAF
R/W	-	-	-	-	-	R	R	R
初始值	-	-	-	-	-	0	0	0
位编号	位符号	说明						
7~3	-	-						

2	RTCMF	RTC 毫秒中断标志，写 1 清 0
1	RTCHF	RTC 半秒中断标志，写 1 清 0
0	RTCAF	RTC 闹钟中断标志，写 1 清 0

14.3 RTC 控制例程

- **RTC** 写入时间 写时、分、秒程序如下：

```
--------------------------------------------------------------------------------
#define RTCE          (1<<7)
void RTC_WriteHour(unsigned char hour)     //hour=0~23
{
    RTCON |= RTCWE;         //写入时间使能
    RTCH = hour;            //写入小时值
    Delay_50us();           //必须延时 50 微秒
    RTCON &= ~RTCWE;        //写入时间禁能
}
void RTC_WriteMinute(unsigned char minute)//minute=0~59
{
    RTCON |= RTCWE;         //写入时间使能
    RTCM = minute;          //写入分值
    Delay_50us();           //必须延时 50 微秒
    RTCON &= ~RTCWE;        //写入时间禁能
}
void RTC_WriteSecond(unsigned char second)//second=0~59
{
    RTCON |= RTCWE;         //写入时间使能
    RTCS = second;          //写入秒值
    Delay_50us();           //必须延时 50 微秒
    RTCON &= ~RTCWE;        //写入时间禁能
}
--------------------------------------------------------------------------------
```

- **RTC** 设置闹钟时间

例如，设置闹钟时间为 11:30:0，时、分、秒比较全使能，程序如下：

```
--------------------------------------------------------------------------------
#define SCE(N)          (N<<4)    //N=0~1
#define MCE(N)          (N<<3)    //N=0~1
#define HCE(N)          (N<<2)    //N=0~1
#define RTC_AF          (1<<0)
Void RTM_init(void)
{
    RTAH = 11;          //设置闹钟小时
```

```
    RTAM = 30;          //设置闹钟分
    RTAS = 0;           //设置闹钟秒
    RTCON |= SCE(1)|MCE(1)|HCE(1);      //时、分、秒比较使能
}
void RTC_ISR (void) interrupt 13
{
    if(RTCIF & RTC_AF)          //闹钟中断
    {
        RTCIF = RTC_AF;
        //闹钟中断服务程序

    }
    ......
}
```

◆ **RTC** 初始化

RTC 初始化程序如下：

```
#define XLCKE       (1<<3)
#define XLSTA       (1<<2)
void RTC_init(void)
{
    CKCON |= XLCKE;                     //开启 XOSCL 时钟
    while(!(CKCON & XLSTA));            //等待 XOSCL 时钟稳定
    RTCON = RTCE(1) | MSE(1) | HSE(1);  //RTC 使能，毫秒中断使能，半秒中断使能
    RTC_WriteHour(10);                  //写入小时
    RTC_WriteMinute(30);                //写入分
    RTC_WriteSecond(0);                 //写入秒
    RTM_init();                         //设置闹钟
    RTMSS = 0;                          //设置毫秒中断时间
    INT8EN = 1;                         //开启 RTC 中断
}
void RTC_ISR (void) interrupt 13
{
    if(RTCIF & RTC_MF)          //毫秒中断
    {
        RTCIF = RTC_MF;
        //毫秒中断服务程序

    }
    if(RTCIF & RTC_HF)          //半秒中断
    {
        RTCIF = RTC_HF;
        //半秒中断服务程序
```

```
    }
    if(RTCIF & RTC_AF)              //闹钟中断
    {
        RTCIF = RTC_AF;
        //闹钟中断服务程序

    }
}
```

---

15 通用输入输出口（GPIO）及复用定义

15.1 功能简介

通用输入/输出口用于芯片和外部进行数据传输，MJ603 系列芯片最大封装有 26 个 I/O 引脚，每个 I/O 引脚都是复用功能引脚，不仅能独立编程为输入/输出口，而且还能设置为其他功能引脚。每个引脚都分配了功能设置寄存器 PnxF 和 PnxC（分别对应引脚 Pnx，其中 n=0~7, 代表 P0~P3，x=0~7, 代表 Pn.0~Pn.7），用户可通过寄存器 PnxF 和 PnxC 配置引脚的主功能和其他选项。

每个 I/O 可通过 PnxPUP/PnxPDP(PnxF[7]/PnxF[6]) 使能上/下拉电阻，强/弱上/下拉电阻由 PU_SEL/PD_SEL(PnxC[5]/PnxC[4])选择，当 PU_SEL/PD_SEL 为 1 时，选择为强上/下拉，否则为弱上/下拉，默认为强上/下拉。

当 I/O 设置为输出模式时，当 PnxOPR(PnxF[5])为 1 时，I/O 为开漏输出模式。

I/O 为推挽输出时，通过 DRV(PnxC[3:2])可设置 IO 推挽输出驱动强度，通过 SR(PnxC[1:0])可设置 IO 输出翻转斜率，当 I/O 输出电平翻转时，由于电感效应，会在 I/O 端口产生过冲信号，此过冲信号对芯片系统有可能造成一定影响，降低 I/O 输出强度及 I/O 速度可有效降低过冲信号幅度，在应用时，此两项参数可灵活配置。

I/O 为输入模式时，可通过 SMIT_EN(PnxC[6])位选择斯密特模式或反相器模式，选择为反相器模式时，高低电平的触发门限值为 1/2 VDD，默认为斯密特模式。

另外，P00~P04 可复用为 LED COM 引脚，可通过 SINK_EN(P0xC[7])设置为高灌电平模式，在高灌电流模式下，灌电流大于 60mA（测试条件可参考电气特性章节说明）。

GPIO 的主要特性如下：

- 可配置为高阻模式
- I/O 结构可独立设置强上拉、弱上拉、强下拉、弱下拉电阻
- 输出模式可选开漏输出或推挽输出
- 数据输出锁存支持读-修改-写
- 支持 1.8~5.5V 宽电压范围
- 设为推挽输出时，可独立设置 IO 驱动强度
- 设为推挽输出时，可独立设置 IO 输出速度
- LED COM 可设置高灌电流，灌电流可达 60mA（测试条件可参考电气特性章节说明）

GPIO 推挽模式结构图如图 15-1-1 所示。

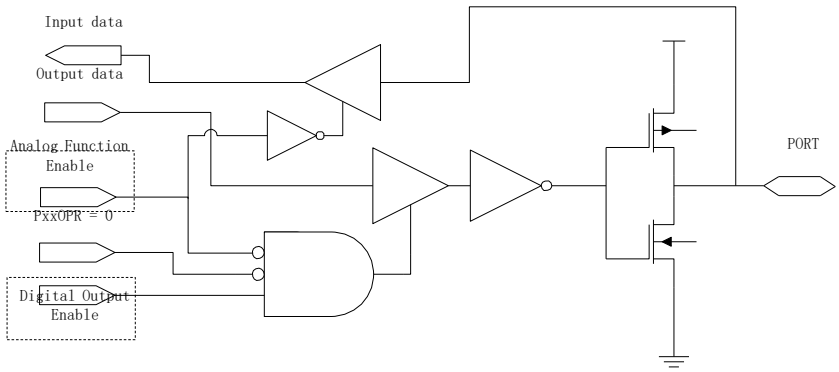


图 **15-1-1 I/O** 推挽模式结构示意图

GPIO 开漏模式结构图如图 15-1-2 所示。

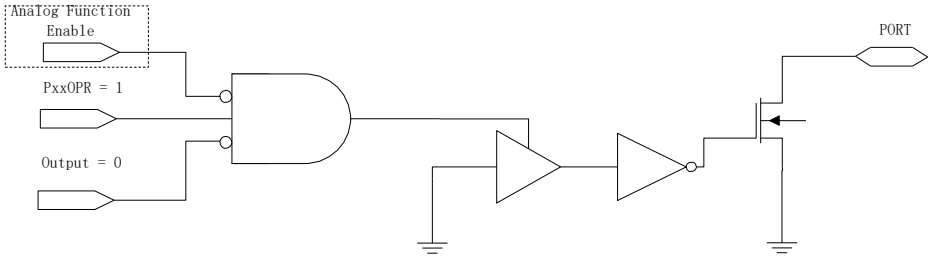


图 **15-1-2 I/O** 开漏模式结构示意图

GPIO 下拉结构图如图 15-1-3 所示。

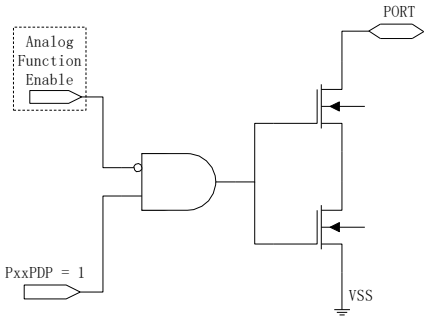


图 **15-1-3 I/O** 下拉模式结构示意图

GPIO 上拉结构图如图 15-1-4 所示。

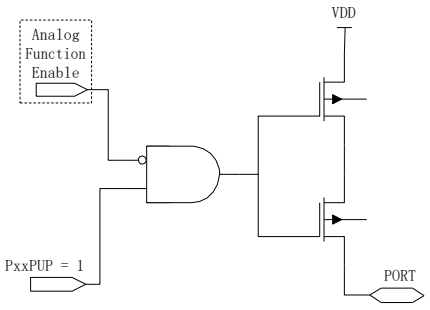


图 **15-1-4 I/O** 上拉模式结构示意图

15.2 引脚寄存器描述

表 **15-2-1** 寄存器 **P0**

80H	7	6	5	4	3	2	1	0
P0	P07	P06	P05	P04	P03	P02	P01	P00
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	P0x	引脚 P0x 的数据寄存器，管脚功能设置为 GPIO 时有效 0：设为输入时 P0x 电平为低，设为输出时 P0x 输出低电平 1：设为输入时 P0x 电平为高，设为输出时 P0x 输出高电平

表 **15-2-2** 寄存器 **P1**

90H	7	6	5	4	3	2	1	0
P1	P17	P16	P15	P14	P13	P12	P11	P10
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	P1x	引脚 P1x 的数据寄存器，管脚功能设置为 GPIO 时有效 0：设为输入时 P1x 电平为低，设为输出时 P1x 输出低电平 1：设为输入时 P1x 电平为高，设为输出时 P1x 输出高电平

表 **15-2-3** 寄存器 **P2**

A0H	7	6	5	4	3	2	1	0
P2	-	-	-	-	-	-	P21	P20
R/W	-	-	-	-	-	-	R/W	R/W
初始值	-	-	-	-	-	-	0	0

位编号	位符号	说明
1~0	P2x	引脚 P2x 的数据寄存器，管脚功能设置为 GPIO 时有效 0：设为输入时 P2x 电平为低，设为输出时 P2x 输出低电平 1：设为输入时 P2x 电平为高，设为输出时 P2x 输出高电平

表 **15-2-4** 寄存器 **P3**

B0H	7	6	5	4	3	2	1	0
P3	P37	P36	P35	P34	P33	P32	P31	P30
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
位编号	位符号	说明						
7~0	P3x	引脚 P3x 的数据寄存器，管脚功能设置为 GPIO 时有效 0：设为输入时 P3x 电平为低，设为输出时 P3x 输出低电平 1：设为输入时 P3x 电平为高，设为输出时 P3x 输出高电平						

表 **15-2-5** 引脚功能控制寄存器

8000H	7	6	5	4	3	2	1	0
P00F	P00PUP	P00PDP	P00OPR	-	-	P00S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8001H	7	6	5	4	3	2	1	0
P01F	P01PUP	P01PDP	P01OPR	-	-	P01S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8002H	7	6	5	4	3	2	1	0
P02F	P02PUP	P02PDP	P02OPR	-	-	P02S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8003H	7	6	5	4	3	2	1	0
P03F	P03PUP	P03PDP	P03OPR	-	-	P03S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8004H	7	6	5	4	3	2	1	0
P04F	P04PUP	P04PDP	P04OPR	-	-	P04S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8005H	7	6	5	4	3	2	1	0
P05F	P05PUP	P05PDP	P05OPR	-	-	P05S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0

8006H	7	6	5	4	3	2	1	0
P06F	P06PUP	P06PDP	P06OPR	-	-	P06S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8007H	7	6	5	4	3	2	1	0
P07F	P07PUP	P07PDP	P07OPR	-	-	P07S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8008H	7	6	5	4	3	2	1	0
P10F	P10PUP	P10PDP	P10OPR	-	-	P10S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8009H	7	6	5	4	3	2	1	0
P11F	P11PUP	P11PDP	P11OPR	-	-	P11S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	1	1	0
800AH	7	6	5	4	3	2	1	0
P12F	P12PUP	P12PDP	P12OPR	-	-	P12S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	1	1	0
800BH	7	6	5	4	3	2	1	0
P13F	P13PUP	P13PDP	P13OPR	-	-	P13S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
800CH	7	6	5	4	3	2	1	0
P14F	P14PUP	P14PDP	P14OPR	-	-	P14S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
800DH	7	6	5	4	3	2	1	0
P15F	P15PUP	P15PDP	P15OPR	-	-	P15S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
800EH	7	6	5	4	3	2	1	0
P16F	P16PUP	P16PDP	P16OPR	-	-	P16S		

R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
800FH	7	6	5	4	3	2	1	0
P17F	P17PUP	P17PDP	P17OPR	-	-	P17S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8010H	7	6	5	4	3	2	1	0
P20F	P20PUP	P20PDP	P20OPR	-	-	P20S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	1	1
8011H	7	6	5	4	3	2	1	0
P21F	P21PUP	P21PDP	P21OPR	-	-	P21S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
8018H	7	6	5	4	3	2	1	0
P30F	P30PUP	P30PDP	P30OPR	-	-	P30S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	1	0	0
8019H	7	6	5	4	3	2	1	0
P31F	P31PUP	P31PDP	P31OPR	-	-	P31S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	1	0	0
801AH	7	6	5	4	3	2	1	0
P32F	P32PUP	P32PDP	P32OPR	-	-	P32S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
801BH	7	6	5	4	3	2	1	0
P33F	P33PUP	P33PDP	P33OPR	-	-	P33S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
801CH	7	6	5	4	3	2	1	0
P34F	P34PUP	P34PDP	P34OPR	-	-	P34S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0

801DH	7	6	5	4	3	2	1	0
P35F	P35PUP	P35PDP	P35OPR	-	-	P35S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
801EH	7	6	5	4	3	2	1	0
P36F	P36PUP	P36PDP	P36OPR	-	-	P36S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0
801FH	7	6	5	4	3	2	1	0
P37F	P37PUP	P37PDP	P37OPR	-	-	P37S		
R/W	R/W	R/W	R/W	-	-	R/W		
初始值	0	0	0	-	-	0	0	0

位编号	位符号	说明
7	PnxPUP	上拉电阻使能控制位 0: 上拉电阻关闭 1: 上拉电阻打开
6	PnxPDP	下拉电阻使能控制位 0: 下拉电阻关闭 1: 下拉电阻打开
5	PnxOPR	开漏使能控制位，引脚设为数字输出时才有效 0: 开漏关闭 1: 开漏打开

备注：*Pnx → n=0~3,* 代表 *P0~P3*

x=0~7, 代表 *Pn.0~Pn.7*

表 **15-2-6** 引脚控制寄存器 **PxnC**

8120H	7	6	5	4	3	2	1	0
P00C	SINK_EN	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	R/W	R/W	R/W	R/W	R/W		R/W	
初始值	0	1	1	1	1	1	1	1
8121H	7	6	5	4	3	2	1	0
P01C	SINK_EN	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	R/W	R/W	R/W	R/W	R/W		R/W	
初始值	0	1	1	1	1	1	1	1
8122H	7	6	5	4	3	2	1	0
P02C	SINK_EN	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	R/W	R/W	R/W	R/W	R/W		R/W	

初始值	0	1	1	1	1	1	1	1
8123H	7	6	5	4	3	2	1	0
P03C	SINK_EN	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	R/W	R/W	R/W	R/W	R/W		R/W	
初始值	0	1	1	1	1	1	1	1
8124H	7	6	5	4	3	2	1	0
P04C	SINK_EN	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	R/W	R/W	R/W	R/W	R/W		R/W	
初始值	0	1	1	1	1	1	1	1
8125H	7	6	5	4	3	2	1	0
P05C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8126H	7	6	5	4	3	2	1	0
P06C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8127H	7	6	5	4	3	2	1	0
P07C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8128H	7	6	5	4	3	2	1	0
P10C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8129H	7	6	5	4	3	2	1	0
P11C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
812AH	7	6	5	4	3	2	1	0
P12C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
812BH	7	6	5	4	3	2	1	0
P13C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
812CH	7	6	5	4	3	2	1	0
P14C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
812DH	7	6	5	4	3	2	1	0

P15C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
812EH	7	6	5	4	3	2	1	0
P16C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
812FH	7	6	5	4	3	2	1	0
P17C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8130H	7	6	5	4	3	2	1	0
P20C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8131H	7	6	5	4	3	2	1	0
P21C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8138H	7	6	5	4	3	2	1	0
P30C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
8139H	7	6	5	4	3	2	1	0
P31C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
813AH	7	6	5	4	3	2	1	0
P32C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
813BH	7	6	5	4	3	2	1	0
P33C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
813CH	7	6	5	4	3	2	1	0
P34C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
813DH	7	6	5	4	3	2	1	0
P35C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	

R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
813EH	7	6	5	4	3	2	1	0
P36C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1
813FH	7	6	5	4	3	2	1	0
P37C	-	SMIT_EN	PU_SEL	PD_SEL	DRV[1:0]		SR[1:0]	
R/W	-	R/W	R/W	R/W	R/W		R/W	
初始值	-	1	1	1	1	1	1	1

备注：

1. PxnC 代表 *P00C~P37C*，其中 *x=0,1,2,3* 代表 *P0~P3*，*n=0,1,2,…,7(*当 *X=2* 时 *n=0,1)*。

2. SINK_EN 位只存在于具备 *LED COM* 功能的 *IO* 的控制寄存器上。

位编号	位符号	说明
7	SINK_EN	高灌电流使能位，1 有效 备注：*IO* 必须设置为推挽输出模式
6	SMIT_EN	IO为输入功能时模式选择位 0：反相器模式 1： SMIT 模式
5	PU_SEL	上拉电阻选择位 0：弱上拉（上拉电阻为 45K） 1：强上拉（上拉电阻为 10K）
4	PD_SEL	下拉电阻选择位 0：弱下拉（下拉电阻为 45K） 1：强下拉（下拉电阻为 15K）
3~2	DRV	输出强度选择位，范围：0~3，数值越大，驱动能力越强
1~0	SR	输出斜率控制位，范围：0~3，数值越大，IO 翻转斜率越高（速度越快）

表 **15-2-7** 引脚复用功能映射表

取值 名称	0	1	2	3	4	5	6	7
P00S	高阻	数字输入	数字输出	COM0	TK19/LED[19]	ERC	高阻	高阻
P01S	高阻	数字输入	数字输出	COM1	TK18/LED[18]	高阻	高阻	高阻
P02S	高阻	数字输入	数字输出	COM2	TK17/LED[17]	高阻	高阻	高阻
P03S	高阻	数字输入	数字输出	COM3	TK16/LED[16]	SEG9	高阻	高阻
P04S	高阻	数字输入	数字输出	COM4	TK15/LED[15]	SEG8	高阻	高阻
P05S	高阻	数字输入	数字输出	SEG7	TK14/LED[14]	高阻	高阻	高阻
P06S	高阻	数字输入	数字输出	SEG6	TK13/[13]	高阻	PWM5	高阻
P07S	高阻	数字输入	数字输出	SEG5	TK12/LED[12]	高阻	PWM4	高阻

P10S	高阻	数字输入	数字输出	高阻	TK0/LED[0]	ADC_CH0/ADC_VREF	高阻	高阻
P11S	高阻	数字输入	数字输出	UART1_TX	TK1/LED[1]	ADC_CH1	I2C_SDA	
P12S	高阻	数字输入	数字输出	UART1_RX	TK2/LED[2]	ADC_CH2	I2C_SCL	高阻
P13S	高阻	数字输入/T0	数字输出	高阻	TK3/LED[3]	ADC_CH3	高阻	高阻
P14S	高阻	数字输入/T1	数字输出	高阻	TK4/LED[4]	ADC_CH4	高阻	高阻
P15S	高阻	数字输入/T2	数字输出	高阻	TK5/LED[5]	ADC_CH5	PWM0	高阻
P16S	高阻	数字输入	数字输出	高阻	TK6/LED[6]	ADC_CH6	PWM1	高阻
P17S	高阻	数字输入	数字输出	SEG0	TK7/LED[7]	ADC_CH7	高阻	高阻
P20S	高阻	数字输入	数字输出	RESET	-	-	-	-
P21S	高阻	数字输入	数字输出	TK_CAP	-	-	-	-
P30S	高阻	数字输入	数字输出	UART0_TX	IIC_SDA	高阻	高阻	高阻
P31S	高阻	数字输入	数字输出	UART0_RX	IIC_SCL	高阻	高阻	高阻
P32S	高阻	数字输入	数字输出	32K_XIN	-	-	-	-
P33S	高阻	数字输入	数字输出	32K_XOUT	-	-	-	-
P34S	高阻	数字输入	数字输出	SEG4	TK11/LED[11]	高阻	PWM3	高阻
P35S	高阻	数字输入	数字输出	SEG3	TK10/LED[10]	高阻	PWM2	高阻
P36S	高阻	数字输入	数字输出	SEG2	TK9/LED[9]	T2CP	高阻	高阻
P37S	高阻	数字输入/T2EX	数字输出	SEG1	TK8/LED[8]	高阻	高阻	高阻

15.3 GPIO 引脚控制例程

- 引脚功能设置 例如，P20 设置为推挽输出，程序如下：

```
--------------------------------------------------------------------------------
P20F = 2;
--------------------------------------------------------------------------------
```

P20 设置为开漏输出，程序如下：

```
--------------------------------------------------------------------------------
P20F = (1<<5)|2;
--------------------------------------------------------------------------------
```

P20 设置为开漏输出，并且打开强上拉，程序如下：

```
--------------------------------------------------------------------------------
P20C |= (1<<5);
P20F = (1<<7)｜(1<<5)｜2;
--------------------------------------------------------------------------------
```

P20 设置为输入功能，并且打开弱上拉，程序如下：

```
---------------------------------------------------------------------------------------------
P20C &= ~(1<<5);
P20F = (1<<7)｜1;
---------------------------------------------------------------------------------------------
```

P20 设置为 LCD/LED SEG31，程序如下：

```
---------------------------------------------------------------------------------------------
P20F = 3;
---------------------------------------------------------------------------------------------
```

P00 设置为 LED COM，并使能高灌电流模式，程序如下：

```
---------------------------------------------------------------------------------------------
P00C |= (1<<7);
P00F = 3;
---------------------------------------------------------------------------------------------
```

16 通用异步收发器（UART）

16.1 UART0

16.1.1 功能简介

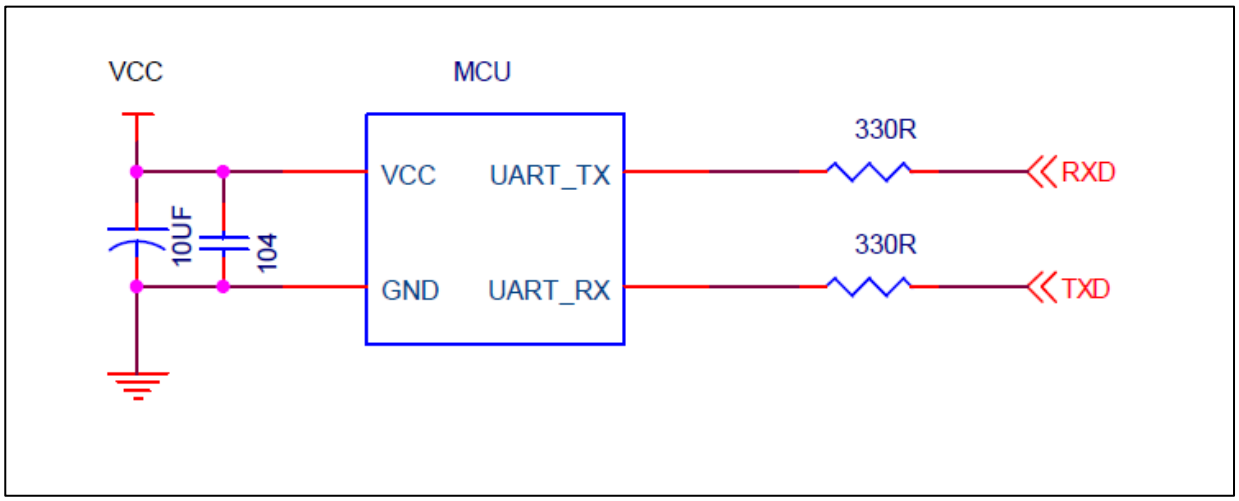
UART0 是一个全双工同步/异步串行数据收发器（全双工意味着可以同时发送和接收数据），与标准 8051 基本兼容。UART0 接收器有一字节的缓存，也就是接收完的一个字节数据会被送到缓存寄存器，同时接收器可以接收新的数据，当然，在新的一字节数据接收完之前，前面接收的一字节数据必须被读取，否则会被新数据覆盖。寄存器 S0BUF 是 UART0 的发送/接收数据寄存器，在物理上，S0BUF 实际是两个寄存器，一个是数据发送寄存器，另一个是数据接收寄存器，写 S0BUF 会将数据写入发送寄存器并启动数据发送，而读 S0BUF 会读取接收寄存器中接收到的一字节数据。

UART0 有 4 种工作模式，如表 16-1-1-1 所示。

SM00	SM10	模式	描述	波特率
0	0	0	同步移位模式	Fclk/12
0	1	1	8 位异步模式	波特率为 2*SMOD*CPUCLK*(定时器 1/2 溢出率)/32，详见 T2CON 中 UCKS
1	0	2	9 位异步模式	当 SMOD＝0 时，波特率为 Fclk/64 当 SMOD＝1 时，波特率为 Fclk/32
1	1	3	8 位异步模式	波特率为 2*SMOD*CPUCLK*(定时器 1/2 溢出率)/32，详见 T2CON 中 UCKS

备注：由于定时器 *2* 的时钟是直接来自系统时钟，没有经过分频，所以选择定时器 *2* 作为 *UART0* 时钟发生器会有更高的波特率，在系统时钟为 *3.6864MHz* 时，最高波特率可达 *115200*。

表 **16-1-1-1　UART0** 通信工作模式



以上元件参数仅供参考。

图 **16-1-1-2UART0/1** 参考电路图

- 模式 **0**

在模式 0，UART0 同步收发数据。引脚 TX 输出移位时钟，引脚 RX 用于输出或接收数据。传输数据为 8 位数据，从最低位开始传输，波特率固定为主时钟频率的 1/12。写入数据到寄存器 S0BUF 会启动 UART0 发送。作为接收器，需要设置寄存器 S0CON 的 REN=1 和清除 RI0 标志，当接收到一字节数据时，RI0 会置 1。

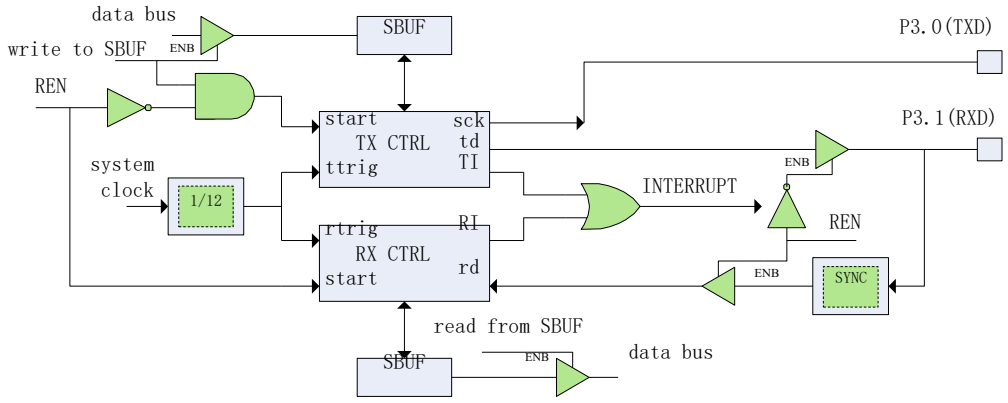


图 **16-1-1-3　UART0** 模式 **0** 示意图

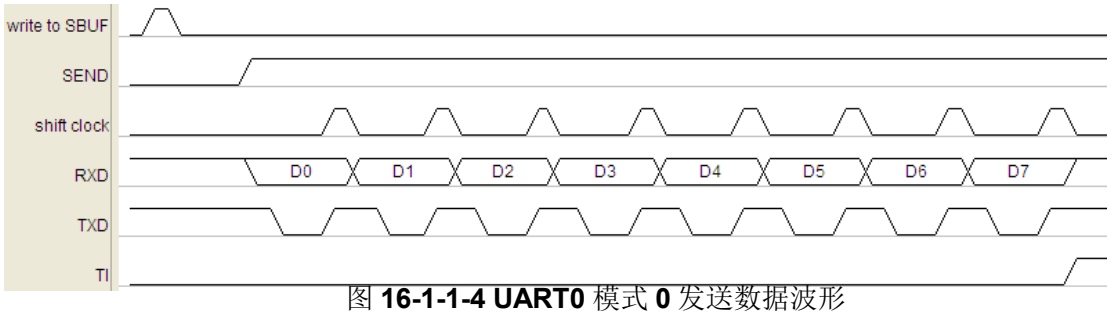


图 **16-1-1-4 UART0** 模式 **0** 发送数据波形

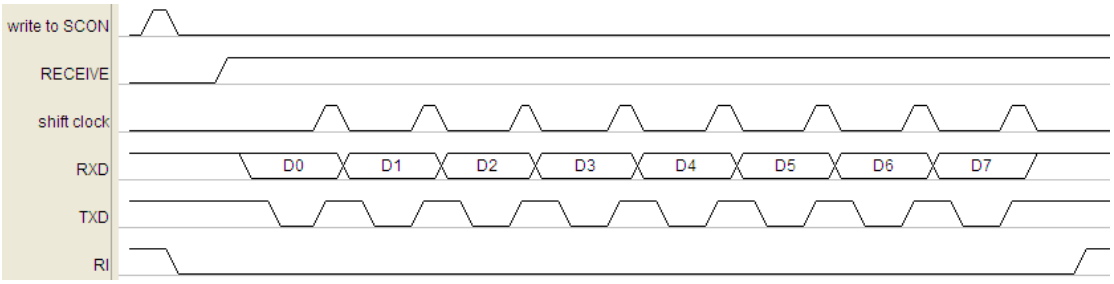


图 **16-1-1-5** 串口 **0** 模式 **0** 接收数据波形

- 模式 **1**

在模式 1，UART0 可异步同时收发 8 位数据。可通过设置 UCKS 位（详见寄存器 T2CON）来选择定时器 1 或定时器 2 的溢出信号作为 UART0 的时钟，相应地，设置定时器的溢出率也就可以调整 UART0 的波特率。另外，可通过 SMOD 位（详见寄存器 PCON）来选择波特率倍频。

写入数据到寄存器 S0BUF 会启动 UART0 发送。第一个传送的位是开始位（为 0），然后是 8 位数据（低位先发），最后传送的是停止位（为 1）。

在接收状态，UART0 通过检测引脚 RX 的下降沿来同步。传送过程完成后，8 位数据存放在寄存器 S0BUF，有效停止位值存放在 RB80 位（S0CON[2]）。

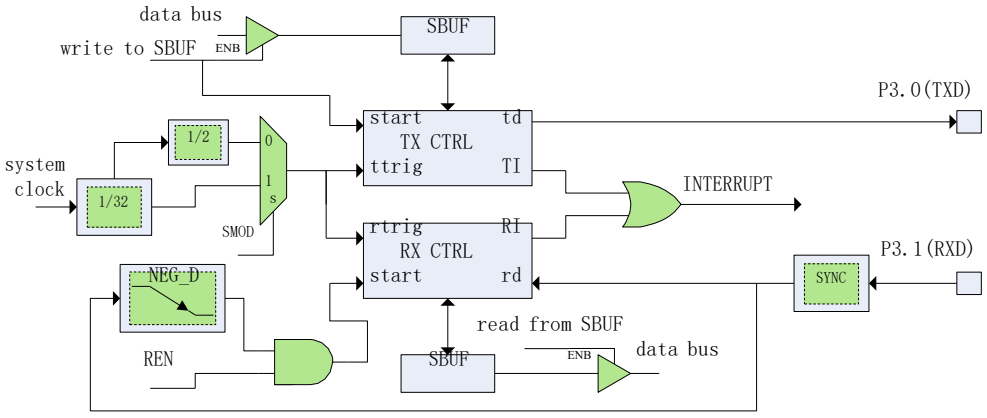


图 **16-1-1-6　UART0** 模式 **1** 示意图

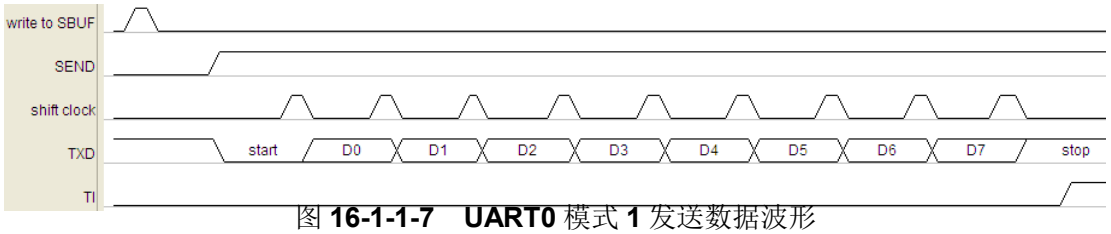


图 **16-1-1-7　UART0** 模式 **1** 发送数据波形

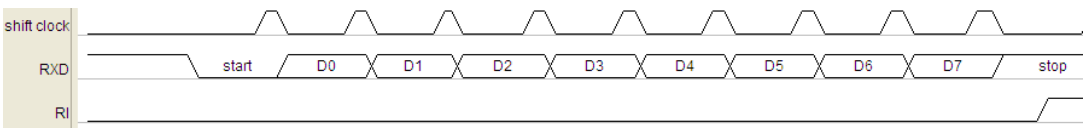


图 **16-1-1-8　UART0** 模式 **1** 接收数据波形

- 模式 **2**

在模式 2，UART0 可异步同时收发 9 位数据，通过设置寄存器 PCON 的 SMOD 位可选择波特率固定为 Fsys/32 或 Fsys/64。

写入数据到寄存器 S0BUF 会启动 UART0 发送。第一个传送的位是开始位（为 0），然后是 9 位数据（低位先发），第 9 位数据是寄存器 S0CON 的 TB80 位，最后传送的是停止位（为 1）。

传送过程通过写 S0BUF 寄存器开启。首先传送的是开始位（一直为 0），然后是 9 位数据，首先是传输数据的最低位，第 9 位是 S0CON 寄存器的 TB80，最后是传送停止位（一直为 1）。

在接收状态，UART0 通过检测引脚 RX 的下降沿来同步。传送过程完成后，低 8 位数据存放在寄存器 S0BUF，第 9 位数据存放在 RB80 位（S0CON[2]）。

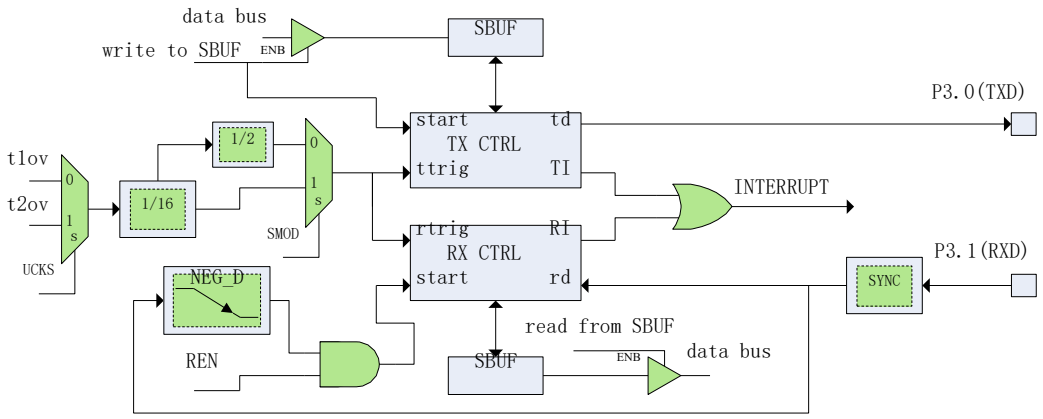


图 **16-1-1-9 UART0** 模式 **2** 示意图

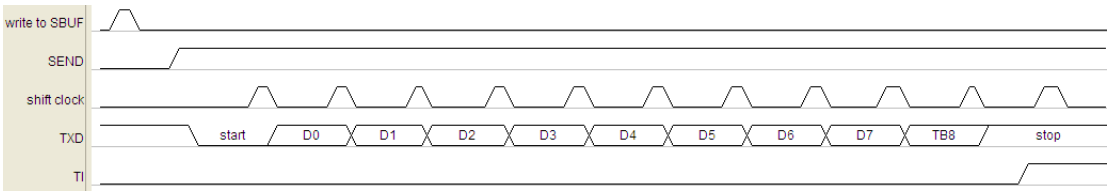


图 **16-1-1-10 UART0** 模式 **2** 发送数据波形

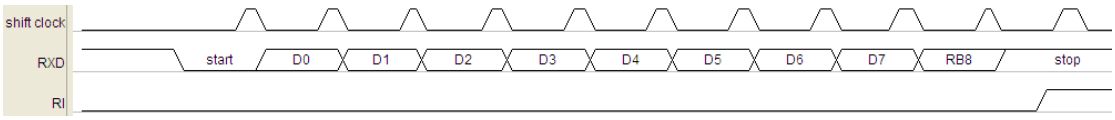


图 **16-1-1-11 UART0** 模式 **2** 接收数据波形

- 模式 **3**

模式 2 和模式 3 的唯一不同的是模式 3 的波特率可通过定时器 1 或定时器 2 来产生，可参考模式 1 的示意图。波特率设置可参考模式 1 介绍，而其他功能描述参考模式 2。

- **UART0** 多机通信

在 UART0 模式 2 和 3 中有一个专门适用于多机通信的机制。当寄存器 S0CON 的 SM20 位置 1，只有接收到第 9 位数据为 1（RB80=1）的从机才会产生接收中断，利用这个功能可进行多机通信，从机将它们的 SM20 位都置为 1，主机传送从机的地址时将第 9 位数据设为 1，这样所有的从机都会产生接收中断；从机的软件用它们自己的地址和接收的地址进行比较，如果一致，被寻址的从机设置 SM20=0，然后主机继续传送后面的数据时设置第 9 位为 0，因为其他的从机 SM20 仍然设为 1，这样就只有被寻址的从机才会产生接收中断。

16.1.2 寄存器描述

表 **16-1-2-1** 寄存器 **S0CON**

98H	7	6	5	4	3	2	1	0
S0CON	SM00	SM10	SM20	REN0	TB80	RB80	TI0	RI0
R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	SM00	串口 0 模式选择位，详见表 17-1-1-1
6	SM10	
5	SM20	多机通信使能位，1 有效
4	REN0	串行接收使能位，1 有效
3	TB80	发送的第 9 位数据 在模式 2 和 3，这个位用于 UART0 发送数据，对应发送数据的第 9 位 （例如奇偶校验或多机通信），由软件控制
2	RB80	接收的第 9 位数据 在模式 2 和 3，这个位用于 UART0 接收数据，对应接收数据第 9 位；模式 1 时该位为停止位；如果 SM2=1，该位为多主机令牌位；在模式 0 这个位没有使用
1	TI0	发送中断标志，1 有效，写 0 清除
0	RI0	接收中断标志，1 有效，写 0 清除

表 **16-1-2-2** 寄存器 **S0BUF**

99H	7	6	5	4	3	2	1	0
S0BUF	S0BUF[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	S0BUF	发送接收缓冲器 写 S0BUF 将启动发送所写的数据 读 S0BUF 将读取已经接收的数据

16.2 UART1

16.2.1 介绍

UART1 是一个全双工异步串行数据收发器，和 UART0 相同的是，UART1 也有一字节的接收缓存。UART1 有两种不同的工作模式，如表 16-2-1-1 所示。

SM1	模式	描述	波特率
0	A	9 位异步模式，与 UART0 的模式 2 和 3 相同	CPUCLK/(32*(1024-S1REL))
1	B	8 位异步模式，与 UART0 的模式 1 相同	CPUCLK/(32*(1024-S1REL))

表 **16-2-1-1 UART1** 工作模式

UART1 的工作原理与 UART0 的异步模式（模式 1/2/3）相同，只是波特率的配置方法有所区别，无论是模式 A，还是模式 B，都可以参考 UART0 的工作波形。和 UART0 不同的是，UART1 设计了专门的波特率发生器，波特率通过寄存器 S1RELL、S1RELH 来配置。
备注：*UART1* 的波特率不能通过 *PCON* 寄存器的 *SMOD* 位来设置倍频。

图 16-2-1-2 是 UART1 的原理示意图。

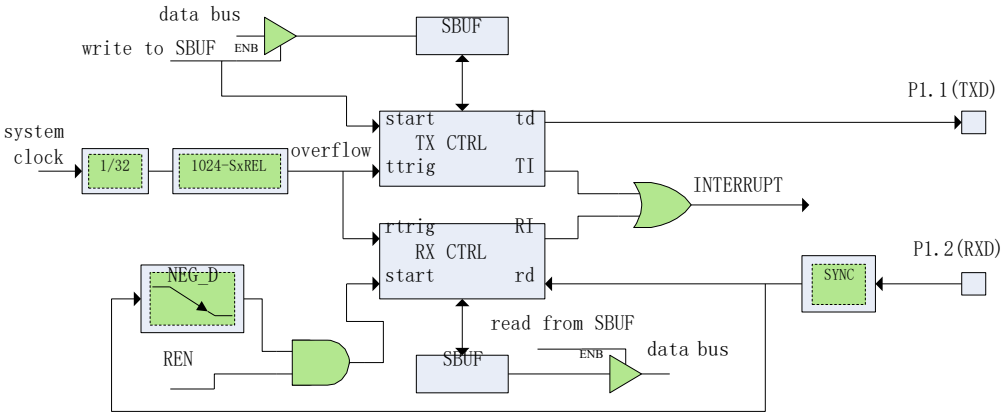


图 **16-2-1-2UART1** 工作原理示意图

- 模式 **A**

在模式 A，UART1 可异步同时收发 9 位数据。写入数据到寄存器 S1BUF 会启动 UART1 数据发送。第一个传送的位是开始位（为 0），然后是 9 位数据（低位先发），第 9 位数据是寄存器 S1CON 的 TB81 位，最后传送的是停止位（为 1）。在接收状态，UART1 通过检测引脚 RX 的下降沿来同步。传送过程完成后，低 8 位数据存放在寄存器 S1BUF，第 9 位数据存放在 RB81 位。

- 模式 **B**

  模式 B 和模式 A 不同的是，模式 B 是 8 位数据传输，停止位存放的是有效停止位。其他功能和模式 A 一致。

- **UART1** 多机通信

在 UART1 模式 A 中有一个专门适用于多机通信的机制。当寄存器 S1CON 的 SM21 位置 1，只有接收到第 9 位数据为 1（RB81=1）的从机才会产生接收中断，利用这个功能可进行多机通信，从机将它们的 SM21 位都置为 1，主机传送从机的地址时将第 9 位数据设为 1，这样所有的从机都会产生接收中断；从机的软件用它们自己的地址和接收的地址进行比较，如果一致，被寻址的从机设置 SM21=0，然后主机继续传送后面的数据时设置第 9 位为 0，因为其他的从机 SM21 仍然设为 1，这样就只有被寻址的从机才会产生接收中断。

16.2.2 UART1 寄存器描述

表 **16-2-2-1** 寄存器 **S1CON**

9AH	7	6	5	4	3	2	1	0
S1CON	SM1	U1IE	SM21	REN1	TB81	RB81	TI1	RI1
R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	SM1	UART1 模式选择位，详见表 16-2-1-1
6	U1IE	UART1 中断使能位，1 有效
5	SM21	多机通信使能位，1 有效
4	REN1	串行接收使能位，1 有效
3	TB81	发送数据的第 9 位 在模式 A，这个位用于 UART1 传送数据，对应传送数据的第 9 位 （例如奇偶校验或多主机通信），由软件控制
2	RB81	接收数据的第 9 位 在模式 A，这个位用于 UART1 接收数据，对应接收数据的第 9 位； 在模式 B，这个位是接收到的停止位
1	TI1	传送中断标志位，1 有效，写 1 清 0
0	RI1	接收中断标志位，1 有效，写 1 清 0

表 **16-2-2-2** 寄存器 **S1BUF**

9BH	7	6	5	4	3	2	1	0
S1BUF	S1BUF[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	S1BUF	UART1 收发缓冲器 写 S1BUF 将开始发送所写的数据 读 S1BUF 将得到已经接收的数据

表 **16-2-2-3** 寄存器 **S1RELL**、**S1RELH**

9CH	7	6	5	4	3	2	1	0
S1RELL	S1RELL[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
9DH	7	6	5	4	3	2	1	0
S1RELH	-	-	-	-	-	-	S1REL[9:8]	
R/W	-	-	-	-	-	-	R/W	
初始值	-	-	-	-	-	-	0	0

位编号	位符号	说明
9~0	S1REL	波特率配置寄存器 波特率为 CPUCLK/(32 * (1024 - S1REL))

17 I² C 接口

17.1 功能简介

I2C 模块支持芯片与外围 I2C 器件以标准 I2C 协议进行串行数据传输，可设置为主机或从机模式，通过合理配置可使 I2C 支持标准/快速/高速模式。

17.2 I2C 主要特点

- 简单且强大而灵活的通讯接口，双向两线总线
- 可设置为主机或从机模式
- 可以工作于发送器模式或接收器模式
- 7 位从机地址
- 支持多主机仲裁
- 支持广播功能

17.3 I2C 功能描述

I2C 模块支持 I2C 标准总线协议。I2C 总线用 2 根线在设备间传输数据，分别为 SCL（串行时钟线）和 SDA（串行数据线），如图 19-3-1 所示。由于 I2C 端口是开漏结构，所以 I2C 总线上必须有上拉电阻，上拉电阻可以外接也可以在芯片内部打开。每个连接在总线上的设备都有一个唯一的 7 位地址。

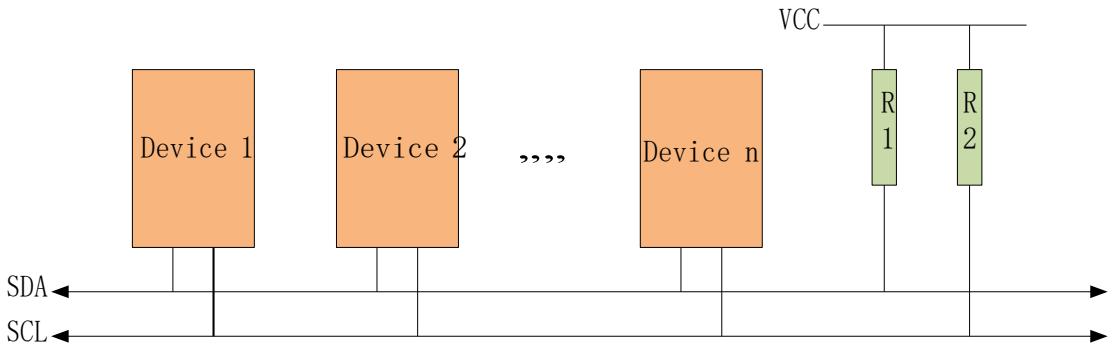


图 **17-3-1 I2C** 总线互连图

I2C 模块原理示意图如图图 17-3-2 所示。

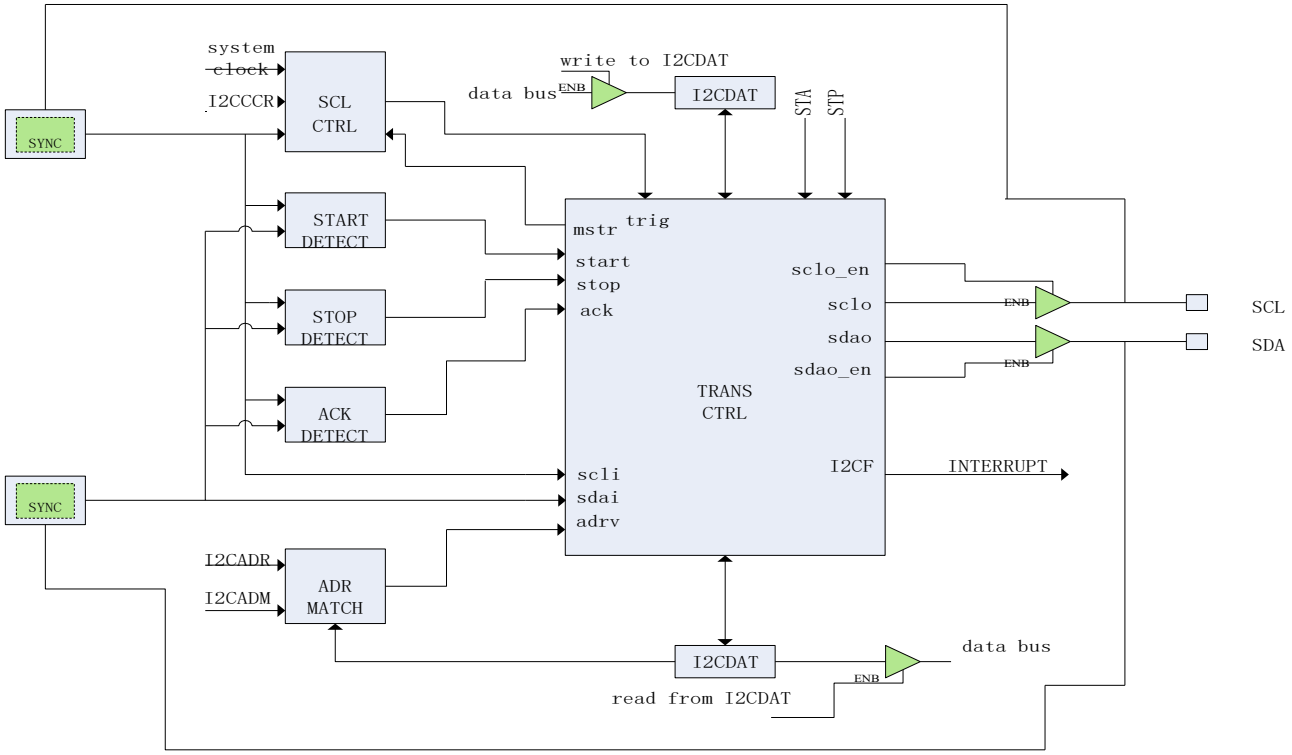


图 **17-3-2　I2C** 模块原理示意图

- **I2C** 模式选择

I2C 可以在以下 4 种模式中的一种运行：从机发送模式、从机接收模式、主机发送模式、主机接收模式。默认情况下，I2C 处于从机模式。I2C 在产生开始信号后自动从从机模式切换到主机模式，当仲裁失败或产生 STOP 信号后又自动切回从机模式。

- **I2C** 总线数据传输格式

一般情况下，标准的 I2C 通信由四部分组成：开始信号、从机地址传输、数据传输和结束信号。I2C 总线上传送的数据均为 8 位，高位先发，每发送一个字节后都必须跟随一个应答位，每次通信的数据字节数没有限制；在全部数据传送结束后，由主机发送停止信号，结束通信。

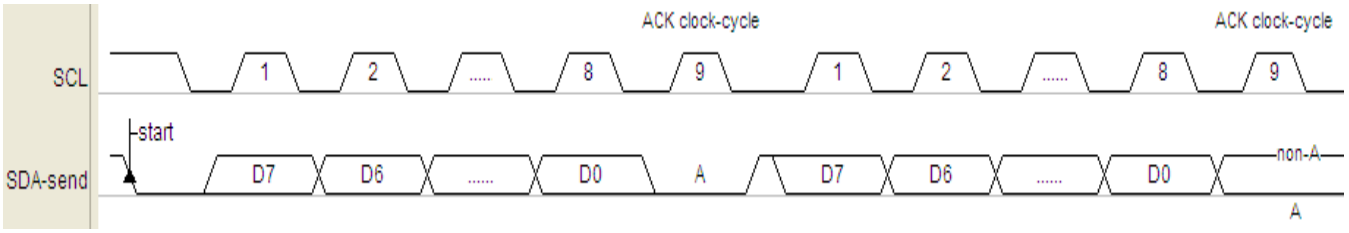


图 **17-3-3　I2C** 总线数据传输格式

- 通信过程

在主机模式下，I2C 接口启动数据传输并产生时钟信号。串行数据传输总是以 START 信号开始，以 STOP 信号结束。START 信号和 STOP 信号都是在主机模式下通过软件控制产生的，START 信号通过设置 STA=1 产生，而 STOP 信号通过设置 STP=1 产生。

在从机模式下，I2C 接口能识别自身地址（7 位地址）和广播地址。软件能通过 GCE 位使能或禁止广播地址的识别。

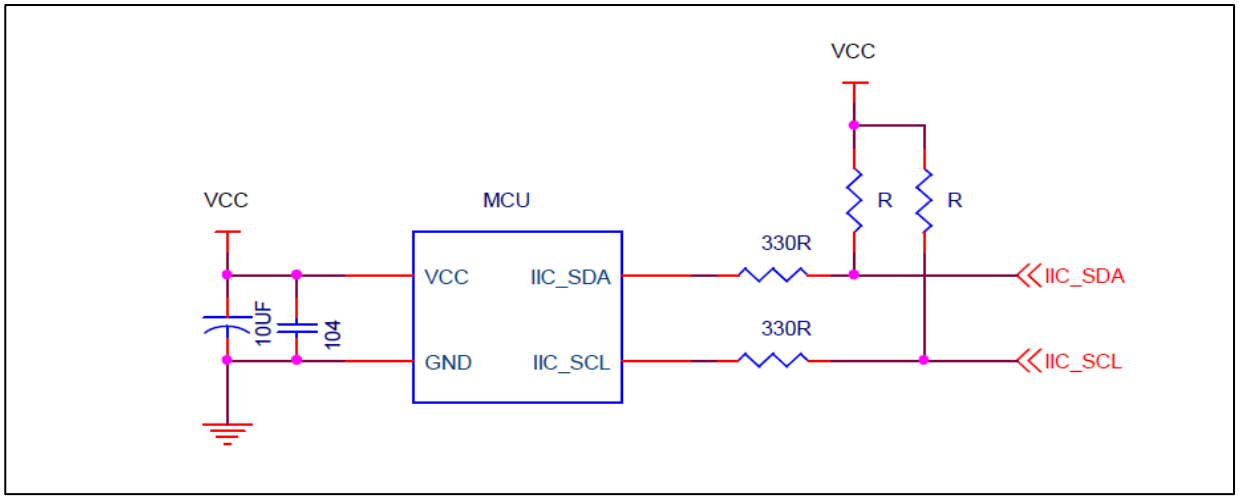
地址和数据以字节为单位进行传输，地址会跟在 START 信号之后由主机发送。在一个字节传输的 8 个时钟后的第 9 个时钟周期内，接收器必须回送一个应答位给发送器。应答位通过 AAK 位设置，设置应答位必须在一个字节传输完之前设置，接收器完成一个字节接收时，应答信号自动产生。数据传输过程中，数据发送/接收完一字节、仲裁失败等事件都会产生中断标志 I2CF，而事件的状态则由寄存器 I2CSTA 指示（详细请参考寄存器 I2CSTA 介绍），软件应在产生中断标志后根据事件的状态设置数据传输的下一步操作，清除中断标志 I2CF 将启动下一步操作。当中断标志 I2CF 产生时，如果 SHD=1，在没有清除 I2CF 之前，SCL 会被从机拉低，主机检测到 SCL 被释放后才会进行下一步操作；如果 SHD=0，从机不会拉低 SCL，这样设计是为了兼容主机是软件模拟 I2C 的应用，此时，主机的软件必须等待足够长的时间让从机响应每字节数据传输的处理。

- **I2C** 时钟设置

当 I2C 接口作为从机时，SCL 的时钟由主机输入，和从机的时钟配置无关。作为从机时，I2C 的采样时钟由 SMPDIV(I2CCCR[7:5])设置，当 SMPDIV 不为 0 时，滤波功能自动启动。作为主机时，SCL 的输出时钟频率由 SMPDIV 和 I2CCKD(I2CCCR[4:0])决定（具体请查看寄存器部分介绍）。

17.4 I2C 通信引脚的映射

为了方便硬件设计，I2C 通信引脚可以有不同的映射，由寄存器 I2CIOS 设置不同的值来选择。详细见寄存器 I2CIOS 的描述。



重要提醒：以上元件参数仅供参考。

图 **17-4-1** **IIC** 参考电路图

17.5 寄存器描述

表 **17-5-1** 寄存器 **I2CCON**

B1H	7	6	5	4	3	2	1	0
I2CCON	I2CE	I2CIE	STA	STP	SHD	AAK	CBSE	STFE
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	1	0	0	0	0	1	0	0

位编号	位符号	说明
7	I2CE	I2C 模块使能位，1 有效
6	I2CIE	I2C 中断使能位，1 有效
5	STA	I2C 发送 START 信号控制位，1 有效，检测到 START 信号后将自动清 0
4	STP	I2C 发送 STOP 信号控制位，1 有效，检测到 STOP 信号后将自动清 0
3	SHD	I 为 1 时，如果 I2CF 为 1，那么当 SCL 变低之后，I2CF 将会使 SCL 保持在低的状态
2	AAK	I2C 发送 ACK 信号控制位，1 有效 备注： 当 I2C 接口配置为从机模式时，这一位须预先置 1，否则即使地址匹配也不会回复 ACK，从而无法被寻址。
1	CBSE	CBUS 兼容使能位 当这一位设置为 1 时，将会使传输忽略 ACK 位的状态判断，以兼容 CBUS 总线。同时需要注意，因为 CBUS 总线的地址都是 7 位的，所以还要配置 HCE 位为 0。
0	STFE	为 1 时，I2C 模块检测到 START 信号时将置位 I2CF

表 **17-5-2** 寄存器 **I2CADR**

B2H	7	6	5	4	3	2	1	0
I2CADR	GCE	I2CADR[6:0]						
R/W	R/W	R/W						
初始值	1	0	0	0	0	0	0	0

位编号	位符号	说明
7	GCE	识别广播地址（00H）使能位，1 有效
6~0	I2CADR	I2C 从机地址，作为从机时有效 备注： （在 *AAK* 为 *1* 的前提下）*7* 位地址模式时，接收的第一个地址字节高 *7* 位和 *I2CADR* 匹配，则回复 *ACK*，进入从机模式。

表 **17-5-3** 寄存器 **I2CADM**

B3H	7	6	5	4	3	2	1	0
I2CADM	SPFE	I2CADM[6:0]						
R/W	R/W	R/W						
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	SPFE	SPFE 位为 1 时，I^2C 模块检测到 STOP 信号时将置位 I2CF
6~0	I2CADM	I2C 从地址按位屏蔽寄存器，为从机时有效 当 I2CADM[n](n=0~6)=1 时，对应的地址位 I2CADR[n]将不比对（即认为无论收到 1 还是 0 都算匹配）。

表 **17-5-4** 寄存器 **I2CCCR**

B4H	7	6	5	4	3	2	1	0
I2CCCR	SMPDIV[2:0]			I2CCKD[4:0]				
R/W	R/W							
初始值	0	0	1	0	0	0	0	0

位编号	位符号	说明
7~5	SMPDIV	I2C 采样时钟设置，I2C 采样时钟为 I2C 工作时钟的 2 的 smpdiv 次幂分频，即： 000：$F_{sample}=F_{i2cclk}$ 001：$F_{sample}=F_{i2cclk}/2$ 010：$F_{sample}=F_{i2cclk}/4$ ... 111：$F_{sample}=F_{i2cclk}/128$
4~0	I2CCKD	I2C SCL 输出时钟频率设置，SCL 输出时钟频率为采样频率的(I2CCKD +1)分频，即： $F_{scl}= F_{sample}/(I2CCKD +1)$ 备注： *1.* 当 *SMPDIV= 0* 时，如果 设置 *I2CCKD* 小于 *9*，将自动按 *9* 计算。 *2.* 当 *SMPDIV>0* 时，如果设置 *I2CCKD* 小于 *7*，将自动按 *7* 计算。 备注： *1.* 当 *I2CCCR[7:5] = 0* 时，如果对 *I2CCCR[4:0]*写小于 *9* 的值，将自动按 *9* 的值计算。 *2.* 当 *I2CCCR[7:5] > 0* 时，如果对 *I2CCCR[4:0]*写小于 *7* 的值，将自动按 *7* 的值计算。

表 **17-5-5** 寄存器 **I2CDAT**

B5H	7	6	5	4	3	2	1	0
I2CDAT	I2CDAT[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	I2CDAT	发送和接收数据缓存 备注： 当 *I2CF* 为 *1* 时，建议改写/读取 *I2CDAT* 时，让 *I2CF* 保持在 *1*，等处理完成之后再清除 *I2CF*，以继续传输，这样可以避免总线发生不必要的错误。

表 **17-5-6** 寄存器 **I2CSTA**

B6H	7	6	5	4	3	2	1	0
I2CSTA	I2CSTA[7:0]							
R/W	R							
初始值	1	1	1	1	1	0	0	0

位编号	位符号	说明
7~0	I2CSTA	I2C 状态寄存器 00H: （主/从）总线错误 08H: （主/从）检测到 START 信号（只在 STFE=1 时才有效） 18H: （主）已发送地址+写位，已接收到应答信号 20H: （主）已发送地址+写位，无接收到应答信号 28H: （主）已发送/接收一字节数据，已检测到应答信号 30H: （主）已发送/接收一字节数据，无检测到应答信号 38H: （主）失去仲裁（主机失去仲裁后会变为从机） 40H: （主）已发送地址+读位，已接收到应答信号 48H: （主）已发送地址+读位，无接收到应答信号 60H: （从）已接收地址+写位，已发送出应答信号 70H: （主/从）已接收广播地址，已发送出应答信号（主机或从机都会变为从机） 80H: （从）已发送/接收一字节数据，已检测到应答信号 88H: （从）已发送/接收一字节数据，无检测到应答信号 A0H: （主/从）检测到 STOP 信号（只在 SPFE=1 时才有效） A8H: （从）已接收地址+读位，已发送出应答信号 F8H: （主/从）总线空闲

表 **17-5-7** 寄存器 **I2CFLG**

B7H	7	6	5	4	3	2	1	0
I2CFLG	-	-	-	-	-	-	-	I2CF
R/W	-	-	-	-	-	-	-	R
初始值	-	-	-	-	-	-	-	0

位编号	位符号	说明
7~1	-	-

0	I2CF	I2C 中断标志，1 有效，写 1 清 0 备注： *1.* 每字节地址或数据传输完成后（收到/发送完 *ACK/NAK*），将置位 *I2CF*。 *2.* 总线出错时，将置位 *I2CF*。 *3.* 当 *STFE=0* 时，检测到 *START* 信号，*I2CF* 不会置 *1*。 *4.* 当 *SPFE=0* 时，检测到 *STOP* 信号，*I2CF* 不会置 *1*。

表 **17-5-8** 寄存器 **I2CIOS**

8101H	7	6	5	4	3	2	1	0
I2CIOS	I2CKS	-	-	-	-	-	-	I2CS
R/W	R/W	-	-	-	-	-	-	R/W
初始值	0	-	-	-	-	-	-	0

位编号	位符号	说明
7	I2CKS	I2C 工作时钟选择位 0：系统时钟 1：IRCH
6~1	-	-
0	I2CS	I2C 引脚选择位 0：SCL 在引脚 P31，SDA 在引脚 P30 1：SCL 在引脚 P12，SDA 在引脚 P11

17.6 I2C 控制例程

I2C 作为主机例程

例如，主机循环向从机写入 20 字节数据，程序如下：

```
//I2CCON 定义
#define I2CE(N)         (N<<7)
#define I2CIE(N)        (N<<6)
#define STA(N)          (N<<5)
#define STP(N)          (N<<4)
#define CKHD(N)         (N<<3)
#define AAK(N)          (N<<2)
#define CBSE(N)         (N<<1)
#define STFE(N)         (N<<0)
//I2CADR 定义
#define    GCE          (1<<7)
//I2CFLG 定义
#define I2CF            (1<<0)
#define I2C_ADDR        0xA0

unsigned  char  txIndex;
unsigned  char  regAddr;
unsigned char txLength;
unsigned char txBuf[20]={0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19};
void I2C_init(void)
{
    I2CIOS = 0;          //选择 P36,P37 作为 I2C 通信引脚
    P11F = 6|(1<<7); //设置 P36 作为 I2C SDA，并打开上位
    P12F = 6|(1<<7); //设置 P36 作为 I2C SCL，并打开上位
    I2CCON = I2CE(1) | I2CIE(1) | STA(0) | STP(0)| CKHD(1) | AAK(1)| CBSE(0) | STFE(1);      /*使能 I2C，开启 I2C
                                                中断，设置 I2CF 置 1 后拉低 SCL，设置 START 信号产生中断*/
    I2CCCR = 5;    //设置 I2C 时钟分频
    INT6EN = 1;    //使能 INT6 中断
}
void INT6_ISR(void) interrupt 11
{
    unsigned char Sta_Temp;
    if(I2CFLG & I2CF)                    //I2C 中断
    {
        Sta_Temp = I2CSTA;           //读取 I2C 状态
        if(Sta_Temp == 0x08)         //START 信号已产生
        {
            I2CDAT = I2C_ADDR;       //发送 I2C 从机地址+写位
        }
```

```
        else if(Sta_Temp == 0x18)     //地址+写位已发送，收到从机应答
        {
            I2CDAT = regAddr;         //发送数据首地址
            txIndex = 0;
        }
        else if(Sta_Temp == 0x28)     //数据已发送，已收到从机应答
        {
            if(txIndex >= txLength)   //判断是否所有数据已发送完
            {
                I2CCON |= STP(1);  //如果已发送完所有数据则发送 STOP 信号
                iicEnd = 1;           //置位结束标志
            }
            else
            {
                I2CDAT = txBuf[txIndex+regAddr]; //如未发送则发送下一字节数据
                txIndex++;
            }
        }
        I2CFLG    |= I2CF;                  //清除 I2C 中断标志
    }
}
void main(void)
{
    I2C_init();
    EA = 1; //开启全局中断
    while(1)
    {
        regAddr = 0;              //复位数据地址
        iicEnd = 0;               //结束标志清 0
        txLength = 10;            //设置发送数据长度
        I2CCON |= STA(1);         //发送 START 信号，启动数据发送
        While(!iicEnd);           //等待数据发送完
    }
}
```

例如，主机循环从从机读取 20 字节数据，程序如下：

```
bit rdSession;
bit iicEnd;
unsigned char rxIndex;
unsigned char rxLength;
unsigned char rxBuf[20];
void I2C_init(void)
{
```

```
    I2CIOS = 0;                 //选择 P36,P37 作为 I2C 通信引脚
    P11F = 6|(1<<7);            //设置 P36 作为 I2C SDA，并打开上位
    P12F = 6|(1<<7);            //设置 P36 作为 I2C SCL，并打开上位
    I2CCON = I2CE(1) | I2CIE(1) | STA(0) | STP(0)| CKHD(1) | AAK(1)| CBSE(0) | STFE(1);      /*使能 I2C，开启 I2C
                                                中断，设置 I2CF 置 1 后拉低 SCL，设置 START 信号产生中断*/
    I2CCCR = 5;                 //设置 I2C 时钟分频
    INT6EN = 1;                 //使能 INT6 中断
}

void INT6_ISR(void) interrupt 11
{
    unsigned char Sta_Temp;
    if(I2CFLG & I2CF)    //I2C 中断
    {
        Sta_Temp = I2CSTA;                  //读取 I2C 状态
        if(Sta_Temp == 0x08)                //START 信号已产生
        {
            if(rdSession)
            {
                I2CDAT = I2C_ADDR|1;  //读数据阶段，发送从机地址+读位
            }
            else
            {
                I2CDAT = I2C_ADDR;    //写数据阶段，发送从机地址+写位
            }
        }
        else if(Sta_Temp == 0x18)           //地址+写位已发送，收到从机应答
        {
            I2CDAT    = regAddr;            //发送数据首地址
        }
        else if(Sta_Temp == 0x28)           //已发送或接收到一字节数据，收到从机应答
        {
            if(rdSession)                   //判断是写数据阶段还是读数据阶段
            {
                if(rxIndex >= rxLength)  //如果是读数据阶段，判断是否已读取完数据
                {
                    I2CCON |= STP(1);  //如果已读取完，发送 STOP 信号
                }
                else
                {
                    rxBuf[rxIndex+regAddr] = I2CDAT;   //读取一字节数据
                    rxIndex++;
                    if(rxIndex <= (rxLength-1))
                    {
                        I2CCON |= AAK(1);   //如果不是最后一字节数据，发送应答信号
```

```
                    }
                    else
                    {
                        I2CCON &= ~AAK(1);//最后一字节数据发送不应答信号
                    }
                }
            }
            else
            {
                I2CCON |= STA(1);       //如果是写数据阶段，重新发送 START 信号
                rdSession = 1;
            }
        }
        else if(Sta_Temp == 0x30)       //主机发送/接收一字节数据，没有收到应答信号
        {
            if(rdSession)
            {
                if(rxIndex >= rxLength) //判断如果是最后一字节数据，则发送 STOP 信号
                {
                    I2CCON |= STP(1);
                    iicEnd = 1;         //置位结束标志
                }
            }
        }
        else if(Sta_Temp == 0x40)       //从机地址+读位已发送，已收到应答信号
        {
            I2CCON |= AAK(1);           //先设置好应答位
            rxIndex=0;
        }
        I2CFLG   |= I2CF;               //清除 I2C 中断标志
    }
}
void main(void)
{
    I2C_init();
    EA = 1; //开启全局中断
    while(1)
    {
        regAddr = 0;            //复位数据地址
        iicEnd = 0;             //结束标志清 0
        rxLength= 20;           //设置发送数据长度
        I2CCON |= STA(1);       //发送 START 信号，启动 I2C 通信
        While(!iicEnd);         //等待数据接收完
        //在此处可读取 I2C 接收缓存 rxBuf
    }
```

```
}
```

◆ **I2C** 作为从机例程 作为从机，支持主机写入或读取数据，程序如下：

```
unsigned  char  rxIndex;
unsigned char regAddr;
bit iicReadMode;
bit iicEnd;
unsigned char xdata rxBuf[20];
unsigned char txBuf[20]={0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19};

void I2C_init(void)
{
    I2CIOS = 0;          //选择 P36,P37 作为 I2C 通信引脚
    P11F = 6|(1<<7); //设置 P36 作为 I2C SDA，并打开上位
    P12F = 6|(1<<7); //设置 P36 作为 I2C SCL，并打开上位
    I2CCON = I2CE(1) | I2CIE(1) | STA(0) | STP(0)| CKHD(1) | AAK(1)| CBSE(0) | STFE(0);      /*使能 I2C，开启 I2C
                                        中断，设置 I2CF 置 1 后拉低 SCL，设置 START 信号不产生中断*/
    I2CADR = (I2C_ADDR>>1);    //设置从机 I2C 地址
    INT6EN = 1;    //使能 INT6 中断
}
void INT6_ISR(void) interrupt 11 // using 1
{
    unsigned char Sta_Temp;
    if(I2CFLG & I2CF)                    //I2C 中断
    {
        Sta_Temp = I2CSTA;               //读取 I2C 状态
        if(Sta_Temp == 0x60)                 //接收到从机地址+写位
        {
            rxIndex = 0xff;
            iicReadMode = 0;   //设置标志为写状态
            I2CCON |= AAK(1);       //预设应答位
        }
        else if(Sta_Temp == 0x80)
        {
            if(iicReadMode)          //判断当前为读状态还是写状态
            {
                rxIndex++;
                I2CDAT = rxBuf[rxIndex+regAddr]; //发送一字节数据
            }
            else                             //接收到一字节数据
            {
                if(rxIndex == 0xff)   //第一个数据为数据首地址
```

```
                {
                    regAddr=I2CDAT;
                    rxIndex=0;
                    I2CCON |= AAK(1);
                }
                else                        //数据
                {
                    rxBuf[rxIndex+regAddr] = I2CDAT; //接收一个字节数据
                    rxIndex++;
                    I2CCON |= AAK(1);
                }
            }
        }
        else if(Sta_Temp==0xA8)                 //接收到从机地址+读位，应答信号已发送
        {
            I2CDAT = rxBuf[rxIndex+regAddr];        //预设待发送的一字节数据
            iicReadMode = 1;                //设置当前为读状态
        }
        I2CFLG    |= I2CF;    //清除 I2C 中断标志
    }
}
void main(void)
{
    I2C_init();
    EA = 1;    //开启全局中断
    while(1)
    {
    }
}
```

---

18 LCD/LED 驱动

18.1 LCD 驱动

18.1.1 功能简介

内置 LCD 驱动最大可支持 5com x 8seg、4comx 9seg、3comx10seg，共 13 个驱动引脚。可编程占空比为：1/2、1/3、1/4、1/5，可编程偏压比为：1/2、1/3、1/4。可编程 LCD 驱动强度为 8 个等级。

图 20-1-1-1 是 LCD 的原理示意图。

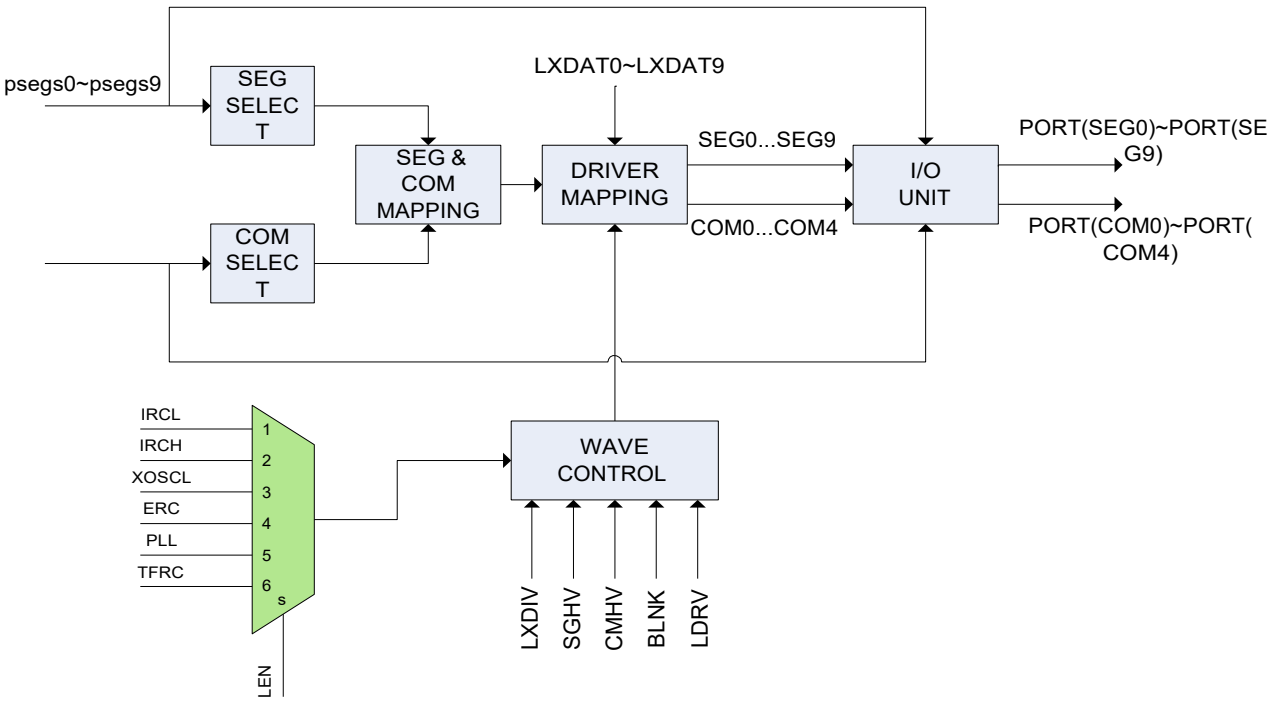


图 **18-1-1-1 LCD** 原理示意图

18.1.2 LCD 偏压

LCD 可编程偏压比为：1/2、1/3、1/4，以下分别是其对应的信号图。

- **LCD** 偏压比 **1/2**

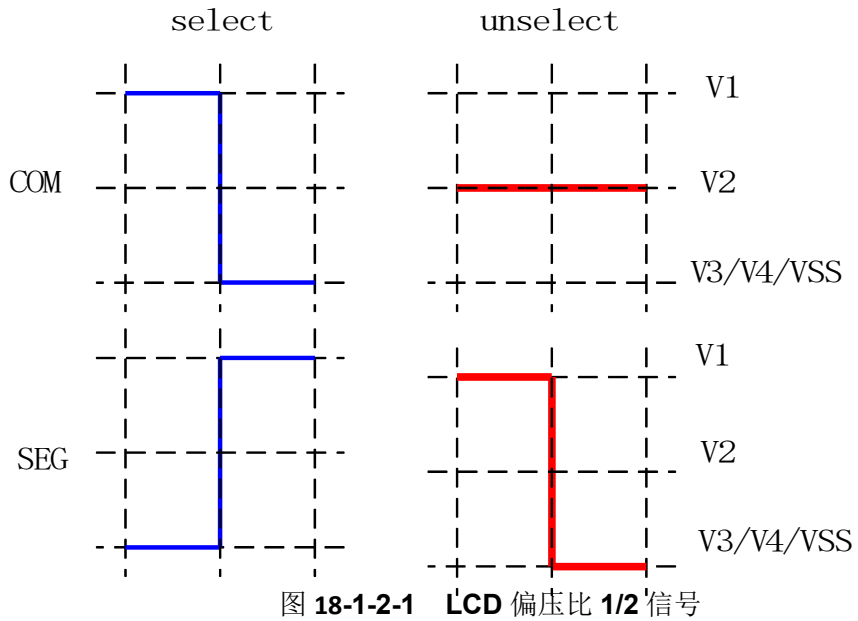


图 **18-1-2-1** **LCD** 偏压比 **1/2** 信号

- **LCD** 偏压比 **1/3**

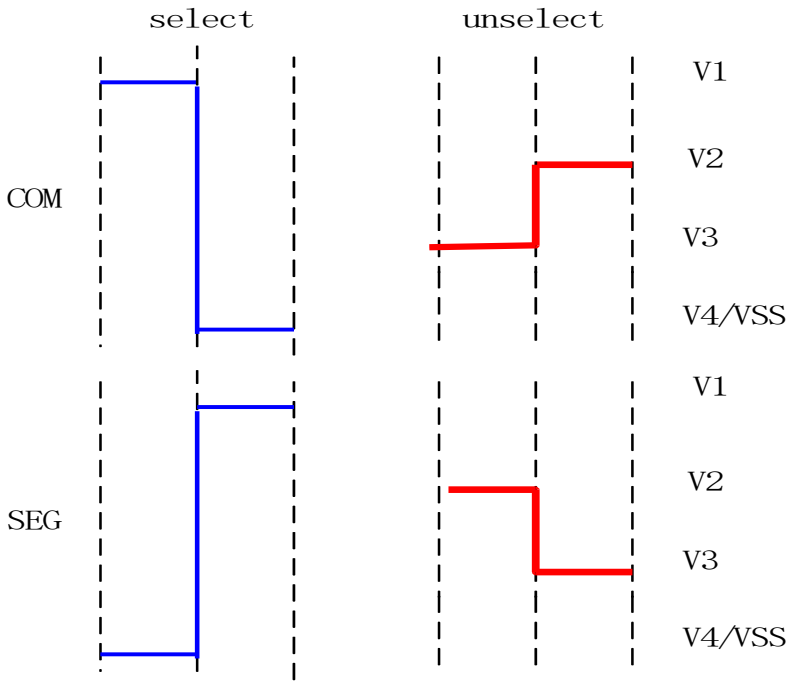


图 **18-1-2-2** **LCD** 偏压比 **1/3** 信号

- **LCD** 偏压比 **1/4**

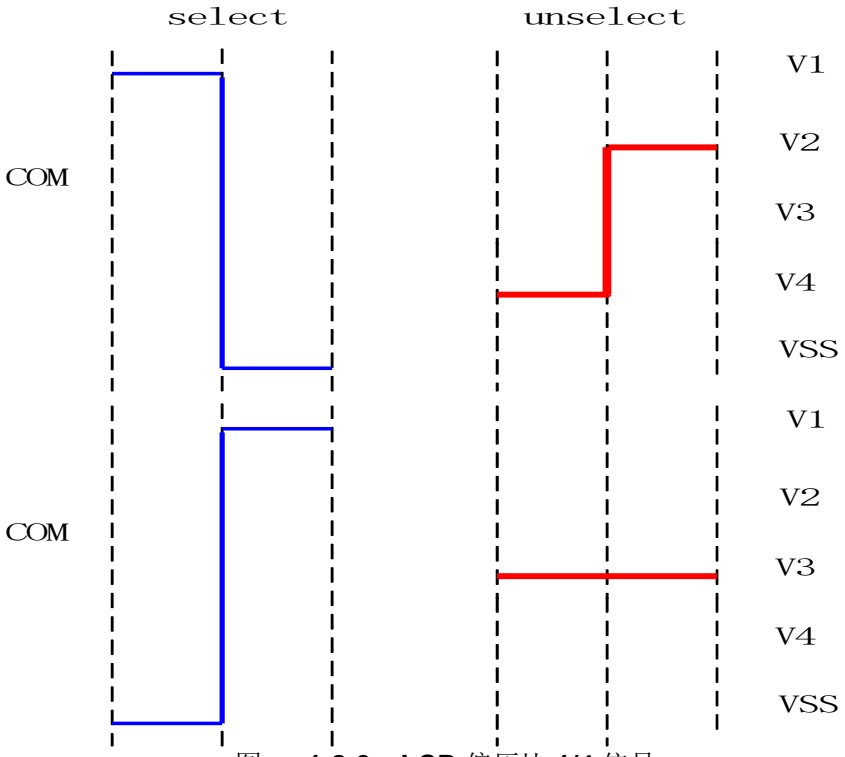


图 **18-1-2-3** **LCD** 偏压比 **1/4** 信号

18.1.3 LCD 功能描述

LCD 模式通过设置 LMOD=1 来选择。LCD 驱动可以通过 LEN 选择时钟源，当时钟源被选择后，LCD 驱动同时被使能；要注意的是，在设置时钟源时必须要确定该时钟源是被打开并且正常工作。寄存器 LXDIV 是 LCD 时钟分频器，针对不同的时钟源可设置不同的分频系数，LCD 扫描的帧频率典型值为 64Hz。LCD 驱动有 8 级驱动强度可调，通过 LDRV 来设置，不同的驱动强度输出电压不同，在应用时可应针对不同的 LCD 显示器调整此数值。为满足不同的功耗要求，LCD 驱动有 4 级驱动电流选择，通过 DMOD 来设置，当驱动电流越小时，驱动本身功耗也越小，但 LCD 引脚上出现的杂波也越大。

LCD 可编程的占空比为：1/2、1/3、1/4、1/5，占空比是由使能的 COM 数量决定的，例如使能了 3 个 COM，那占空比就为 1/3，使能了 5 个 COM，占空比就为 1/5。而 COM 的使能不需要按 COM 引脚编号的顺序进行，可以任意的组合，但使能的 COM 引脚按序号从小到大对应实际的 COM0、COM1、COM2...，例如使能了引脚 COM1、COM3、COM4 为 COM 口，那 COM1 对应实际 COM0,COM3 对应实际 COM1，COM4 对应实际 COM2，占空比为 1/3。SEG 引脚也可任意使能，使能的 SEG 引脚按序号从小到到大对应实际的 SEG0、SEG1、SEG2...，例如使能引脚 SEG3、SEG5、SEG7...,那 SEG3 对应实际 SEG0,SEG5 对应实际 SEG1，SEG7 对应实际 SEG2...。没有被使能的 COM 和 SEG 引脚可以设置为其他功能使用，和 LCD 驱动没有冲突。

LCD 驱动有 10 字节的 LCD 显示缓存。LCD 显示缓存是与实际的 COM,SEG 对应的，10 字节显示缓存按顺序分别对应 SEG0~SEG9，而 COM0~COM4 对应每个字节的 0~4 位。显示缓存是通过索引寄存器 INDEX 和数据寄存器 LXDAT 来访问的，设置 INDEX 为 0~9 按顺序对应 SEG0~SEG9 的显示缓存。

18.2 LED 驱动

18.2.1 功能简介

内置 LED 驱动最大可支持 5com x 8seg、4comx 9seg、3comx10seg，与 LCD 驱动共用显示缓存和驱动引脚。LED 驱动有 8 级亮度可调。内置了全局闪烁功能，在扫描频率为 256Hz 时以 1Hz 的频率闪烁。LED COM 可设置高灌电流模式（设置方法详见 IO 章节介绍），LED 为共阴极接法时可实现高亮度显示效果。

图 20-2-1-1 是 LED 的原理示意图。

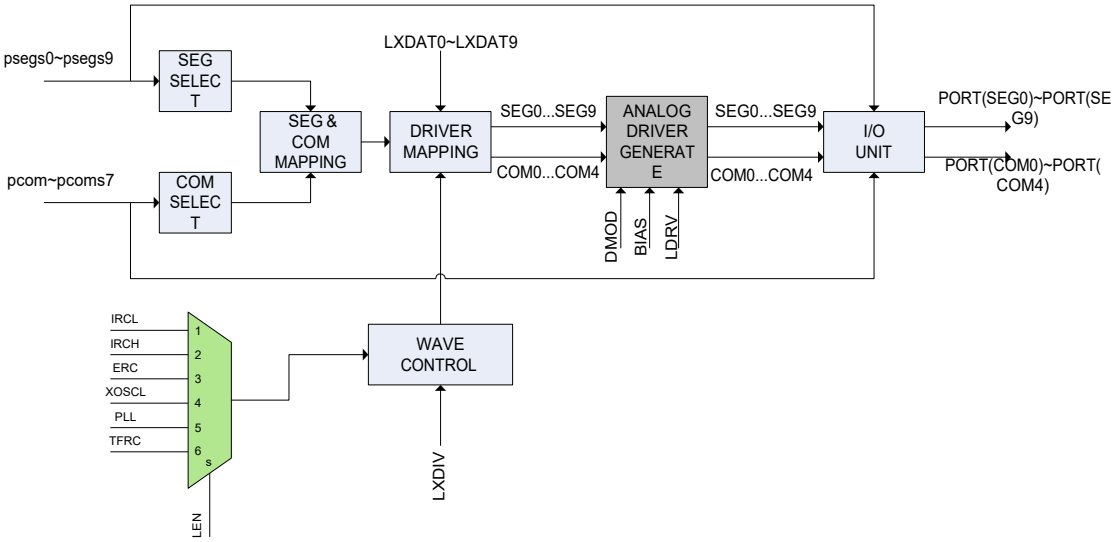


图 18-**2-1-1 LED** 原理示意图

图 18-2-1-1 是 LED 的驱动波形图。

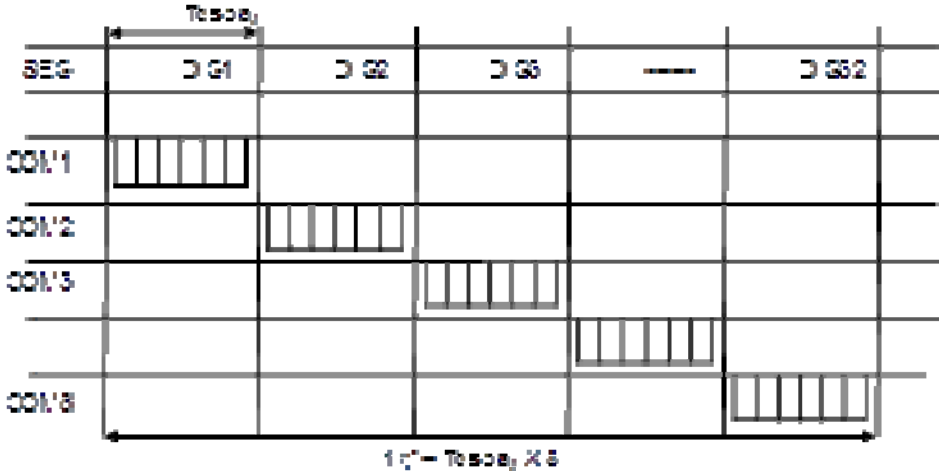


图 **18-2-1-2 LED** 驱动波形

18.2.2 LED 功能描述

LED 模式通过设置 LMOD=0 来选择。LED 驱动可以通过 LEN 选择时钟源，当时钟源被选择后，LED 驱动同时被使能；要注意的是，在设置时钟源时必须要确定该时钟源是被打开并且正常工作。寄存器 LXDIV 是 LED 时钟分频器，针对不同的时钟源可设置不同的分频系数，LED 扫描的帧频率典型值为 256Hz。LED 驱动有 8 级亮度可调，通过 LDRV 来设置，不同的数值对应不同的占空比。

和 LCD 驱动相同的是，LED 的基本占空比也是由使能的 COM 数量决定的，COM 引脚和 SEG 引脚也可以任意组合，可以参考 LCD 部分的相关描述。

LED 驱动和 LCD 驱动共同显示缓存，显示缓存和 COM、SEG 口的对应关系和显示缓存的访问方式都和 LCD 驱动一样，也可参考 LCD 驱动部分的相关描述。

18.3 LCD/LED 寄存器描述

表 **18-3-1** 寄存器 **LXCON**

E1H	7	6	5	4	3	2	1	0
LXCON	LEN[2:0]			LMOD	-	-	-	-
R/W	R/W			R/W	-	-	-	-
初始值	0	0	0	0	-	-	-	-

位编号	位符号	说明
7~5	LEN	LCD/LED 时钟选择位 001：IRCL 010：IRCH 011：XOSCL 100：ERC 101：PLL 110：TFRC 其他:模块关闭
4	LMOD	模式选择位 0：LCD 模式 1：LED 模式
3~0	-	-

表 **18-3-2** 寄存器 **LXCFG**

E2H	7	6	5	4	3	2	1	0
LCD 模式								
LXCFG	DMOD[1:0]		BIAS[1:0]		-	LDRV[2:0]		
R/W	R/W		R/W		-	R/W		
初始值	0	0	0	0	-	0	0	0

LED 模式								
LXCFG	-	-	COMHV	SEGHV	BLNK	LDRV[2:0]		
R/W	-	-	R/W	R/W	R/W	R/W		
初始值	-	-	0	0	0	0	0	0

位编号	位符号	说明
LCD 模式		
7~6	DMOD	LCD 驱动电流大小选择位 00：5uA 左右 01：40uA 左右 10：80uA 左右 11：130uA 左右
5~4	BIAS	LCD 偏压选择位 01：1/2 Bias 10：1/3 Bias 其他：1/4 Bias
3	-	-
2~0	LDRV	LCD 驱动强度控制位 000：Level 1（最小） 001：Level 2 ... 111：Level 8（最大）
LED 模式		
7~6	-	-
5	COMHV	当为 0/1 时，表示 COM 输出 0/1 有效
4	SEGHV	当为 0/1 时，表示 SEG 输出 0/1 有效
3	BLNK	LED 全局闪烁控制位，1 有效 备注： 闪烁时间为 128 帧 LED 的输出。例如，LED 工作时钟选择 32.768KHz，同时 LXDIV=0，此时 LED 的输出频率为 256Hz，那么闪烁的频率是 1Hz。
2~0	LDRV	LED 亮度调节位 000：Level 1（最暗） 001：Level 2 010：Level 3 011：Level 4 100：Level 5 101：Level 6 110：Level 7 111： Level 8（最亮）

表 **18-3-3** 寄存器 **LXDIV**

E4H	7	6	5	4	3	2	1	0
LXDIVL	LXDIV[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
E5H	7	6	5	4	3	2	1	0
LXDIVH	-	-	-	-	LXDIV[11:8]			
R/W	-	-	-	-	R/W			
初始值	-	-	-	-	0	0	0	0

位编号	位符号	说明
15~12	-	-
11~0	LXDIV	LXD 时钟分频器 LCD 扫描帧频率=LXD 时钟频率/((LXDIV+1)*512) LED 选择高速时钟扫描帧频率=LXD 时钟频率/((LXDIV+1)*1024) LED 选择低速时钟扫描帧频率=LXD 时钟频率/((LXDIV+1)*128) 备注： 1. LED 选择高速时钟表示，LED 工作时钟选择 IRCH/ERC/PLL/TFRC；LED 选择低速时钟表示，LED 工作时钟选择 IRCL/XOSCL。 2. 当 LCD/LED 时钟选择为 IRCL 时，时钟频率为 IRCL 的 1/4。 3. LCD 典型扫描帧频率为 64Hz，LED 典型扫描帧频率为 256Hz。

表 **18-3-4** 寄存器 **LXDAT**

E3H	7	6	5	4	3	2	1	0
LXDAT	-	-	-	LXDAT[7:0]				
R/W	-	-	-	R/W				
初始值	-	-	-	0	0	0	0	0

备注：*LXDAT* 是带索引的寄存器，设置 *INDEX=0~9* 分别对应 *LXDAT0~LXDAT9*

位编号	位符号	说明
7~0	LXDAT	显示缓存读写寄存器

表 **18-3-5** 寄存器 **LXCAD**

8117H	7	6	5	4	3	2	1	0
LXCAD	-	-	-	-	CAD_MOD[1:0]		CAD_LTH[1:0]	
R/W	-	-	-	-	R/W		R/W	
初始值	-	-	-	-	0	1	0	1

位编号	位符号	说明
7~4	-	-
3~2	CAD_MOD	LCD_CAD 模式选择 00：关闭 LCD_CAD 01：LCD_CAD 长度由数字信号长度决定 其他：LCD_CAD 长度由模拟信号控制 备注：在低功耗模式可选择关闭 *LCD_CAD*，电流可降低约 *5~10uA*。
1~0	CAD_LTH	模拟信号控制 LCD_CAD 长度选择，仅在 CAD_MOD=2/3 时有效 00 ： 4us 01 ： 8us 10：12us 11：12us

表 **18-3-6 LCD/LED** 显示缓存

INDEX	SEG	COM0	COM1	COM2	COM3	COM4
0	0	BIT0	BIT1	BIT2	BIT3	BIT4
1	1	BIT0	BIT1	BIT2	BIT3	BIT4
2	2	BIT0	BIT1	BIT2	BIT3	BIT4
3	3	BIT0	BIT1	BIT2	BIT3	BIT4
4	4	BIT0	BIT1	BIT2	BIT3	BIT4
5	5	BIT0	BIT1	BIT2	BIT3	BIT4
6	6	BIT0	BIT1	BIT2	BIT3	BIT4
7	7	BIT0	BIT1	BIT2	BIT3	BIT4
8	8	BIT0	BIT1	BIT2	BIT3	-
9	9	BIT0	BIT1	BIT2	-	-

18.4 LCD 驱动控制例程

例如，要驱动 LCD 为 5com×8seg，1/4bias，程序如下：

```
--------------------------------------------------------------------------------
#define XLCKE           (1<<3)
#define XLSTA           (1<<2)

#define LEN_XOSCL       (3<<5)
#define LMOD_lcd        (0<<4)
#define DMOD_5ua        (0<<6)
#define BIAS_1_4        (0<<4)
#define LCDRV_LEV(N)        (N) //N=0~7
```

```
void LCD_init(void)
{
    unsigned char i;

    P00F = 3; //设置 P00 为 COM0
    P01F = 3; //设置 P01 为 COM1
    P02F = 3; //设置 P02 为 COM2
    P03F = 3; //设置 P03 为 COM3
    P04F = 3; //设置 P04 为 COM4

    P17F = 3; //设置 P17 为 SEG0
    P37F = 3; //设置 P37 为 SEG1
    P36F = 3; //设置 P36 为 SEG2
    P35F = 3; //设置 P35 为 SEG3
    P34F = 3; //设置 P34 为 SEG4
    P07F = 3; //设置 P07 为 SEG5
    P06F = 3; //设置 P06 为 SEG6
    P05F = 3; //设置 P05 为 SEG7

    CKCON |= XLCKE;              //打开 XOSCL 时钟
    while(!(CKCON & XLSTA));     //等待 XOSCL 时钟稳定

    LXDIVH = 0;                  //设置时钟分频，分频后 LCD 帧频率为 64Hz
    LXDIVL = 0;
    LXCFG =    DMOD_5ua | BIAS_1_4 | LCDRV_LEV(7);   //设置 LCD 驱动电流、bias、驱动强度
    LXCON =    LEN_XOSCL | LMOD_lcd;                 //选择 LCD 时钟源为 XOSCL、设置为 LCD 模式

    for(i=0;i<8;i++)             //写 LCD 显示缓存
    {
        INDEX = i;               //设置显示缓存索引
        LXDAT = 0;               //写入缓存，当前写 0 为清屏
    }
}
-------------------------------------------------------------------------------
```

18.5 LED 驱动控制例程

例如，要驱动 LED 为 5com×8seg，共阴接法，程序如下：

```
-------------------------------------------------------------------------------
#define XLCKE           (1<<3)
#define XLSTA           (1<<2)
```

```
#define LEN_XOSCL       (3<<5)
#define LMOD_led        (1<<4)
#define CMHV(N)         (N<<5)    //N=0~1
#define SGHV(N)         (N<<4)    //N=0~1
#define BLNK(N)         (N<<3)    //N=0~1
void LED_init(void)
{
    unsigned char i;

    P00F = 3; //设置 P00 为 COM0
    P01F = 3; //设置 P01 为 COM1
    P02F = 3; //设置 P02 为 COM2
    P03F = 3; //设置 P03 为 COM3
    P04F = 3; //设置 P04 为 COM4

    P17F = 3; //设置 P17 为 SEG0
    P37F = 3; //设置 P37 为 SEG1
    P36F = 3; //设置 P36 为 SEG2
    P35F = 3; //设置 P35 为 SEG3
    P34F = 3; //设置 P34 为 SEG4
    P07F = 3; //设置 P07 为 SEG5
    P06F = 3; //设置 P06 为 SEG6
    P05F = 3; //设置 P05 为 SEG7


    CKCON |= XLCKE;             //打开 XOSCL 时钟
    while(!(CKCON & XLSTA));    //等待 XOSCL 时钟稳定

    LXDIVH = 0;                 //设置时钟分频，分频后 LED 帧频率为 256Hz
    LXDIVL = 0;
    LXCFG = CMHV(0) | SGHV(1) | BLNK(0) | LEDRV_LEV(7);    /*设置 COM 低有效，SEG 高有效，全屏闪烁
                                                            关闭，驱动亮度为最大*/
    LXCON =   LEN_XOSCL | LMOD_led;         //选择 LED 时钟源为 XOSCL、设置为 LED 模式

    for(i=0;i<8;i++)            //写 LED 显示缓存
    {
        INDEX = i;              //设置显示缓存索引
        LXDAT = 0;              //写入缓存，当前写 0 为清屏
    }
}
```

---

19 PWM

19.1 PWM 功能简介

MJ603 系列芯片最多有 6 通道 PWM 输出，PWM 周期和占空比可在 16 位范围内任意配置。每路 PWM 都支持边沿对齐模式和中心对称模式。此外，PWM 还支持死区控制及互补输出，设置为互补模式时，6 路 PWM 组成 3 对互补通道。PWM 的这种特性是专门针对无刷直流电机驱动而设计的。

19.2 PWM 功能描述

每路 PWM 通道都有一个专门的 16 位计数器，PWM 的周期通过寄存器 PWMDIV 来设置，而寄存器 PWMDUT 则对应 PWM 的占空比。PWM 通过寄存器 PWMEN 使能，寄存器 PWMEN 的每一位对应 PWM 的一个通道。PWM 模块有一个 PWM 数据更新寄存器 PWMUPD，当改写寄存器 PWMDIV、PWMDUT 和 PWMCKD 时，寄存器 PWMUPD 必须置位相应位才会更新数据，当数据刷新之后 PWMUPD 相应位自动清 0。PWM 可通过 PWMTOG 位设置 PWM 引脚输出反相。PWM 有多种时钟源可以选择，时钟源是以两路 PWM 为单位进行设置的，分别是：PWM0 和 PWM1、PWM2 和 PWM3、PWM4 和 PWM5，也就是说，每组 PWM 的时钟源是共同设置的，时钟源通过 PWM0、PWM2、PWM4 对应的控制寄存器 PWMCON 的 PWMCKS 来选择。另外，每路 PWM 的时钟分频可通过 PWMCKD 独立设置。

- 边沿对齐模式和中心对齐模式

PWM 的边沿对齐模式和中心对齐模式通过 PWMMS 位来选择。PWM 使能后，PWM 计数器从 0 开始累加计数，当计数值小于 PWMDUT 时，PWM 引脚输出高电平（PWMTOG=0），当计数值大于或等于 PWMDUT 时，PWM 引脚输出低电平（PWMTOG=0）。在边沿对齐模式下，当计数值与 PWMDIV 相等时，一个 PWM 周期完成，PWM 计数器重新置 0 并开始下一周期计数。在中心对齐模式下，当计数值达到 PWMDIV 值时，计数方向反转，开始递减计数，在此阶段内，同样是计数值小于 PWMDUT 时，PWM 引脚输出高电平（PWMTOG=0），计数值大于或等于 PWMDUT 时，PWM 引脚输出低电平（PWMTOG=0）；当计数递减到 0 时，一个 PWM 周期完成，计数重新开始下一个周期累加计数。

当边沿对齐模式和中心对齐模式单路 PWM 输出波形见图 19-2-1 和图 19-2-2（注：以下所有 PWM 波形满足条件 PWMDIV>PWMDUT>0）。如图所示，设置相同的 PWMDIV 和 PWMDUT 值，中心对齐模式一个 PWM 周期是边沿对齐模式的两倍。

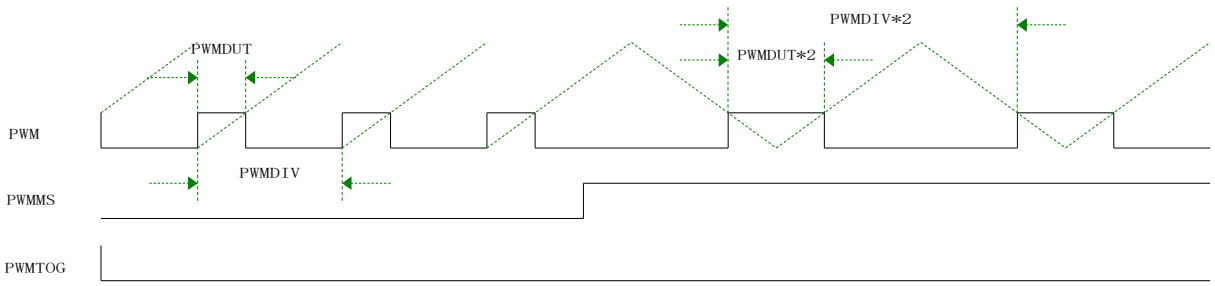


图 **19-2-1　PWMTOG＝0** 时 **PWM** 输出波形

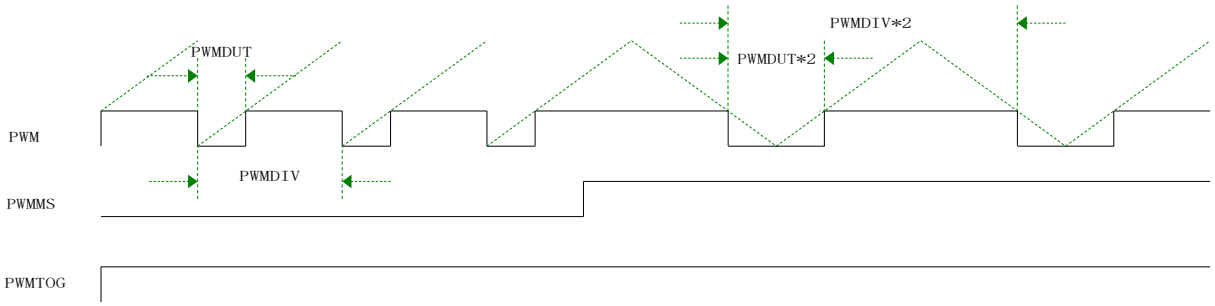


图 **19-2-2　PWMTOG＝1** 时 **PWM** 输出波形

值得注意的是，当 PWMDIV=0 时，PWM 引脚直接输出 PWM 时钟，如果 PWMCKD=0，PWM 引脚输出的是所选的时钟源的时钟信号；当 PWMDIV 不为 0，而 PWMDUT=0 时，PWM 引脚输出低电平（PWMTOG＝0）；当 PWMDUT>=PWMDIV>0 时，PWM 引脚输出高电平（PWMTOG＝0）。

- 互补模式

在互补模式下，6 路 PWM 可组成 3 对互补通道：PWM0 和 PWM1、PWM2 和 PWM3、PWM4 和 PWM5。PWM 的互补模式是通过 PWM1、PWM3、PWM5 的控制寄存器 PWMCON 的 PWMMOD 位设置的。在互补模式，PWM 对齐方式、周期、占空比、预分频时钟都是由 PWM0、PWM2、PWM4 对应的寄存器设置，只有 PWMTOG 仍是由各通道对应寄存器独立控制。PWM 互补模式原理图如图 21-1-3 所示。

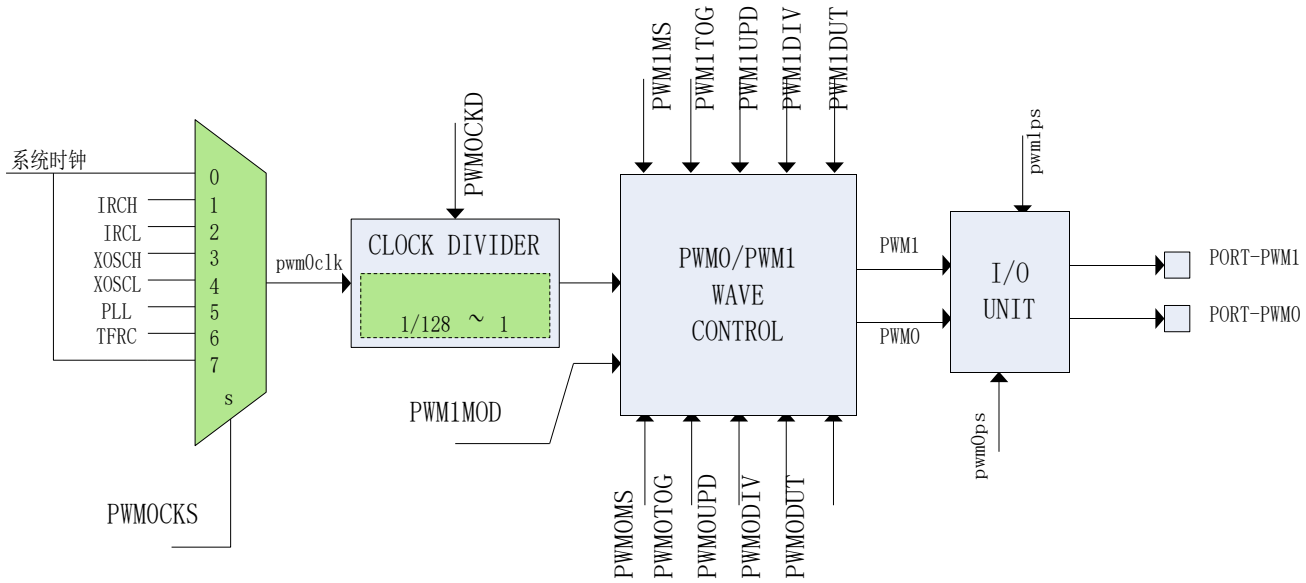


图 **19-2-3　PWM0**、**PWM1** 原理示意图

每组 PWM 输出的波形在相位上互补，如图 19-1-4 和图 19-1-5 所示（以 PWM0、PWM1 为例）。

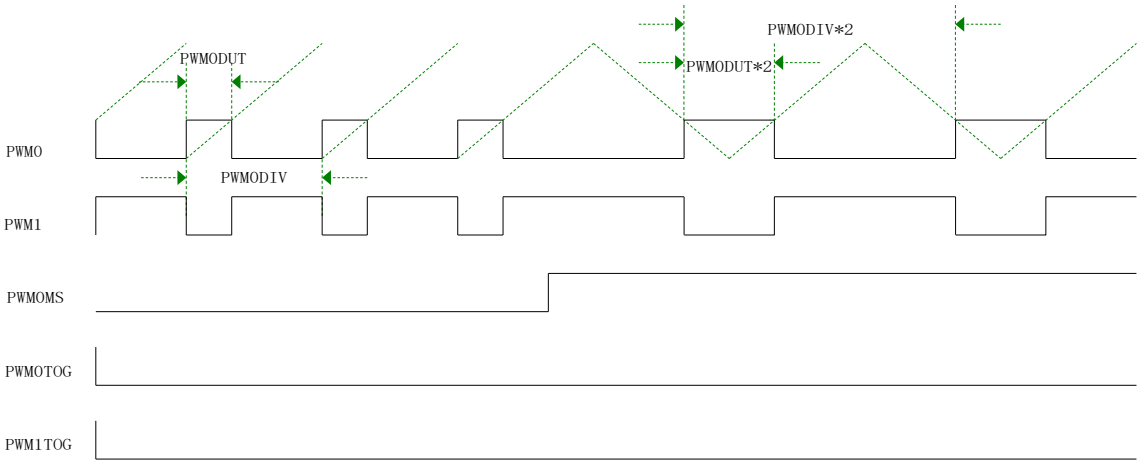


图 **19-2-4　PWMTOG＝0** 时 **PWM0**、**PWM1** 输出互补波形

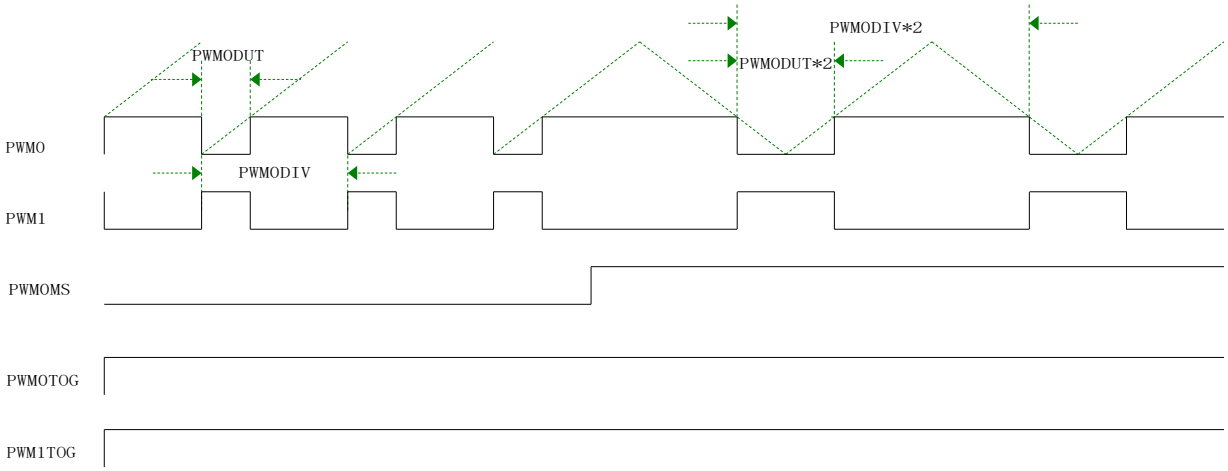


图 **19-2-5　PWMTOG＝1** 时 **PWM0**、**PWM1** 输出互补波形

- 死区控制

在桥形驱动电路中，为防止上下半桥同时导通，需要在 PWM 互补信号中插入死区控制。死区时间由 PWM1、PWM3、PWM5 对应的寄存器 PWMDIV 和 PWMDUT 设置，PWMDIV 设置的是左边的死区时间，而 PWMDUT 设置的是右边的死区时间。设置死区时间需要满足以下条件（以 PWM0、PWM1 为例）： 在边沿对齐模式，PWMDIV1<PWMDUT0 且 PWMDUT1<(PWMDIV0 – PWMDUT0); 在中心对齐模式，PWMDIV1<(PWMDIV0 – PWMDUT0)*2 或 PWMDUT1<(PWMDIV0 – PWMDUT0)*2。

死区控制输出波形如图 21-1-6 所示（以 PWM0、PWM1 为例）。

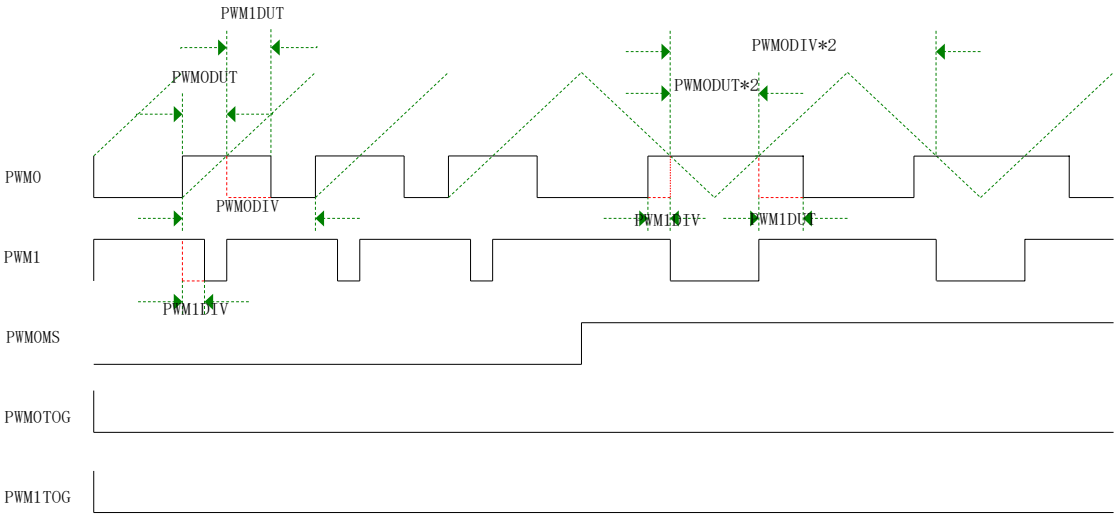


图 **19-2-6　PWMTOG＝0** 时 **PWM0**、**PWM1** 死区控制波形

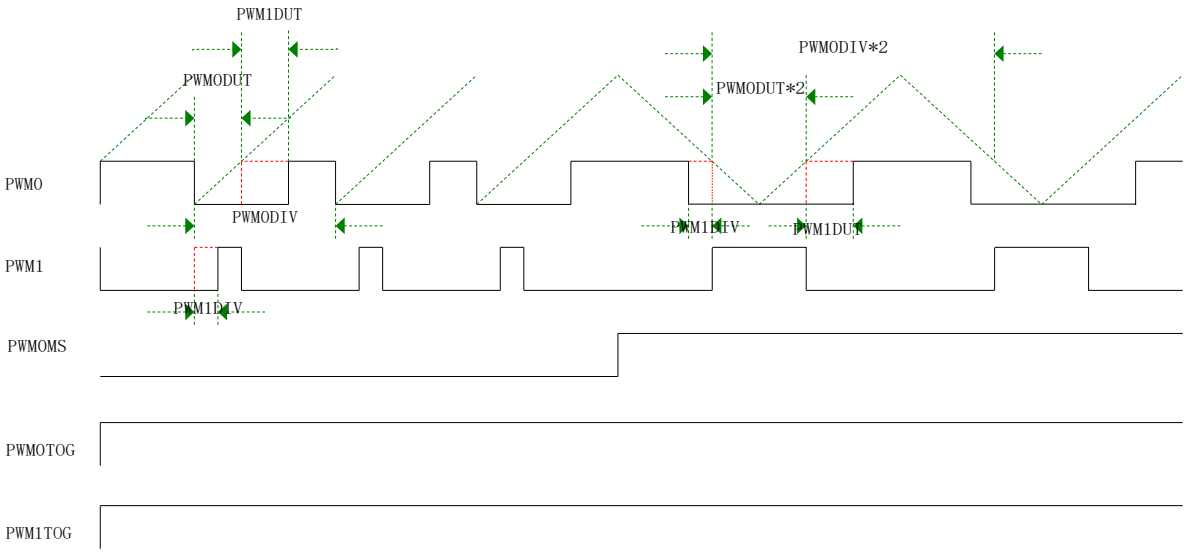


图 **19-2-7　PWMTOG＝1** 时 **PWM0**、**PWM1** 死区控制波形

- **PWM** 中断

PWM 中断通过寄存器 PWMCON 的 PWMTIE、PWMZIE、PWMPIE、PWMNIE 位使能，PWMTIE 位对应的是 PWM 计数器计数到顶点（即等于 PWMDIV)产生的中断，PWMZIE 位对应的是 PWM 计数器计数到最低点（即等于 0）时产生的中断，PWMNIE 对应的是输出引脚下降沿产生的中断，PWMPIE 对应的是输出引脚上升沿产生的中断。其中，在边沿对齐模式，没有 PWMTIE 位和 PWMZIE 位对应的中断。寄存器 PWMAIF、PWMBIF、PWMCIF 是 6 个通道的中断状态寄存器，其中，PWMxTIF、PWMxZIF、PWMxNIF、PWMxPIF 分别对应使能位 PWMTIE、PWMZIE、PWMNIE、PWMPIE。

另外，PWM 可通过寄存器 PWMCMX 设置多少次中断事件发生才产生一次中断，例如设置 PWMCMX=3、PWMPIE=1，那么 PWM 引脚 4 次上升沿才会产生一次上升沿中断。

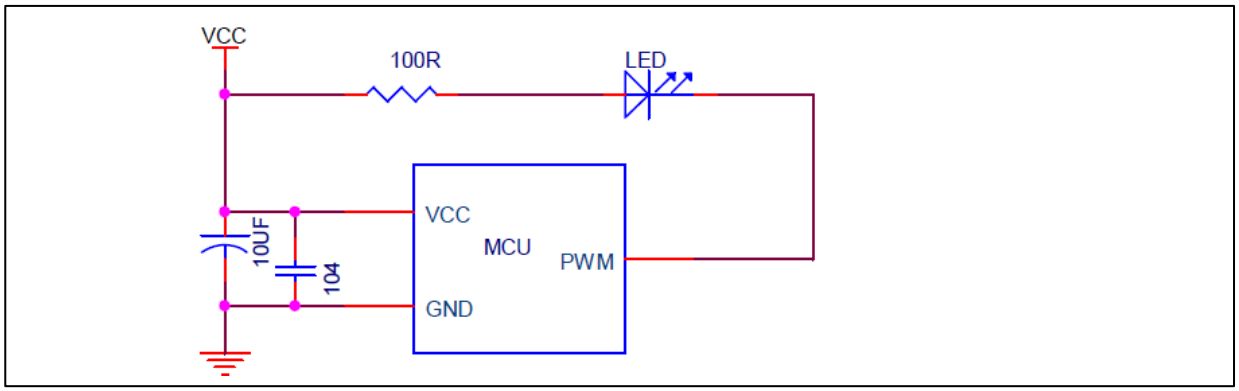


图 **19-2-8** **PWM** 驱动 **LED** 灯参考电路（以上设计电路和元件参数仅供参考）

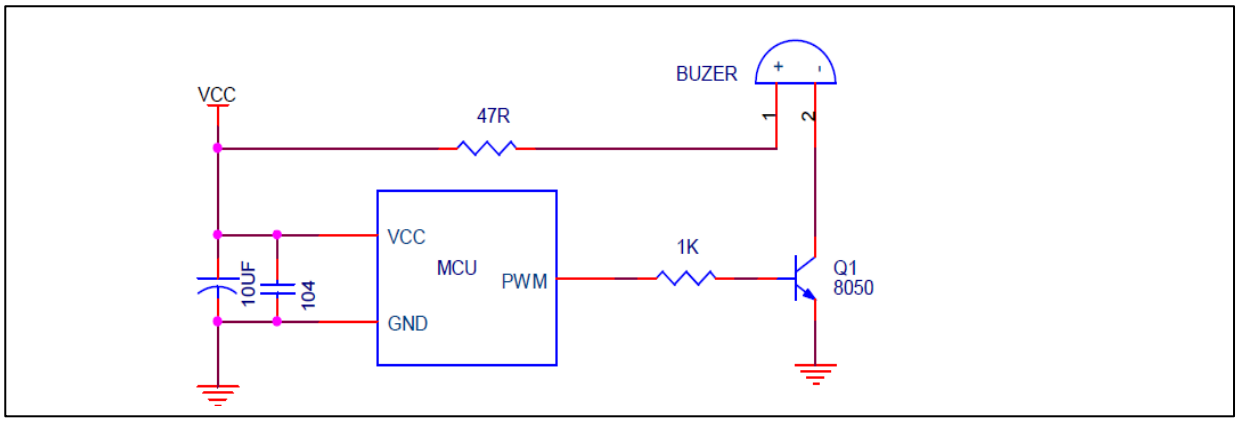


图 **19-2-9** **PWM** 驱动直流蜂鸣器参考电路（以上设计电路和元件参数仅供参考）

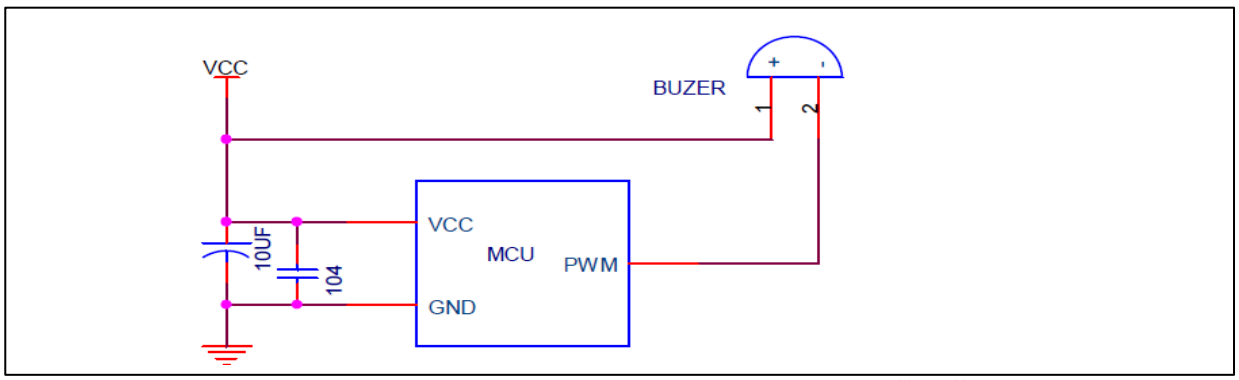
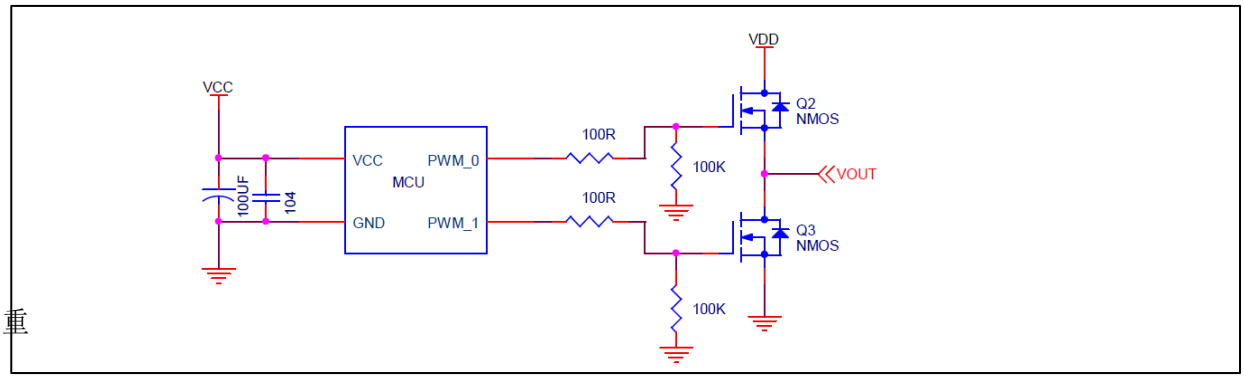


图 **19-2-9** **PWM** 驱动交流蜂鸣器参考电路（以上设计电路和元件参数仅供参考）



重

图 **19-2-9 PWM** 互补参考电路（以上设计电路和元件参数仅供参考）

19.3 PWM 寄存器描述

表 **19-3-1** 寄存器 **PWMEN**

D8H	7	6	5	4	3	2	1	0
PWMEN	-	-	PWMEN[5:0]					
R/W	-	-	R/W					
初始值	0	0	0	0	0	0	0	0
位编号		位符号	说明					
5~0		PWMEN	5~0 位分别对应 PWM 通道 5~0 的使能控制位，1 有效					

表 **19-3-2** 寄存器 **PWMUPD**

D9H	7	6	5	4	3	2	1	0
PWMUPD	-	-	PWMUPD[5:0]					
R/W	-	-	R/W					
初始值	-	-	0	0	0	0	0	0
位编号		位符号	说明					
5~0		PWMUPD	5~0 位对应 PWM 通道 5~0 的数据更新使能控制位，1 有效 备注： 配置某个通道的数据（*PWMDIV/PWMDUT/PWMCKD*）之后，把 *PWMUPD* 对应通道的位置 *1*，这样才会使数据在 *PWM* 计数器溢出之后更新进去，而对应位在数据更新完成之后将会自动清 *0*。					

表 **19-3-3** 寄存器 **PWMCMX**

DAH	7	6	5	4	3	2	1	0
PWMCMX	PWMCMX[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

备注：*PWMCMX* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *PWMCMX0~PWMCMX5*

位编号	位符号	说明
7~0	PWMCMX	PWM 各通道中断有效的间隔次数设置寄存器。 间隔次数=PWMCMX+1，例如设置 INDEX=0、PWMCMX=7，那么 PWM0 所有中断都会产生 8 次中断事件时才会置位中断标志。

表 **19-3-4** 寄存器 **PWMCON**

DBH	7	6	5	4	3	2	1	0
PWMCON①	PWMTIE	PWMZIE	PWMPIE	PWMNIE	PWMMS	PWMCKS[2:0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W		
初始值	0	0	0	0	0	0	0	0
PWMCON②	PWMTIE	PWMZIE	PWMPIE	PWMNIE	PWMMS	-	-	PWMMOD
R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W
初始值	0	0	0	0	0	-	-	0

位编号	位符号	说明
备注： *PWMCON*①是 *PWM0/PWM2/PWM4* 通道的控制寄存器 *PWMCON*②是 *PWM1/PWM3/PWM5* 通道的控制寄存器 *PWMCON* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *PWMCON0~PWMCON5*		
7	PWMTIE	PWM 计数器顶点中断使能控制位，1 有效
6	PWMZIE	PWM 计数器最低点中断使能控制位，1 有效
5	PWMPIE	PWM 上升沿中断使能控制位，1 有效
4	PWMNIE	PWM 下降沿中断使能控制位，1 有效
3	PWMMS	PWM 模式选择位 0：边沿对齐模式 1：中心对齐模式
2~0	PWMCKS	PWM 工作时钟选择位 001：IRCH 010：IRCL 011：ERC 100：XOSCL 101：PLL 110：TFRC 其他：系统时钟 备注：

		PWM0/PWM1，都由 *PWMCKS0* 来配置； *PWM2/PWM3*，都由 *PWMCKS2* 来配置； *PWM4/PWM5*，都由 *PWMCKS4* 来配置；
0	PWMMOD	互补模式使能寄存器，1 有效 备注： 设置 *PWMMOD1=1*，*PWM0*、*PWM1* 进入互补模式 设置 *PWMMOD3=1*，*PWM2*、*PWM3* 进入互补模式 设置 *PWMMOD5=1*，*PWM4*、*PWM5* 进入互补模式

表 **19-3-5** 寄存器 **PWMCFG**

DCH	7	6	5	4	3	2	1	0
PWMCFG	PWMTOG	PWMCKD[6:0]						
R/W	R/W	R/W						
初始值	0	0	0	0	0	0	0	0
备注：*PWMCFG* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *PWMCFG0~PWMCFG5*								
位编号	位符号	说明						
7	PWMTOG	PWM 输出取反使能寄存器，1 有效						
6~0	PWMCKD	PWM 工作时钟预分频配置寄存器 0000000：不分频 0000001：2 分频 0000010：3 分频 ..... 1111110：127 分频 1111111：128 分频						

表 **19-3-6** 寄存器 **PWMDIVL**、**PWMDIVH**

DDH	7	6	5	4	3	2	1	0
PWMDIVL	PWMDIV[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
DEH	7	6	5	4	3	2	1	0
PWMDIVH	PWMDIV[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
备注：*PWMDIV* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *PWMDIV0~PWMDIV5*								

位编号	位符号	说明
15~0	PWMDIV	PWM 周期配置寄存器 在互补模式下，PWMDIV1/PWMDIV3/PWMDIV5 有不同的含义，参考寄存器 PWMDUT 相关描述

表 **19-3-7** 寄存器 **PWMDUTL**、**PWMDUTH**

DFH	7	6	5	4	3	2	1	0
PWMDUTL	PWMDUT[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
D1H	7	6	5	4	3	2	1	0
PWMDUTH	PWMDUT[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

备注：*PWMDUT* 是带索引的寄存器，设置 *INDEX=0~7* 分别对应 *PWMDUT0~PWMDUT5*

位编号	位符号	说明
15~0	PWMDUT	PWM 占空比配置寄存器 在互补模式下， PWMDUT1/PWMDUT3/PWMDUT5 有不同的含义，如下 表：

PWMDIV1	控制 PWM0/PWM1 的左边的死区的宽度
PWMDUT1	控制 PWM0/PWM1 的右边的死区的宽度
PWMDIV3	控制 PWM2/PWM3 的左边的死区的宽度
PWMDUT3	控制 PWM2/PWM3 的右边的死区的宽度
PWMDIV5	控制 PWM4/PWM5 的左边的死区的宽度
PWMDUT5	控制 PWM4/PWM5 的右边的死区的宽度

表 **19-3-8** 寄存器 **PWMAIF**

D2H	7	6	5	4	3	2	1	0
PWMAIF	PWM1TIF	PWM1ZIF	PWM1PIF	PWM1NIF	PWM0TIF	PWM0ZIF	PWM0PIF	PWM0NIF
R/W	R	R	R	R	R	R	R	R
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	PWM1TIF	PWM1 计数器顶点中断标志位，写 1 清 0
6	PWM1ZIF	PWM1 计数器最低点中断标志位，写 1 清 0
5	PWM1PIF	PWM1 上升沿中断标志位，写 1 清 0
4	PWM1NIF	PWM1 下降沿中断标志位，写 1 清 0
3	PWM0TIF	PWM0 计数器顶点中断标志位，写 1 清 0
2	PWM0ZIF	PWM0 计数器最低点中断标志位，写 1 清 0
1	PWM0PIF	PWM0 上升沿中断标志位，写 1 清 0
0	PWM0NIF	PWM0 下降沿中断标志位，写 1 清 0

表 **19-3-9** 寄存器 **PWMBIF**

D3H	7	6	5	4	3	2	1	0
PWMBIF	PWM3TIF	PWM3ZIF	PWM3PIF	PWM3NIF	PWM2TIF	PWM2ZIF	PWM2PIF	PWM2NIF
R/W	R	R	R	R	R	R	R	R
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	PWM3TIF	PWM3 计数器顶点中断标志位，写 1 清 0
6	PWM3ZIF	PWM3 计数器最低点中断标志位，写 1 清 0
5	PWM3PIF	PWM3 上升沿中断标志位，写 1 清 0
4	PWM3NIF	PWM3 下降沿中断标志位，写 1 清 0
3	PWM2TIF	PWM2 计数器顶点中断标志位，写 1 清 0
2	PWM2ZIF	PWM2 计数器最低点中断标志位，写 1 清 0
1	PWM2PIF	PWM2 上升沿中断标志位，写 1 清 0
0	PWM2NIF	PWM2 下降沿中断标志位，写 1 清 0

表 **19-3-10** 寄存器 **PWMCIF**

D4H	7	6	5	4	3	2	1	0
PWMCIF	PWM5TIF	PWM5ZIF	PWM5PIF	PWM5NIF	PWM4TIF	PWM4ZIF	PWM4PIF	PWM4NIF
R/W	R	R	R	R	R	R	R	R
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7	PWM5TIF	PWM5 计数器顶点中断标志位，写 1 清 0
6	PWM5ZIF	PWM5 计数器最低点中断标志位，写 1 清 0
5	PWM5PIF	PWM5 上升沿中断标志位，写 1 清 0
4	PWM5NIF	PWM5 下降沿中断标志位，写 1 清 0
3	PWM4TIF	PWM4 计数器顶点中断标志位，写 1 清 0
2	PWM4ZIF	PWM4 计数器最低点中断标志位，写 1 清 0
1	PWM4PIF	PWM4 上升沿中断标志位，写 1 清 0
0	PWM4NIF	PWM4 下降沿中断标志位，写 1 清 0

19.4 PWM 功能控制例程

◆ **PWM** 单路输出例程

例如，PWM0 输出频率为 30K 的时钟，占空比为 30%，程序如下：

```
---------------------------------------------------------------------------------------------
#define PWM_CH0       0

#define TIE(N)            (N<<7)    //N=0~1
#define ZIE(N)            (N<<6)    //N=0~1
#define PIE(N)            (N<<5)    //N=0~1
#define NIE(N)            (N<<4)    //N=0~1
#define MS(N)             (N<<3)    //N=0~1

#define CKS_IH                (1<<0)

#define TOG(N)                (N<<7)    //N=0~1

void PWM_init(void)
{
    P15F = 6;                 //设置 P15 为 PWM 引脚
    INDEX = PWM_CH0;      //设置 INDEX 指向 PWM0
    PWMCON = TIE(0) | ZIE(0) | PIE(0) | NIE(0) | MS(0) | CKS_IH ;    /*设置 PWM0 中断关闭，设置 PWM0
                                                                    为边沿对齐模式，设置 PWM0 时钟源为 IRCH*/
    PWMCFG = TOG(0) | 0;  //设置反相关闭，时钟不分频

    PWMDIVH     = 0;
    PWMDIVL     = 123;    //3686400/30000=123
    PWMDUTH     = 0;
    PWMDUTL     = 37;     //123*0.3=37

    PWMUPD  |= (1<<PWM_CH0);   //PWM 设置更新
    while(PWMUPD);   //等待 PWM 设置更新完成，在使能 PWM 之前必须要执行此操作

    PWMEN   |= (1<<PWM_CH0);      //PWM0 使能
}
---------------------------------------------------------------------------------------------
```

例如，PWM0 要直接输出 IRCH 的时钟，程序如下：

```
---------------------------------------------------------------------------------------------
void PWM_init(void)
{
```

```
    P15F = 6;             //设置 P15 为 PWM 引脚
    INDEX = PWM_CH0;      //设置 INDEX 指向 PWM0
    PWMCON = TIE(0) | ZIE(0) | PIE(0) | NIE(0) | MS(0) | CKS_IH ;   /*设置 PWM0 中断关闭，设置 PWM0
                                                                    为边沿对齐模式，设置 PWM0 时钟源为 IRCH*/
    PWMCFG = TOG(0) | 0;  //设置反相关闭，时钟不分频

    PWMDIVH     = 0;  //PWMDIV 和 PWMDUT 都设为 0 则直接输出时钟源时钟
    PWMDIVL     = 0;
    PWMDUTH     = 0;
    PWMDUTL     = 0;

    PWMUPD  |= (1<<PWM_CH0);   //PWM 设置更新
    while(PWMUPD);   //等待 PWM 设置更新完成，在使能 PWM 之前必须要执行此操作

    PWMEN   |= (1<<PWM_CH0);     //PWM0 使能
}
-------------------------------------------------------------------------------------------
```

◆ **PWM** 互补输出和死区控制例程

以 PWM0,PWM1 为例，此两路 PWM 输出互补的频率为 30K 的时钟，占空比为 50%，同时插入 2 个时钟周期的死　　区时间，程序如下：

```
-------------------------------------------------------------------------------------------
#define PWM_CH0       0
#define PWM_CH1       1

#define TIE(N)            (N<<7)    //N=0~1
#define ZIE(N)            (N<<6)    //N=0~1
#define PIE(N)            (N<<5)    //N=0~1
#define NIE(N)            (N<<4)    //N=0~1
#define MS(N)             (N<<3)    //N=0~1

#define CKS_IH            (1<<0)

#define TOG(N)            (N<<7)    //N=0~1
#define MOD(N)            (N<<0)    //N=0~1

void PWM_init(void)
{
    P15F = 6;             //设置 P15 为 PWM0 引脚
    P16F = 6;             //设置 P15 为 PWM1 引脚

    INDEX = PWM_CH0;            //设置 INDEX 指向 PWM0
    PWMCON = TIE(0) | ZIE(0) | PIE(0) | NIE(0) | MS(0) | CKS_IH ;   /*设置 PWM0 中断关闭，设置 PWM0
                                                                    为边沿对齐模式，设置 PWM0 时钟源为 IRCH*/
```

```
    PWMCFG = TOG(0) | 0;        //设置反相关闭，时钟不分频

    PWMDIVH     = 0;
    PWMDIVL     = 123;          //3686800/30000=123
    PWMDUTH     = 0;
    PWMDUTL     = 61;           //123*0.5=61

    INDEX = PWM_CH1;            //设置 INDEX 指向 PWM1
    PWMCON = TIE(0) | ZIE(0) | PIE(0) | NIE(0) | MS(0) | MOD(1) ;  //设置 PWM0、PWM1 为互补模式
    PWMCFG = TOG(0) | 0;        //设置反相关闭，时钟不分频

    PWMDIVH     = 0;
    PWMDIVL     = 2;        //左边插入 2 个时钟周期死区，如不需插入死区时间，则设置为 0
    PWMDUTH     = 0;
    PWMDUTL     = 2;        //右边插入 2 个时钟周期死区，如不需插入死区时间，则设置为 0

    PWMUPD  |= (1<<PWM_CH0)|(1<<PWM_CH1);     //PWM 设置更新
    while(PWMUPD);          //等待 PWM 设置更新完成，在使能 PWM 之前必须要执行此操作

    PWMEN   |= (1<<PWM_CH0)|(1<<PWM_CH1);//PWM0、PWM1 使能
}
-------------------------------------------------------------------------------
```

◆ **PWM** 中断例程

例如，PWM0 设为中心对齐模式，开启顶点、底点、上升沿和下降沿中断，程序如下：

```
-------------------------------------------------------------------------------
#define PWM_CH0     0

#define TIE(N)          (N<<7)  //N=0~1
#define ZIE(N)          (N<<6)  //N=0~1
#define PIE(N)          (N<<5)  //N=0~1
#define NIE(N)          (N<<4)  //N=0~1
#define MS(N)           (N<<3)  //N=0~1

#define CKS_IH              (1<<0)

#define TOG(N)              (N<<7)  //N=0~1

void PWM_init(void)
{
    P15F = 6;               //设置 P15 为 PWM 引脚
    INDEX = PWM_CH0;        //设置 INDEX 指向 PWM0
    PWMCON = TIE(1) | ZIE(1) | PIE(1) | NIE(1) | MS(1) | CKS_IH ;   /*设置 PWM0 中断关闭，设置 PWM0
                                                                    为中心对齐模式，设置 PWM0 时钟源为 IRCH*/
    PWMCFG = TOG(0) | 0;  //设置反相关闭，时钟不分频
```

```
    PWMDIVH     = 0;
    PWMDIVL     = 123;    //3686400/30000=123
    PWMDUTH     = 0;
    PWMDUTL     = 37;     //123*0.3=37

    PWMUPD  |= (1<<PWM_CH0);   //PWM 设置更新
    while(PWMUPD);             //等待 PWM 设置更新完成，在使能 PWM 之前必须要执行此操作

    PWMEN   |= (1<<PWM_CH0);   //PWM0 使能

    PWMCMAX = 0;               //设置每个 PWM 周期都产生中断
    INT9EN = 1;                //使能 INT9 中断
}
void INT9_ISR(void) interrupt 14
{
    if(PWMAIF & TIF0)          //边沿对齐模式无效
    {
        PWMAIF = TIF0;
        //顶点中断服务程序
        ...
    }
    if(PWMAIF & ZIF0)          //边沿对齐模式无效
    {
        PWMAIF = ZIF0;
        //底点中断服务程序
        ...
    }
    if(PWMAIF & PIF0)
    {
        PWMAIF = PIF0;
        //上升沿中断服务程序
        ...
    }
    if(PWMAIF & NIF0)
    {
        PWMAIF = NIF0;
        //下降沿中断服务程序
        ...
    }
        ......
}
```

---

20 模/数字转换器（ADC）

20.1 功能简介

模拟/数字转换器是 12 位逐次逼近寄存器型（SAR）ADC，最多提供多达 8 个输入通道。ADC 时钟源是系统时钟，可设置时钟预分频。ADC 有多种参考电压源可选，其中选择内部电压为参考电压时可用于检测芯片供电电压。内置比较器模式，可设定比较器阈值，超出阈值可产生中断。ADC 选择内部电压为参考电压时，内部电压可校正（需用户加载校正参数），避免芯片一致性问题。ADC 和运放结合一起使用，可以把检测信号缩小后再进行转换。

20.2 主要特性

- 12 位的分辨率
- 最多提供多达 8 个输入通道
- 支持 ADC 中断
- 可设置 ADC 时钟预分频
- 多种参考电压可选：内部参考电压、VDD、外部参考电压。
- 支持 VDD 和参考地电压的测量
- 内置运放，支持检测信号缩小，缩小倍数可选
- 输入电压范围：VSS<=VIN<=VDD。

20.3 结构框图

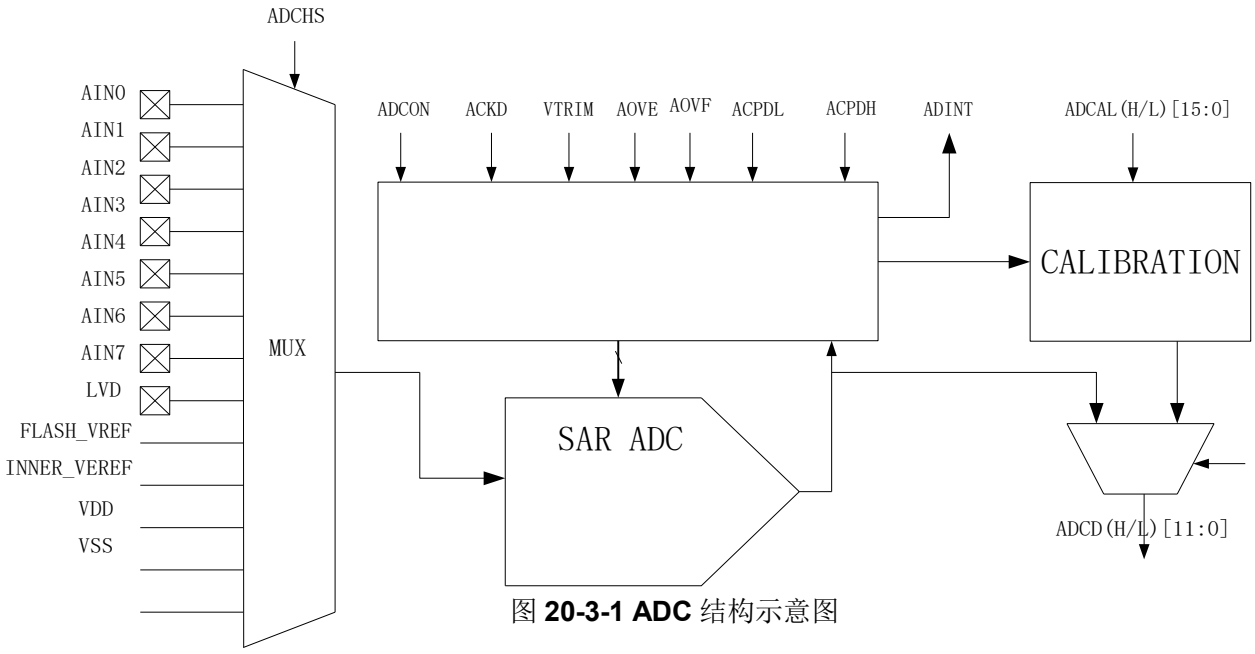


图 **20-3-1 ADC** 结构示意图

20.4 功能描述

ADC 的启动通过 AST 位使能，设置 AST=1 后，ADC 模块对 ADCHS 选择的输入电压源进行模/数转换。ADC 可通过 ACKD 设置时钟预分频，由系统时钟预分频后的时钟作为 ADC 转换时钟。在 ADC 时钟不变的条件下，ADC 的单次转换时间是由 HTME 设置的，转换时间为(13+2^HTME)个 ADC 时钟周期。当转换结束后，12 位的 A/D 值会被加载到寄存器 ADCDH、ADCDL，转换完后的 2.5 个时钟周期，AST 位自动清 0，同时中断标志 ADIF 位会置 1，如果 ADC 中断使能，会产生 ADC 中断。ADC 最小的转换时间为 0.5us。图 22-4-1 为 ADC 的转换时序图。

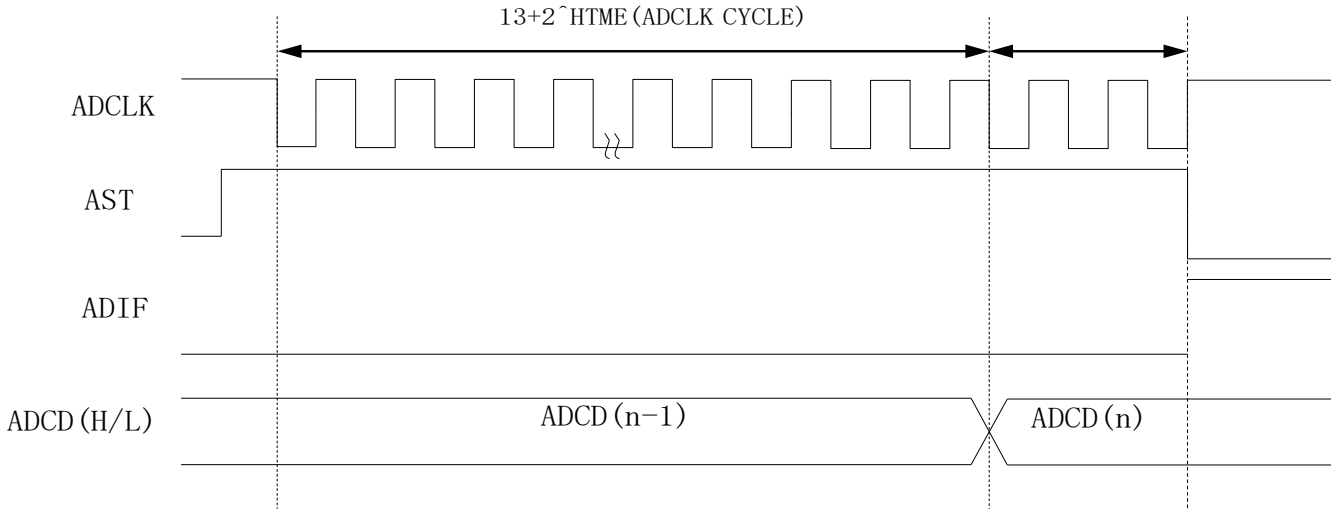


图 **20-4-1 ADC** 时序示意图

- **ADC** 内部参考电压校正

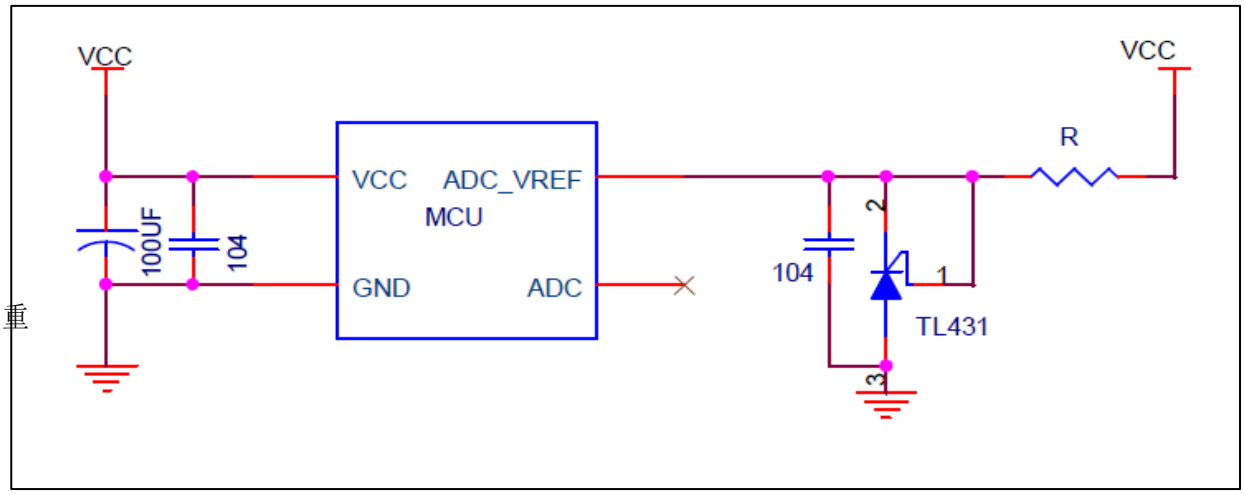
  ADC 内部参考电压标准为 1.5V，但是由于制程的影响，此电压与理论值有一定的偏差，需要在应用时进行校正。芯片内置电压校正机制，可通过设置 VTRIM(ADCFGH[5:0])进行微调，芯片在 CP 测试阶段已对此电压进行校正，校正参数存储于芯片内部信息区，用户只需要在应用程序中把此参数加载到 VTRIM 即可。校正参数加载程序如下所示：

```
---------------------------------------------------------------------------------------------------------------------------
unsigned char read_inner_trim(void)
{
    unsigned char value;
    FSCMD = 0x80;
    PTSH = 0x00;
    PTSL = 0x24;
    FSCMD = 0x81;
    value = FSDAT;
    FSCMD = 0;
    return (value&0x3f);    //返回 ADC 内部电压校正值
}
void adc_init(void)
{
```

```
.....
ADCFGH = (ADCFGH&0xc0)| read_inner_trim(); //把 ADC 内部电压校正值加载到寄存器
.....
}
```

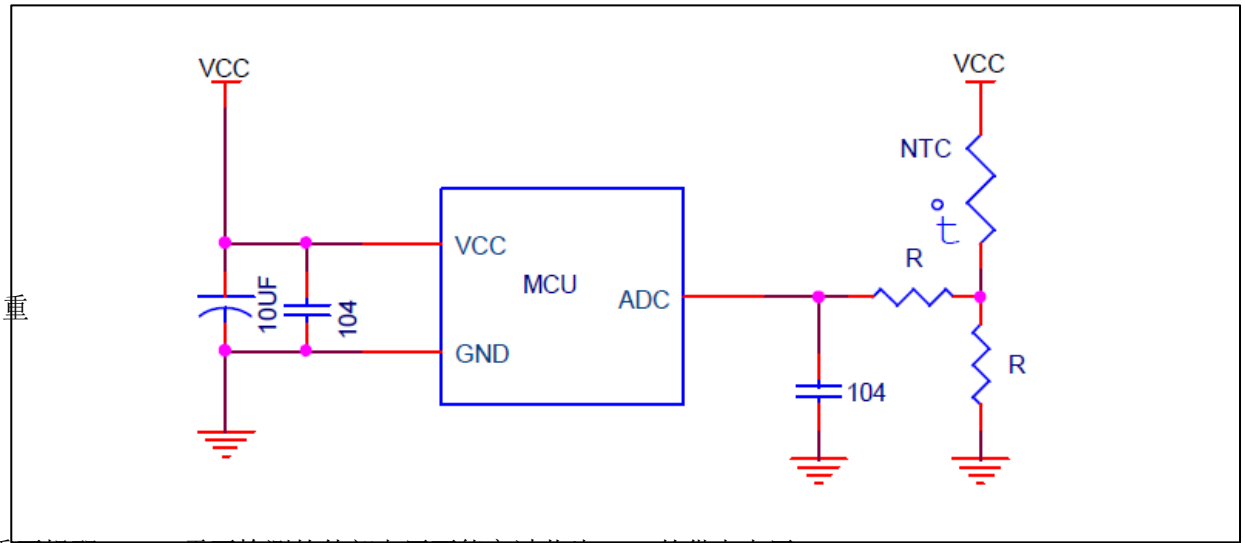
- 运放功能

  运放功能支持对 ADC 检测信号进行缩小，可有效扩展检测信号范围。缩小倍数由 AOPS 设置，详细请参考寄存器部分描述。



重

图 **20-4-2** **ADC** 外部基准应用参考电路（以上设计电路和元件参数仅供参考）



重

重要提醒：*ADC* 需要检测的外部电压不能高过芯片 *VCC* 的供电电压。

图 **20-4-3** **ADC** 外部基准应用参考电路（以上设计电路和元件参数仅供参考）

20.5 寄存器描述

表 **20-5-1** 寄存器 **ADCON**

B9H	7	6	5	4	3	2	1	0
ADCON	AST	ADIE	ADCIF	HTME			VSEL[1:0]	
R/W	R/W	R/W	R/W	R/W			R/W	
初始值	0	0	0	0	1	0	0	0

位编号	位符号	说明
7	AST	ADC 转换开始控制位，写 1 启动转换，转换后硬件自动清 0
6	ADIE	ADC 中断使能位，1 有效
5	ADCIF	ADC 中断标志位，写 1 清 0
4~2	HTME	采样保持周期数为 2 的 HTME 次幂
1~0	VSEL	ADC 参考电压选择位 00: 内部 1.5V(INNER_VREF)作为参考电压 01: 外部 VDD 10: 外部 VREF 11: 内部 1.5V(INNER_VREF)作为参考电压 备注：当参考电压选择为外部 *VREF* 时，*VREF* 的电压必须大于 *1.1V*。

表 **20-5-2** 寄存器 **ADCFGL**

BAH	7	6	5	4	3	2	1	0
ADCFGL	ACKD			-	ADCHS			
R/W	R/W			-	R/W			
初始值	0	0	0	-	0	0	0	0

位编号	位符号	说明
7~5	ACKD	ADC 时钟分频设置 000：不分频 001：2 分频 010：4 分频 … 111：14 分频
4	-	-
3~0	ADCHS	ADC 通道使能选择位域 0000：通道关闭 0001：通道 AD_CH[0](P10)使能 0010：通道 AD_CH[1](P11)使能 0011：通道 AD_CH[2](P12)使能 0100：通道 AD_CH[3](P13)使能 0101：通道 AD_CH[4](P14)使能

		0110：通道 AD_CH[5](P15)使能 0111：通道 AD_CH[6](P16)使能 1000：通道 AD_CH[7](P17)使能 1001：检测 VDD 的 1/4 使能 1010：检测 VSS 使能 其他：通道关闭

表 **20-5-3** 寄存器 **ADCFGH**

BBH	7	6	5	4	3	2	1	0
ADCFGH	AOPS[1:0]		VTRIM					
R/W	R/W		R/W					
初始值	0	0	1	0	0	0	1	1
位编号	位符号		说明					
7~6	AOPS		运放放大倍数选择位域 00：无缩放 01：1/4 倍 10：1/3 倍 11：1/2 倍					
5~0	VTRIM		内部 1.5V 参考电压校正寄存器，校正精度±1mV					

表 **20-5-4** 寄存器 **ADCD**

BCH	7	6	5	4	3	2	1	0
ADCDL	ADCDL[3:0]				-			
R/W	R				-			
初始值	0	0	0	0	-	-	-	-
BDH	7	6	5	4	3	2	1	0
ADCDH	ADCDH[11:4]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
位编号	位符号		说明					
11~0	ADCD		ADC 转换值					

20.6 ADC 控制例程

例如，设置 ADC 参考电压为外部 VDD，采集 ADC 通道 0，ADC 中断开启，程序如下：

```
-------------------------------------------------------------------------------------------
#define AST(N)          (N<<7)    //N=0~1
#define ADIE(N)         (N<<6)    //N=0~1
#define ADIF            (1<<5)    //中断标志
#define HTME(N)         (N<<2)    //N=0~7          //采样时间设置，时间为 2 的 HTME 次方的时钟周期
#define VSEL(N)         (N)       //N=0~3          //选择参考电压 0-内部 1-VDD  2-外部

#define ACKD(N)         (N<<5)   //N=0~7
#define ADCALE(N)       (N<<4)    //N=0~1
#define ADCHS(N)        (N)  //N=0~15       //ADC 通道选择， 1~13 对应通道 0~12
void ADC_init(void)
{
    P10F = 5; //设置 P10 为 ADC 引脚
    ADCON = AST(0) | ADIE(1) | HTME(7) | VSEL(1);  //使能 ADC 中断，设置采样周期，选择 VDD 为参考电压
    ADCFGL = ACKD(1) | ADCHS(1);                   //设置 ADC 时钟分频，选择 ADC 通道为 ADC0
    ADCON |= AST(1);                               //启动 AD 转换
    INT2EN = 1;                                    //INT2 中断使能
}
void ADC_ISR (void) interrupt 7
{
    unsigned int AD_Value;
    if(ADCON & ADIF)
    {
        ADCON |= ADIF;                  //清除 ADC 中断标志
        AD_Value = ADCDH*256 + ADCDL;   //读取 ADC 值
        AD_Value >>= 4;
        ADCON |= AST(1);                //启动下一次 AD 转换
    }
    ......
}
-------------------------------------------------------------------------------------------------------------------------------
```

21 触摸按键（TOUCH KEY）

21.1 功能简介

MJ603 系列芯片的触摸功能模块具有优越的抗干扰性能，可通过 EFT、CS 等测试。触摸模块最大可支持 多达 20 个通道，在应用时 TK_CAP 引脚需接一个 Cx 电容，容值范围 10nF~47nF，电容精度 10%以内，建议 使用涤纶电容、X7R 材质电容或 NPO 材质贴片电容。Cx 可直接影响触摸灵敏度，Cx 容值越小，灵敏度越低， 容值越大，灵敏度越高。

针对有低功耗需求的应用，还设计了芯片在 STOP 模式时仍能正常工作的机制。

21.2 主要特性

- 高抗干扰性能，符合 EMC(CS)标准
- 最大支持 20 个通道
- 支持低功耗模式
- 支持触摸中断
- 支持充放电时钟预分频
- 支持手动和自动启动模式
- 比较器阈值有多级可选
- 触摸可设置内部充电和内部基准，可有效抑制电源低频干扰
- 支持触摸引脚与 LED 驱动引脚复用
- 内置防水补偿机制
- STOP 模式下可设自动唤醒阈值

21.3 结构图

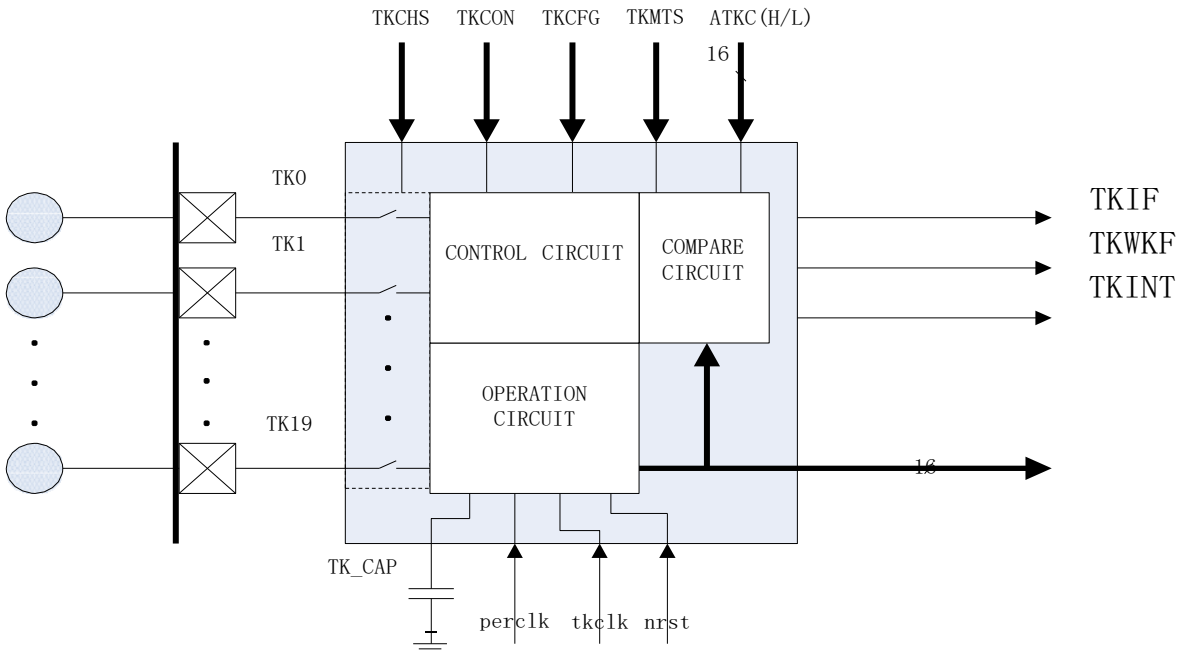


图 **21-3-1** 触摸模块结构图

21.4 功能描述

21.4.1 手动模式和自动模式

在手动模式下，触摸数据采集通过 TKST 位启动。当设置 TKST=1 后，触摸控制开始采集选定通道的触摸数据。通道的选择是以最多 6 个通道为一组的，通过索引寄存器 INDEX 及寄存器 TKCHS 进行设置，每次启动会一次采集完一组通道。当数据采集完成后，TKST 位自动清 0，相应通道的中断标志位 TKIF 置 1，此时可通过设置索引寄存器 INDEX 后从寄存器 TKMS 读取触摸数据。

手动模式和自动模式通过 TMEN 位选择，和手动模式不同的是，自动模式的触摸数据采集是由定时器定时启动的，定时器的时钟源可以是 IRCL 或 XOSCL，由寄存器 CKSEL 的 RTCKS 位选择；定时器定时时间由寄存器 TKMTS 设置。

21.4.2 触摸时钟预分频

触摸控制器对触摸电极充放电的时钟源是 TFRC，充放电的时钟频率对触摸的性能至关重要，当充放电频率太高时，有可能造成对触摸电极的充电不充分从而导致手指触摸时触摸数据变化量变小。触摸时钟预分频通过 TKDIV 进行设置，通过设置合理的值可以使触摸的性能更优。

21.4.3 低功耗模式

为了实现触摸功能的低功耗应用，触摸模块设计了相应的省电机制。在 STOP 模式下，只要触摸的充放电时钟源 TFRC 和低速时钟（IRCL 或 XOSCL）处于开启状态，触摸模块就可以保持正常的充放电和计数。当触摸采集完成后，如果 TWKE=0，触摸采集完成中断会唤醒 CPU，软件在 CPU 唤醒之后可以读取触摸数据，然后再次进入 STOP 模式。另外，触摸模块还设计了触摸阈值自动比较功能，用户可通过阈值设置寄存器设置一组通道的触发阈值，在 STOP 模式，触摸控制器仍然可以将采集的触摸数据和阈值进行比较，当触摸数据超过阈值时，如果 TWKE=1，会产生阈值触发中断并唤醒 CPU，CPU 被唤醒后就可以进行正常的触摸采集和判断。

21.4.4 触摸按键共用 LED 驱动

触摸按键共用 LED 驱动可实现 N 个触摸按键和 N 个触摸指示灯控制只需要(N+1)个引脚。其中，触摸按键和 LED 驱动正端控制共用引脚，LED 负端接 COM，触摸和 LED 控制采用分时的方式实现。

每一路触摸都有单独的控制位 TLENx(x=0~19,对应 TK0~TK19)来使能共用 LED 驱动功能，需要注意的是，对应的触摸引脚功能必须开启。共用 LED 使能后，TLDATx(x=0~19，对应 LED0~LED19)可独立控制每路 LED 灯亮灭。COM 引脚可通过 TLCOMS 来选择。

触摸数据采集和 LED 控制采用分时的方式实现，其中触摸数据采集的时间由 TLCNTK 定义，而 LED 扫描的时间由 TLCNTL 定义。注意，TLCNTK 定义的时间是每组触摸采集的总时间，每组触摸通道数量为 1~6，当实际触摸的时间大于定义的时间时，会产生 TLERR 中断。当计数器计数到 TLCNTK 定义的时间时，产生 TLKOV 中断。触摸采集阶段完成后，便进入 LED 扫描阶段。TLCNTL 定义了 LED 扫描阶段的时间，此时间会影响 LED 扫描的占空比，也就是会影响 LED 的亮度，在应用时可以根据需要调整。在 LED 扫描阶段，当计数器计到 TLCNTL 定义的时间时，会产生 TLLOV 中断，至此，一个完整的触摸共用 LED 周期完成。以下为工作阶段示意图。

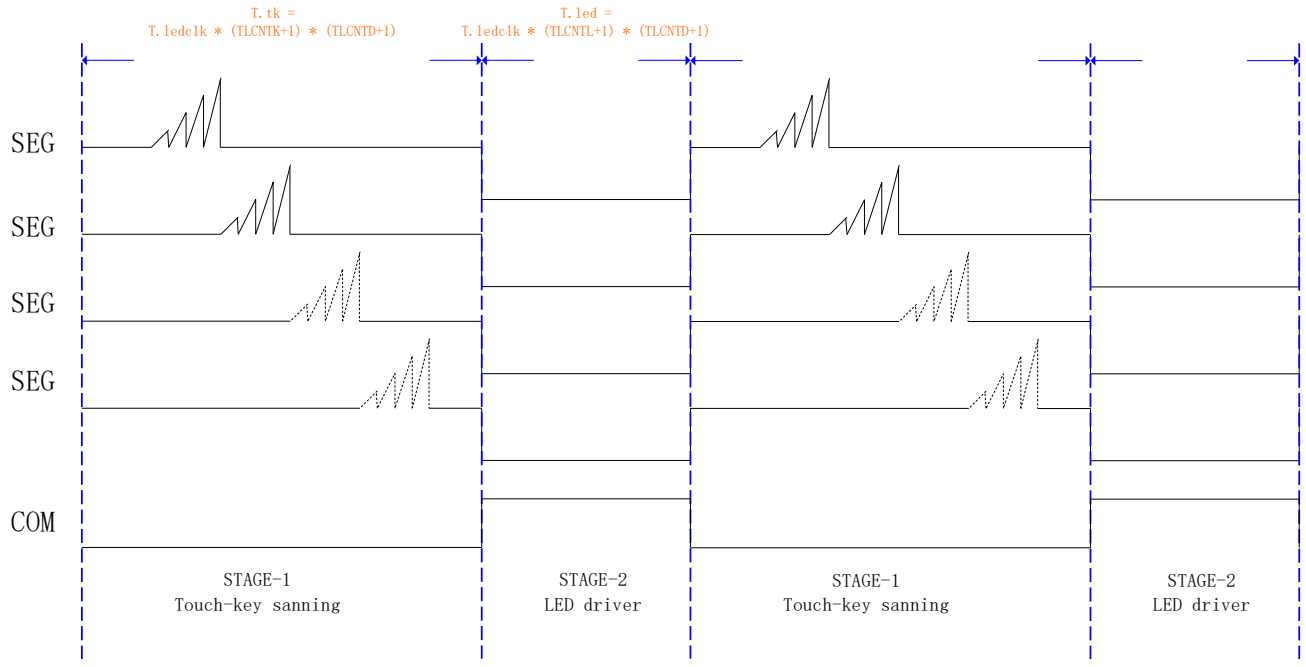


图 **21-5-1** 触摸共用 **LED** 驱动示意图

21.4.5 触摸内部基准和内部运放

触摸模块在内部集成了运放，可通过 TKPWS(TKPWC[1])选择内部运放作为触摸键的充电电源，充电电压通过 VDS(TKPWC[5:4])来选择。另外，触摸也可通过 TKCVS(TKPWC[0])选择内部基准作为触摸内部比较器的阈值电压，内部基准电压通过 VIRS(TKPWC[3:2])来选择。

21.4.6 触摸防水补偿机制

触摸设计了防水补偿机制，可通过 TKPC(TKPWC[7:6])为 2 开启。开启此功能后，非选中的触摸引脚会同步输出与充电频率相同的补偿波形，可有效降低触摸按键之间寄生电容的影响，实现防水的效果。注意：在开启防水补偿时，触摸充电电源最好选择为外部电源。

21.5 寄存器描述

表 **21-5-1** 寄存器 **TKCON**

C1H	7	6	5	4	3	2	1	0
TKCON	TKST	TKIE	TMEN	TWKE	-	VRS[2:0]		
R/W	R/W	R/W	R/W	R/W	-	R/W		
初始值	0	0	0	0	-	0	0	0

位编号	位符号	说明
7	TKST	数据采集启动使能位，1 有效，采集完后自动清 0
6	TKIE	TK 中断使能控制位，1 有效
5	TMEN	启动方式选择位 0:通过 TKST 位控制启动 1:定时器控制启动
4	TWKE	中断触发方式选择位 0:采样完成触发中断 1:超出阈值范围触发中断
3	-	-
2~0	VRS	比较器阈值电压基准选择位（阈值电压与 VDD 电压成正比例） 0：阈值电压最高 … 7：阈值电压最低

表 **21-5-2** 寄存器 **TKCFG**

C2H	7	6	5	4	3	2	1	0
TKCFG	TKDIV			TKTMS				
R/W	R/W			R/W				
初始值	0	0	0	1	1	1	1	1

位编号	位符号	说明
7~5	TKDIV	触摸时钟分频选择 000: 不分频 001：2 分频 010：3 分频 … 111：8 分频
4~0	TKTMS	外挂调制电容放电时间设置 放电时间 = TKTMS x 128 x 时钟周期 在 TKDIV=0 的条件下，放电时间范围是：32us~992us 备注：*TKTMS* 不能设置为 *0*。

表 **21-5-3** 寄存器 **TKPWC**

8103H	7	6	5	4	3	2	1	0
TKPWC	TKPC		VDS		VIRS		TKPWS	TKCVS
R/W	R/W		R/W		R/W		R/W	R/W
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~6	TKPC	触摸按键未采样通道输出控制 00：悬浮 01：输出低 10：输出补偿 备注： *1.* 这一功能仅对被配置为触摸按键功能的引脚有效。 *2.* 如果共用 *LED* 驱动功能使能，那么被选中作为 *LED* 驱动的引脚，这一功能将失效。
5~4	VDS	内部运放输出电压选择 00：2V 01：2.5V 10：3V 11：4V
3~2	VIRS	内部电压基准选择 00：1.0V 01：1.5V 10：2.0V

		11：2.5V
1	TKPWS	充电电源选择 0：选择外部电源 1：选择内部运放输出
0	TKCVS	充电基准电压选择 0：选择外部电压基准 1：选择内部电压基准

表 **21-5-4** 寄存器 **TKMTS**

C3H	7	6	5	4	3	2	1	0
TKMTS	TKMTS[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
7~0	TKMTS	定时模式的启动时间选择寄存器 启动时间=（TKMTS+1）x 32 x 低速时钟周期，如果低速时钟频率为 32.768K，时间范围是 0.977ms - 250ms。

表 **21-5-5** 寄存器 **TKCHS**

C4H	7	6	5	4	3	2	1	0
TKCHS	POL	NPOL	-	TKPS				
R/W	R/W	R/W	-	R/W				
初始值	0	0	-	0	0	0	0	0

备注：*TKCHS* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *TKCHS0~TKCH5*

位编号	位符号	说明
7	POL	ATKnC 阈值比较方向设置位 0：触摸数据小于阈值时触发阈值比较中断 1：触摸数据大于或等于阈值时触发阈值比较中断
6	NPOL	ATKnN 阈值比较方向设置位 0：触摸数据小于阈值时触发阈值比较中断 1：触摸数据大于或等于阈值时触发阈值比较中断
4~0	TKPS	通道选择位域 00000：所有 TK 通道关闭 00001：选择 TK0 00010：选择 TK1 00011：选择 TK2 ..... 10100：选择 TK19 10101：选择内部参考电容

表 **21-5-6** 寄存器 **ATKC**

C5H	7	6	5	4	3	2	1	0
ATKCL	ATKC[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
C6H	7	6	5	4	3	2	1	0
ATKCH	ATKC[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
备注：*ATKC* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *ATKC0~ATKC5*								
位编号	位符号	说明						
15~0	ATKC	比较阈值设置寄存器，当 TWKE=1 时，ATKC0~ATKC5 自动与 TKMS0~TKMS5 比较						

表 **25-5-7** 寄存器 **ATKN**

E6H	7	6	5	4	3	2	1	0
ATKNL	ATKN[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
E7H	7	6	5	4	3	2	1	0
ATKNH	ATKN[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
备注：*ATKC* 是带索引的寄存器，设置 *INDEX=0~5* 分别对应 *ATKC0~ATKC5*								
位编号	位符号	说明						
15~0	ATKN	比较阈值设置寄存器，当 TWKE=1 时，ATK0N~ATK5N 自动与 TK0MS~TK5MS 比较						

表 **21-5-8** 寄存器 **TKMS**

CEH	7	6	5	4	3	2	1	0
TKMSL	TKMS[7:0]							
R/W	R							
初始值	0	0	0	0	0	0	0	0
CFH	7	6	5	4	3	2	1	0
TKMSH	TKMS[15:8]							
R/W	R							
初始值	0	0	0	0	0	0	0	0

备注：*TKMS* 是带索引的寄存器，设置 *TKMS=0~5* 分别对应 *TKMS0~TKMS5*		
位编号	位符号	说明
15~0	TKMS	触摸采样数据寄存器

表 **21-5-9** 寄存器 **TKIF**

C7H	7	6	5	4	3	2	1	0
TKIF	-		TKIF5	TKIF4	TKIF3	TKIF2	TKIF1	TKIF0
R/W	-		R	R	R	R	R	R
初始值	-	-	0	0	0	0	0	0

位编号	位符号	说明
7~6	-	-
5~0	TKIFx(x=5~0)	触摸采集中断标志位，按顺序分别对应 6 个选定通道，当 TWKE=1 时，TKIFx 表示触摸数据超出阈值范围

表 **25-5-10** 寄存器 **TKMAXF**

D5H	7	6	5	4	3	2	1	0
TKMAXF	-	-	TKMXF5	TKMXF4	TKMXF3	TKMXF2	TKMXF1	TKMXF0
R/W	-	-	R	R	R	R	R	R
初始值	-	-	0	0	0	0	0	0

位编号	位符号	说明
7~6	-	-
5~0	TKMXFx(x=5~0)	1 表示 TKxMS 超出 ATKxC 的阈值范围，0 表示 TKxMS 不超出 ATKxC 的阈值，极性由 POLx 决定；如果 TWKE=1，那么置位 TKMXFx 的同时会置位 TKIFx；软件无法对其操作

表 **25-5-11** 寄存器 **TKMINF**

D6H	7	6	5	4	3	2	1	0
TKMINF	-	-	TKMNF5	TKMNF4	TKMNF3	TKMNF2	TKMNF1	TKMNF0
R/W	-	-	R	R	R	R	R	R
初始值	-	-	0	0	0	0	0	0

位编号	位符号	说明
7~6	-	-
5~0	TKMNFx(x=5~0)	1 表示 TKxMS 超出 ATKxN 的阈值范围，0 表示 TKxMS 不超出 ATKxN 的阈值，极性由 NPOLx 决定；如果 TWKE=1，那么置位 TKMNFx 的同时会置位 TKIFx；软件无法对其操作

表 **25-5-12** 寄存器 **TLEN**

8106H	7	6	5	4	3	2	1	0
TLEN (INDEX=0)	TLEN7	TLEN6	TLEN5	TLEN4	TLEN3	TLEN2	TLEN1	TLEN0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
TLEN (INDEX=1)	TLEN15	TLEN14	TLEN13	TLEN12	TLEN11	TLEN10	TLEN9	TLEN8
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
TLEN (INDEX=2)	-	-	-	-	TLEN19	TLEN18	TLEN17	TLEN16
R/W	-	-	-	-	R/W	R/W	R/W	R/W
初始值	-	-	-	-	0	0	0	0

备注：

TLENx=1 (x=0,1,2,…,19)，*LEDx* 要使能，必须对应的 *TKx* 引脚选择触摸按键功能。

位编号	位符号	说明
3~0 (INDEX=2)	TLEN19~TLEN16	LED19~LED16 使能，1 有效
7~0 (INDEX=1)	TLEN15~TLEN8	LED15~LED8 使能，1 有效
7~0 (INDEX=0)	TLEN7~TLEN0	LED7~LED0 使能，1 有效

表 **25-5-13** 寄存器 **TLDAT**

8107H	7	6	5	4	3	2	1	0
TLDAT (INDEX=0)	TLDAT7	TLDAT6	TLDAT5	TLDAT4	TLDAT3	TLDAT2	TLDAT1	TLDAT0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
TLDAT (INDEX=1)	TLDAT15	TLDAT14	TLDAT13	TLDAT12	TLDAT11	TLDAT10	TLDAT9	TLDAT8
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
初始值	0	0	0	0	0	0	0	0
TLDAT (INDEX=2)	-	-	-	-	TLDAT19	TLDAT18	TLDAT17	TLDAT16
R/W	-	-	-	-	R/W	R/W	R/W	R/W
初始值	-	-	-	-	0	0	0	0

位编号	位符号	说明
3~0 (INDEX=2)	TLDAT19~TLDAT16	LED19~LED16 数据，1 表示驱动有效电平
7~0 (INDEX=1)	TLDAT15~TLDAT8	LED15~LED8 数据，1 表示驱动有效电平
7~0 (INDEX=0)	TLDAT7~TLDAT0	LED7~LED0 数据，1 表示驱动有效电平

表 **25-5-14** 寄存器 **TLCON**

8108H	7	6	5	4	3	2	1	0
TLCON	TLEIE	TLKIE	TLLIE	-	-	TLLVS		TLPOL
R/W	R	R/W	R/W	-	-	R/W		R/W

初始值	0	0	0	-	-	0	0	0

位编号	位符号	说明
7	TLEIE	TLERR 中断使能，1 有效
6	TLKIE	TLKOV 中断使能，1 有效
5	TLLIE	TLLOV 中断使能，1 有效
4~3	-	-
2~1	TLLVS	触摸按键扫描阶段，COM 引脚电平选择 00：输出高 01：输出低 10：上拉高 其他：保留
0	TLPOL	LED 驱动阶段，有效驱动电平选择 0：驱动电平高有效 1：驱动电平低有效

表 **25-5-15** 寄存器 **TLFLG**

8109H	7	6	5	4	3	2	1	0
TLFLG	TLERR	TLKOV	TLLOV	-	-	-	-	-
R/W	R/W	R/W	R/W	-	-	-	-	-
初始值	0	0	0	-	-	-	-	-

位编号	位符号	说明
7	TLERR	触摸按键扫描阶段时间设置过短标志 0：表示用户设置的时间足够触摸按键扫描的时间 1：表示用户设置的时间不足，定时器计数完还有通道没扫描完 备注： 此位为硬件自动置位和自动清零位，考虑到用户使用的便捷性，此位也 可以用软件对其写 *1* 清 *0*。
6	TLKOV	触摸按键扫描阶段定时器计满标志，1 表示计满，软件写 1 清 0
5	TLLOV	LED 驱动阶段定时器计满标志，1 表示计满，软件写 1 清 0
4~0	-	-

表 **25-5-16** 寄存器 **TLCKS**

810AH	7	6	5	4	3	2	1	0
TLCKS	-	-	-	-	-	TLCKS[2:0]		
R/W	-	-	-	-	-	R/W		
初始值	-	-	-	-	-	0	0	0

位编号	位符号	说明

7~3	-	-
2~0	TLCKS	LED 驱动工作时钟选择 001：IRCL 010：IRCH 011：XOSCL 其他：关闭

表 **25-5-17** 寄存器 **TLCNTK**

810BH	7	6	5	4	3	2	1	0
TLCNTKL	TLCNTK[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
810CH	7	6	5	4	3	2	1	0
TLCNTKH	TLCNTK[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
15~0	TLCNTK	触摸按键扫描阶段时间配置寄存器 $T_{.tk} = T_{.ledclk}$ x (TLCNTK+1) x (TLDIV+1) 备注： *T.tk*，表示触摸按键扫描阶段的时间；*T.ledclk*，表示 *LED* 驱动电路的工作时钟的周期。

表 **25-5-18** 寄存器 **TLCNTL**

810DH	7	6	5	4	3	2	1	0
TLCNTLL	TLCNTL[7:0]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0
810EH	7	6	5	4	3	2	1	0
TLCNTLH	TLCNTL[15:8]							
R/W	R/W							
初始值	0	0	0	0	0	0	0	0

位编号	位符号	说明
15~0	TLCNTL	LED 驱动阶段时间配置寄存器 $T_{.led} = T_{.ledclk}$x (TLCNTL+1) x (TLDIV+1)

		备注： *T.led*，表示 *LED* 驱动阶段的时间

表 **25-5-19** 寄存器 **TLDIV**

810FH	7	6	5	4	3	2	1	0
TLDIV	-	-	-	-	TLDIV[3:0]			
R/W	-	-	-	-	R/W			
初始值	-	-	-	-	0	0	0	0
位编号	位符号	说明						
7~4	-	-						
3~0	TLDIV	$T_{.ledclk}$ 时钟分频寄存器 分频倍数为（TLDIV+1）						

表 **25-5-20** 寄存器 **TLCOMS**

8110H	7	6	5	4	3	2	1	0
TLCOMS	-	-	-	-	TLCMS[3:0]			
R/W	-	-	-	-	R/W			
初始值	-	-	-	-	0	0	0	0
位编号	位符号	说明						
7~4	-	-						
3~0	TLCMS	COM 引脚选择控制 0001：选择 P0.0 0010：选择 P0.1 0011：选择 P0.2 0100：选择 P0.3 0101：选择 P0.4 其他：关闭						

21.6 触摸控制例程

备注：触摸应用实例请参考本公司标准触摸库软件及相关文档。

22 低电压检测（LVD）

22.1 功能简介

低电压检测（LVD）用于监控芯片自身的供电 VDD，可设置检测电压范围为 1.8V~4.8V。当 VDD 小于所设定的电压值时，可触发中断或复位。

LVD 结构图如图 22-1-1 所示。

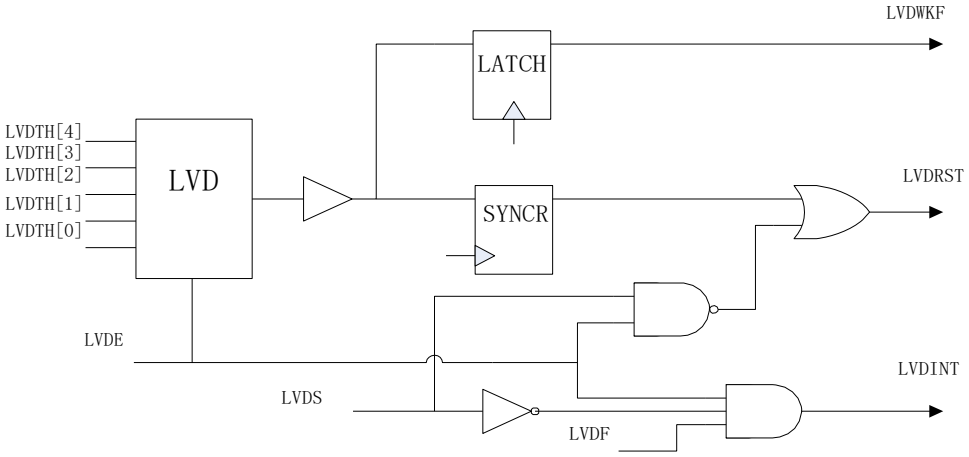


图 **22-1-1 LVD** 模块示意图

22.2 功能描述

LVD 功能通过 LVDE 位使能，而检测的电压则通过 LVDTH 设置。当芯片 VDD 小于所设置的电压时，LVD 功能产生的标志 LVDF 位将置 1，如果 LVDS=0，会产生 LVD 中断，如果 LVDS=1，会产生复位。要注意的是，LVD 复位产生之后，使能位 LVDE 会清零，但是 LVD 产生标志位 LVDF 会保留。如果设置为中断模式，当 VDD 持续低于所设置的电压，只会产生一次中断，直到 VDD 高于所设置的电压后，再次低于门限电压才会再次产生 中断。

备注：受工艺的影响，设置的触发电压与实际值有一定偏差，偏差值小于±*50mv*。

22.3 寄存器描述

表 **22-3-1** 寄存器 **LVDCON**

EFH	7	6	5	4	3	2	1	0
LVDCON	LVDE	LVDS	LVDF	-	LVDTH[3:0]			
R/W	R/W	R/W	R/W	-	R/W			
初始值	0	0	0	-	0	0	0	0

位编号	位符号	说明
7	LVDE	LVD 使能位，1 有效
6	LVDS	LVD 功能选择位 0：中断 1：复位
5	LVDF	LVD 产生标志位，写 1 清 0
4	-	-
3~0	LVDTH	LVD 触发电平选择位域 0000：1.8V 0001：2.0V 0010：2.2V 0011：2.4V 0100：2.6V 0101：2.8V 0110：3.0V 0111：3.2V 1000：3.4V 1001：3.6V 1010：3.8V 1011：4.0V 1100：4.2V 1101：4.4V 1110：4.6V 1111：4.8V

22.4 LVD 控制例程

◆ **LVD** 中断例程

例如，设置 LVD 为中断模式，检测电压为 3V，程序如下：

```
---------------------------------------------------------------------------------------
#define LVDE(N)          (N<<7)    //N=0~1
#define LVDS_reset       (1<<6)
#define LVDS_int         (0<<6)
#define LVDF             (1<<5)
#define LVDTH_3V         6
void LVD_init(void)
{
    LVDCON = LVDE(1) | LVDS_int | LVDTH_3V;  //设置 LVD 使能，设置 LVD 为中断模式，检测电压为 3V
    INT4EN = 1;                              //INT4 中断使能
}

void INT4_ISR (void) interrupt 9             //LVD 中断服务程序
```

```
{
    if(LVDCON & LVDF)
    {
        LVDCON |= LVDF;          //清除 LVD 中断标志
        ...

    }
    ...
}
```

◆ **LVD** 复位例程

例如，设置 LVD 为复位模式，检测电压为 3V，程序如下：

```
#define LVDE(N)          (N<<7)    //N=0~1
#define LVDS_reset       (1<<6)
#define LVDS_int         (0<<6)
#define LVDF             (1<<5)
#define LVDTH_3V         6
void LVD_init(void)
{
    LVDCON = LVDE(1) | LVDS_reset | LVDTH_3V;//设置 LVD 使能，设置 LVD 为复位模式，检测电压为 3V
}
```

---

23 程序下载和仿真

23.1 程序下载

MJ603 系列芯片主要采用 ISP 方式下载程序，芯片通过 UART 接口与下载工具相连接，任意一组 UART 口都 可以用于 ISP。

更多关于程序下载步骤的细节请参考“CACHIP 开发下载工具使用说明”。

23.2 在线仿真

MJ603 系列芯片支持在线仿真，芯片与仿真器之间通过 IIC 接口进行通信。CA51F351S1 默认的 IIC 端口是 P11(IIC SDA)和 P12(IIC SCL), 其他型号芯片默认的 IIC 端口是 P30(IIC SDA)和 P31(IIC SCL)。要注意的是，由于芯片与仿真器间通过 IIC 通信，所以与仿真器连接的 IIC 接口引脚不能设置为其他功能，并且应用程序里不能使用 IIC 功能，否则将无法进入仿真模式。另外，由于 IIC 的通信速度是由主时钟决定，所以应用程序里不能将主时钟设置为低速时钟，也不能进入省电模式，否则都会影响芯片与仿真器间的通信。

当 TSME=0（PCON[3]）时，芯片禁止进入仿真模式。当芯片进入仿真模式后，TSMODE 位（PCON[2]）置 1，应用程序可通过判断此位状态来决定是否切换至低速时钟或进入省电模式。

更多关于仿真功能的细节可参考仿真器的相关文档介绍。

24 电气特性

24.1 极限参数

参数	最小值	最大值	单位
直流供电电压	-0.3	6	V
I/O 引脚输入电压	-0.3	VDD+0.3	V
工作环境温度	-40	85	℃
储存温度	-55	125	℃
CPU 工作频率	-	27	MHz

备注：超过“极限参数”范围有可能对芯片造成损坏，无法预期芯片在上述范围外的工作状态，若长期在标示范围外工作，可能会影响芯片的可靠性。

24.2 直流电气特性

芯片参数	符号	工作电压	最小值	典型值	最大值	单位	测试条件
工作电流	Iop1	VDD=1.8V		0.496		mA	系统时钟为 IRCH(3.6864MHz)，其他时钟关闭，LDO 设置为默认值（高功率模式，输出电压为 1.61V），所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，CPU 执行 NOP 指令
		VDD=3.3V		0.546			
		VDD=5V		0.550			
	Iop2	VDD=1.8V		2.18		mA	系统时钟为 PLL 输出，PLL 设置为 6 倍频，参考时钟 IRCH 频率为 3.6864MHz，其他时钟关闭，LDO 设置为默认值（高功率模式，输出电压为 1.61V），所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，CPU 执行 NOP 指令
		VDD=3.3V		2.47			
		VDD=5V		2.49			
	Iop3	VDD=1.8V		22.6		uA	系统时钟为 IRCL(131kHZ)，其他时钟关闭，LDO 设置为低功率模式，输出电压为 1.61V，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，CPU 执行 NOP 指令
		VDD=3.3V		23.6			
		VDD=5V		24			
	Iop4	VDD=1.8V		12.8		uA	系统时钟为 XOSCL(32.768kHZ)，其他时钟关闭，LDO 设置为低功率模式，输出电压为 1.61V，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，CPU 执行 NOP 指令
		VDD=3.3V		13.3			
		VDD=5V		13.6			
	Iop5	VDD=1.8V		15.1		uA	系统时钟为 XOSCL(32.768kHZ)，其他时钟关闭，LDO 设置为低功率模式，输出电压为 1.61V，打开 LCD 驱动（不外接 LCD 面板），LCD 设置为最小电流驱动、1/3bias、1/4duty、LCD 时钟为 XOSCL，LCD_CAD 关闭（CAD_MOD=0），所有 LCD 引脚打开，其他所有
		VDD=3.3V		17.6			
		VDD=5V		20.3			

							输出引脚无负载，所有数字输入引脚不浮动，其他外设关闭
STOP 模式电流	I_{stp}	VDD=1.8V		5.3		uA	所有时钟关闭，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，LDO 设置为低功率模式，Flash 进入睡眠模式，CPU 进入 STOP 模式。
		VDD=3.3V		5.5			
		VDD=5V		5.7			
IDLE 模式电流	I_{idl1}	VDD=1.8V		0.269		mA	系统时钟设为 IRCH（3.6864MHz），其他时钟关闭，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，LDO 设置为低功率模式，Flash 进入睡眠模式，CPU 进入 IDLE 模式。
		VDD=3.3V		0.284			
		VDD=5V		0.286			
	I_{idl2}	VDD=1.8V		0.469		mA	系统时钟为 PLL 输出，PLL 设置为 6 倍频，参考时钟 IRCH 频率为 3.6864MHz，其他时钟关闭，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，CPU 进入 IDLE 模式。
		VDD=3.3V		0.500			
		VDD=5V		0.504			
	I_{idl3}	VDD=1.8V		13.6		uA	系统时钟设为 IRCL（131KHz），其他时钟关闭，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，LDO 设置为低功率模式，CPU 进入 IDLE 模式。
		VDD=3.3V		13.9			
		VDD=5V		14.2			
	I_{idl4}	VDD=1.8V		10.3		uA	系统时钟设为 XOSCL（32.768KHz），其他时钟关闭，所有输出引脚无负载，所有数字输入引脚不浮动，所有外设关闭，LDO 设置为低功率模式，Flash 进入睡眠模式，CPU 进入 IDLE 模式。
		VDD=3.3V		10.5			
		VDD=5V		10.8			
	I_{idl5}	VDD=1.8V		11.6		uA	系统时钟为 XOSCL(32.768kHZ)，其他时钟关闭，打开 LCD 驱动，LCD 设置为最小电流驱动、1/3bias、1/4duty、LCD_CAD 关闭（CAD_MOD=0），LCD 时钟为 XOSCL，所有 LCD 引脚打开，其他所有输出引脚无负载，所有数字输入引脚不浮动，CPU 进入 IDLE 模式。
		VDD=3.3V		13.7			
		VDD=5V		16.3			
IO 端口输入高电压（斯密特模式开启）	V_{hi1}	VDD=1.8V	0.75	-	1.8	V	-
		VDD=3.3V	1.20		3.3		
		VDD=5V	1.50		5		
IO 端口输入高电压（斯密特模式关闭）	V_{hi2}	VDD=1.8V		0.5*VDD	VDD	V	-
		VDD=3.3V					
		VDD=5V					
IO 端口输入低电压（斯密特模式开启）	V_{lo1}	VDD=1.8V	0	-	0.62	V	-
		VDD=3.3V	0	-	0.85		
		VDD=5V	0	-	1.20		
IO 端口输入低电压（斯密特模式关闭）	V_{lo2}	VDD=1.8V	0	0.5*VDD		V	-
		VDD=3.3V					
		VDD=5V					
IO 端口推电流	I_{pu}	VDD=3.3V	-	6.05	-	mA	IO 设为推挽输出模式，驱动能力设为最大，Vol=VDD-0.3V
		VDD=5V	-	8.46	-		
IO 端口灌电流	I_{ol}	VDD=3.3V	-	13.34	-	mA	IO 设为推挽输出模式，驱动能力设为最大，Vol=GND+0.3V
		VDD=5V	-	19.05	-		
COM 端口灌电	Isi	VDD=3.3V		70		mA	IO 设为推挽输出或 LED COM 引脚功能，驱动能力设

流		VDD=5V		96			为最大，Sink 功能开启，Vol=GND+0.3V
IO 端口强下拉电阻	Rd1	VDD=1.8~5.5V		15		KΩ	-
IO 端口弱下拉电阻	Rd2	VDD=1.8~5.5V	-	45	-	KΩ	-
IO 端口强上拉电阻	Ru1	VDD=1.8~5.5V	-	10	-	KΩ	-
IO 端口弱上拉电阻	Ru2	VDD=1.8~5.5V		45		KΩ	

说明：以上参数是随机抽取的典型芯片测试结果，仅供参考。

24.3 交流电气特性

交流电气特性（VDD=1.8-5.5V, TA=25℃，除非其它说明）

芯片参数	符号	最小值	典型值	最大值	单位	条件
内部低速时钟（IRCL）起振时间	Trc1	-	50	-	us	IRCL 频率为 131K
内部高速时钟（IRCH）起振时间	Trc2	-	10	-	us	IRCH 频率为 3.6864MHz
外部低速时钟（XOSCL）起振时间	Tosc1	-	0.5	1	s	XOSCL 频率为 32.768KHz
PLL 稳定时间	Tpll	-	50	-	us	参考时钟 IRCH 频率为 3.6864MHz，PLL 为 6 倍频
复位脉冲时间	Trst	-	0.5	-	us	

备注：*VDD=3.3V,TA=25*℃,内部高速时钟出厂频率为 *3.6864MHz*，误差小于 *1%.*

24.4 ADC 电气特性

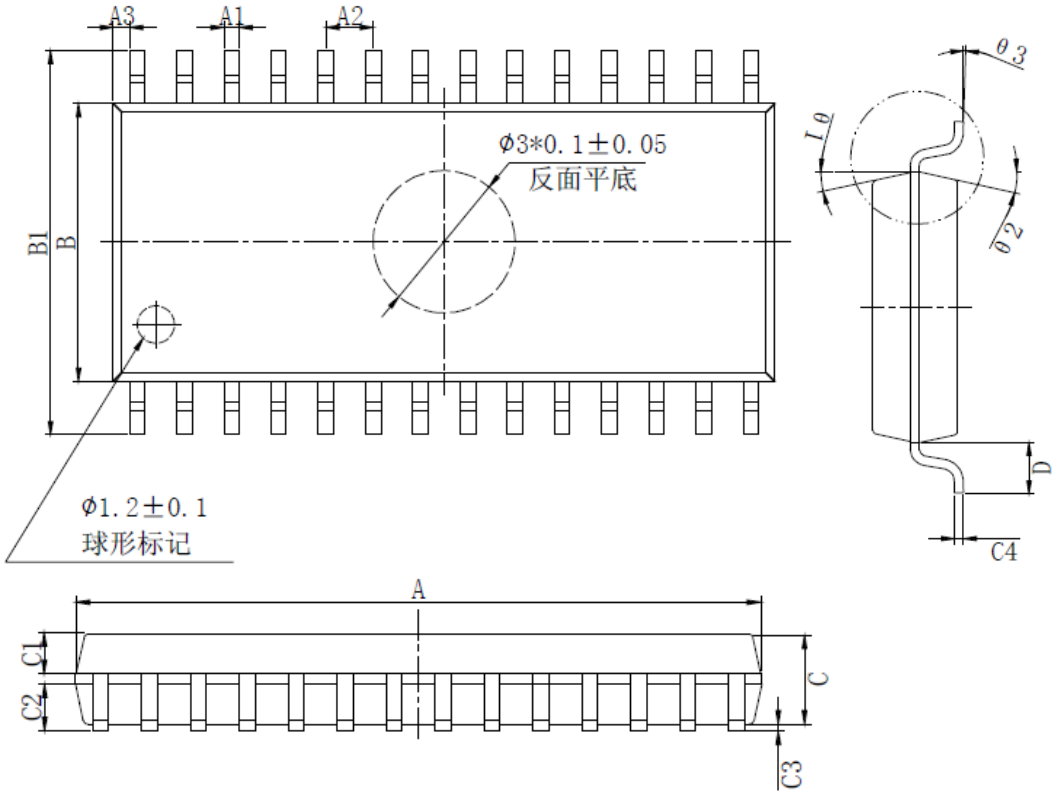
模/数转换器（ADC）电气特性(Ta=25℃,参考电压为 VDD)

芯片参数	符号	最小值	典型值	最大值	单位	条件
工作电压	V_{AD}	1.8		5.5	V	
ADC 精度	NR		11		Bit	GND<=Vin<=Vref，ADC 参考电压为 VDD 或外部基准
			10		Bit	GND<=Vin<=Vref，ADC 参考电压为内部基准
ADC 外部基准电压	Vex	1.1	-	VDD	V	
ADC 输入电压	Vin	0	-	VDD	V	
ADC 输入电阻	Rin	2	-	-	MΩ	VDD=5V
ADC 转换电流	I_{ADC}	-	180	-	uA	VDD=5V
微分非线性误差	DNL	-	-	±3	LSB	VDD=5V
积分非线性误差	INL	-	-	±3	LSB	VDD=5V
满刻度误差	EF	-	±3	±4	LSB	VDD=5V
偏移量误差	Ez	-	±0.5	±1	LSB	VDD=5V
转换时间	T_{CON}	-	16	-	时钟周期	

备注：（*1*）*ADC* 输入电阻是直流条件下 *ADC* 自身的输入电阻；
（*2*）测试 *ADC* 时连接通路的信号源内阻需要小于 *10K*Ω

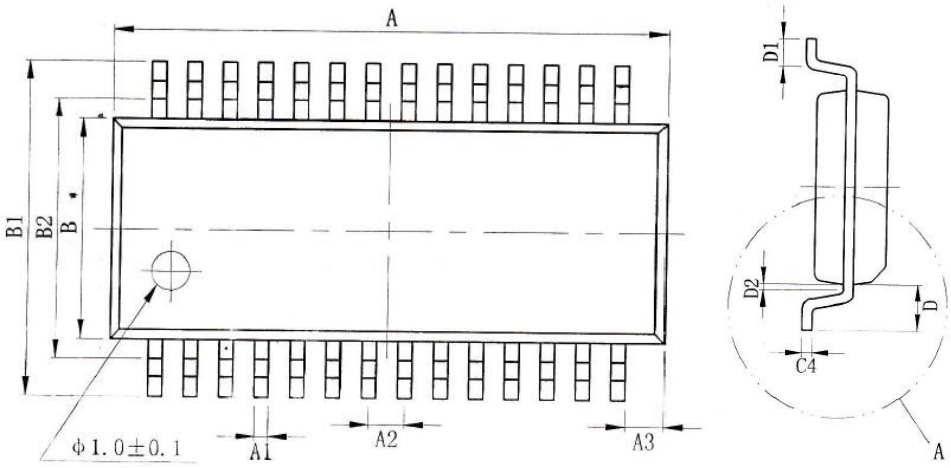
25 封装类型

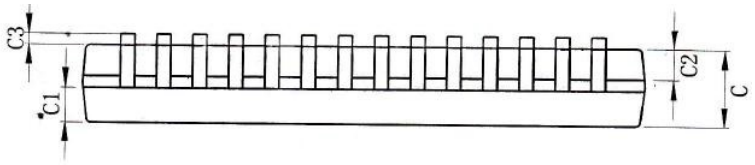
封装形式：**SOP28**



序号	最小值(mm)	标准值(mm)	最大值(mm)
A	17.90	18.00	18.10
A1	0.356	0.40	0.456
A2	1.24	1.27	1.30
A3	---	0.542 TYP	---
B	7.40	7.50	7.60
B1	10.206	10.30	10.406
C	2.18	2.23	2.28
C1	0.938	1.0	1.038
C2	0.938	1.0	1.038
C3	0.03	0.09	0.17
D	1.353	1.40	1.453
C4	0.244	0.25	0.264

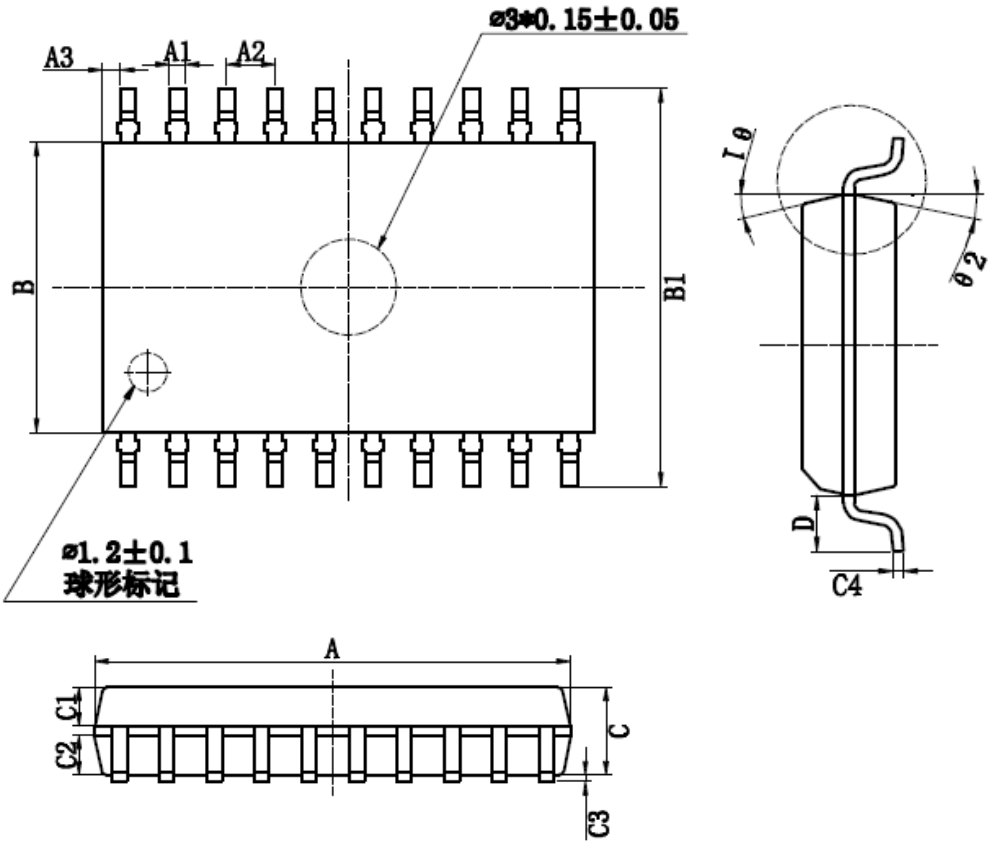
封装形式： **SSOP28**





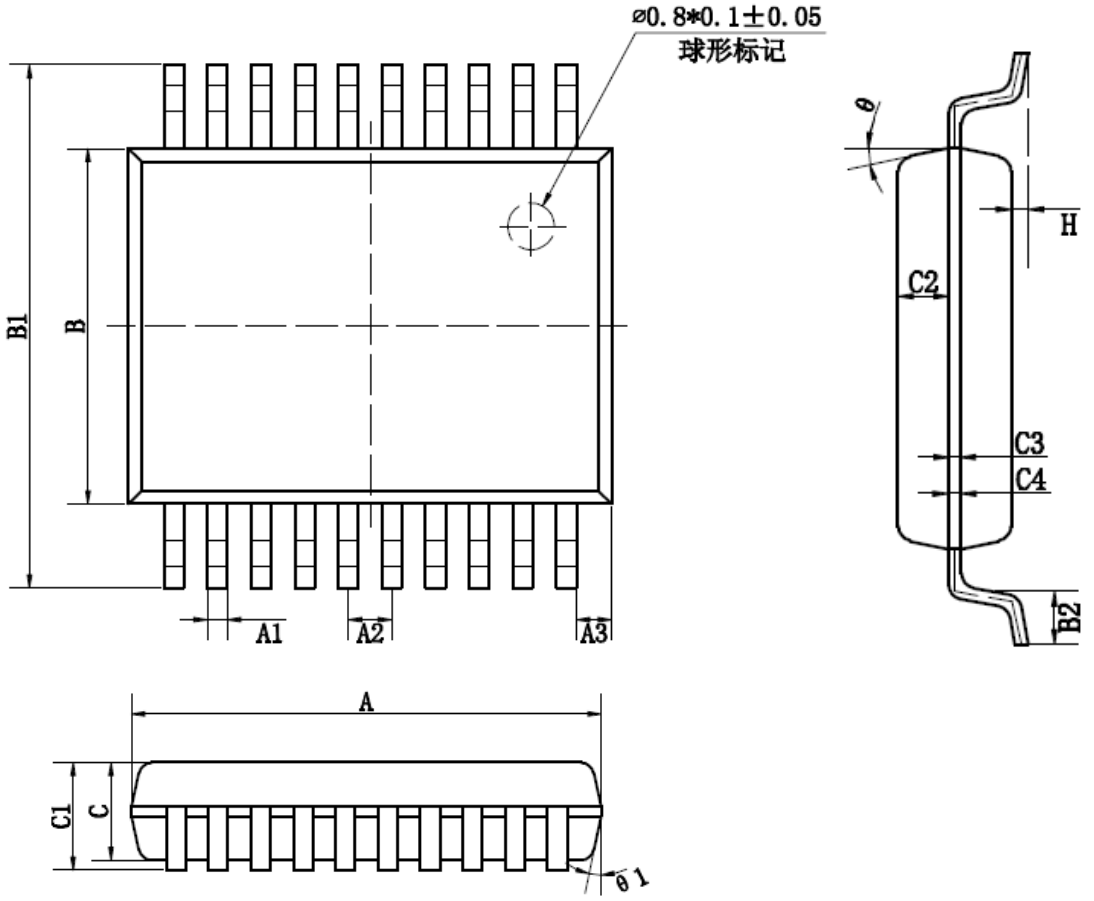
序号	最小值(mm)	标准值(mm)	最大值(mm)
A	9.80	9.90	10.00
A1	---	0.254TYP	---
A2	---	0.635TYP	---
A3	---	0.695TYP	---
B	3.85	3.90	3.95
B1	5.85	6.00	6.24
B2	---	5.00TYP	---
C	1.40	1.50	1.60
C1	0.61	0.66	0.71
C2	0.54	0.59	0.64
C3	0.05	0.15	0.25
C4	0.203	0.215	0.233
D	---	1.05TYP	---
D1	0.40	0.55	0.70
D2	0.15	0.20	0.25

封装形式： **SOP20**



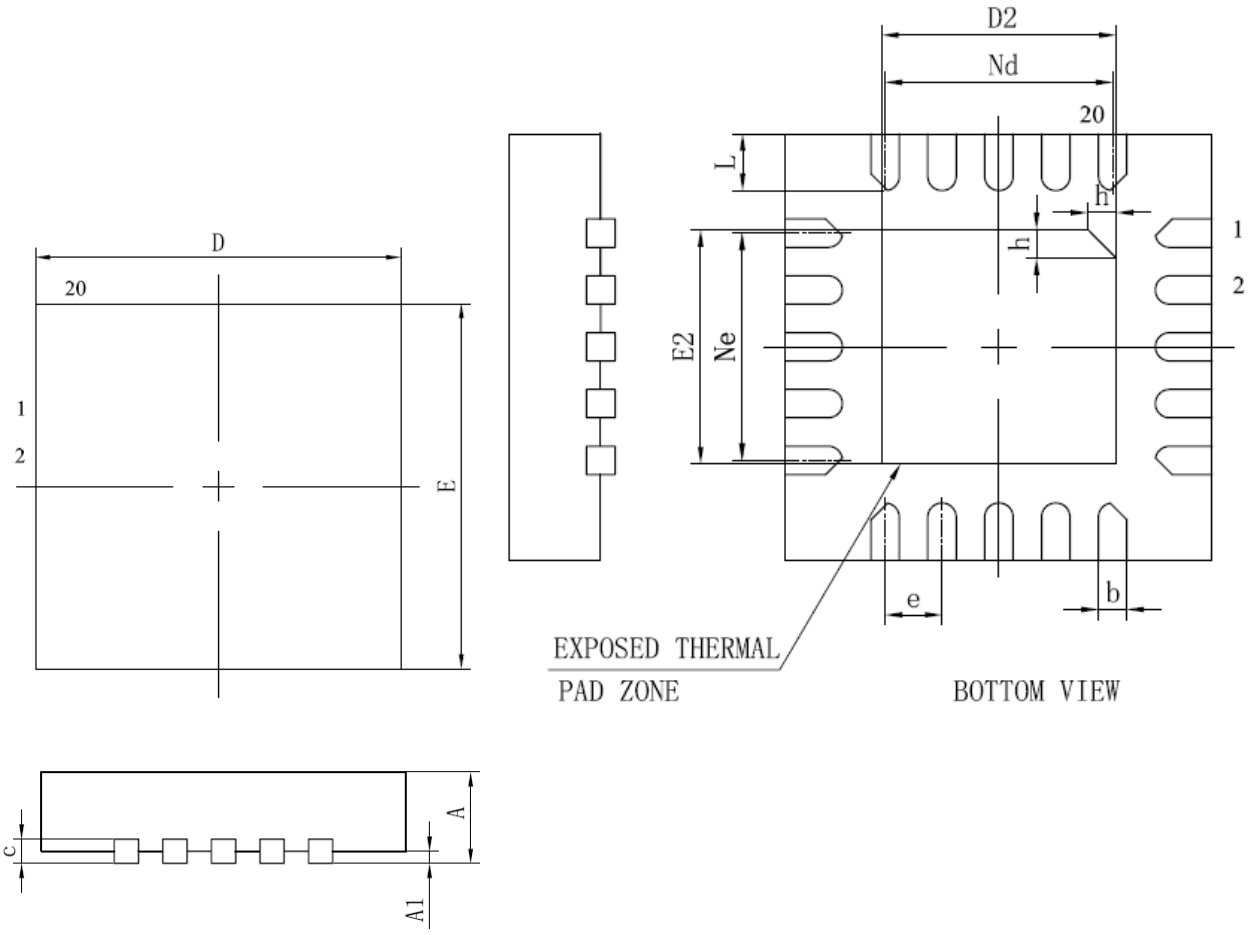
序号	最小值(mm)	标准值(mm)	最大值(mm)
A	12.65	12.70	12.80
A1	0.381	0.40	0.431
A2	1.24	1.27	1.30
A3	0.45	0.455	0.46
B	7.40	7.50	7.60
B1	10.206	10.30	10.406
C	2.18	2.23	2.28
C1	0.938	1.0	1.038
C2	0.938	1.0	1.038
C3	0.145	0.175	0.205
D	1.353	1.40	1.453
C4	0.246	0.25	0.262

封装形式：**SSOP20**



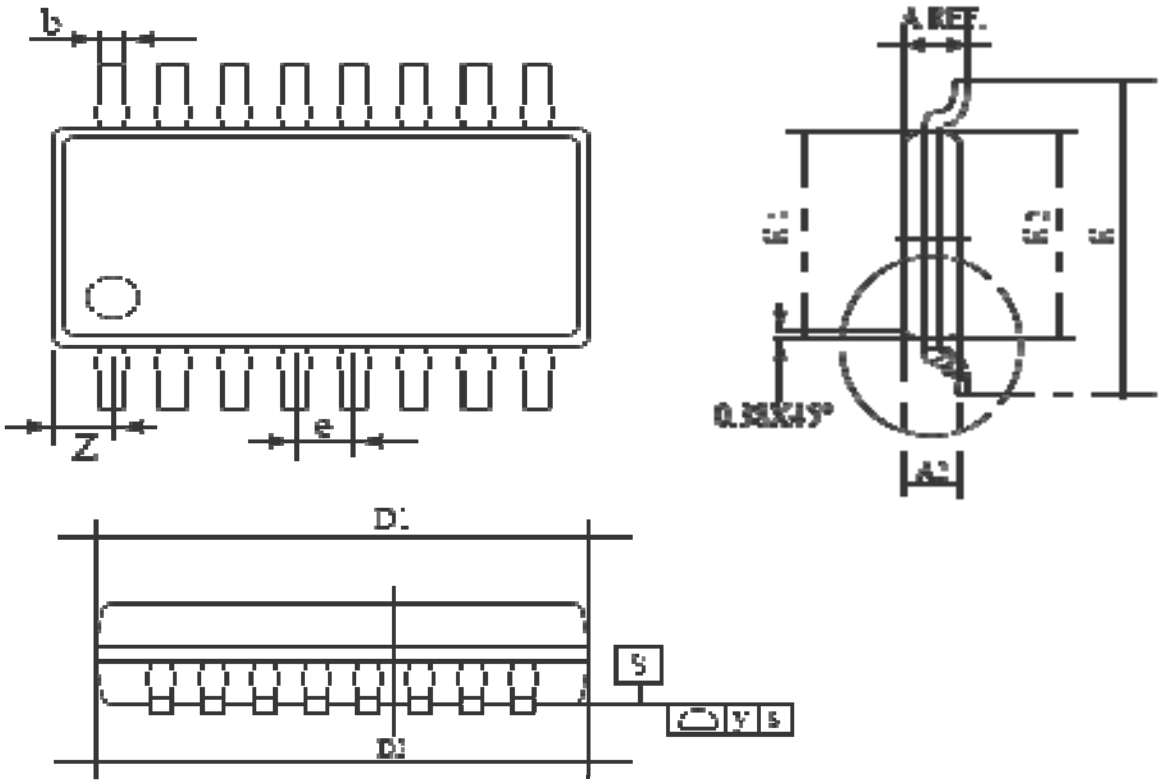
序号	最小值(mm)	标准值(mm)	最大值(mm)
A	7.15	7.20	7.25
A1	---	0.30 TYP	---
A2	---	0.65 TYP	----
A3	---	0.525 TYP	---
B	5.25	5.30	5.35
B1	7.65	7.80	7.95
C	1.45	1.50	1.55
C1	1.65	1.75	1.85
C2	----	0.674	----
C3	----	0.152	----
B2	0.60	0.70	0.80
C4	----	0.172	----

封装形式：**QFN20(3X3MM)**



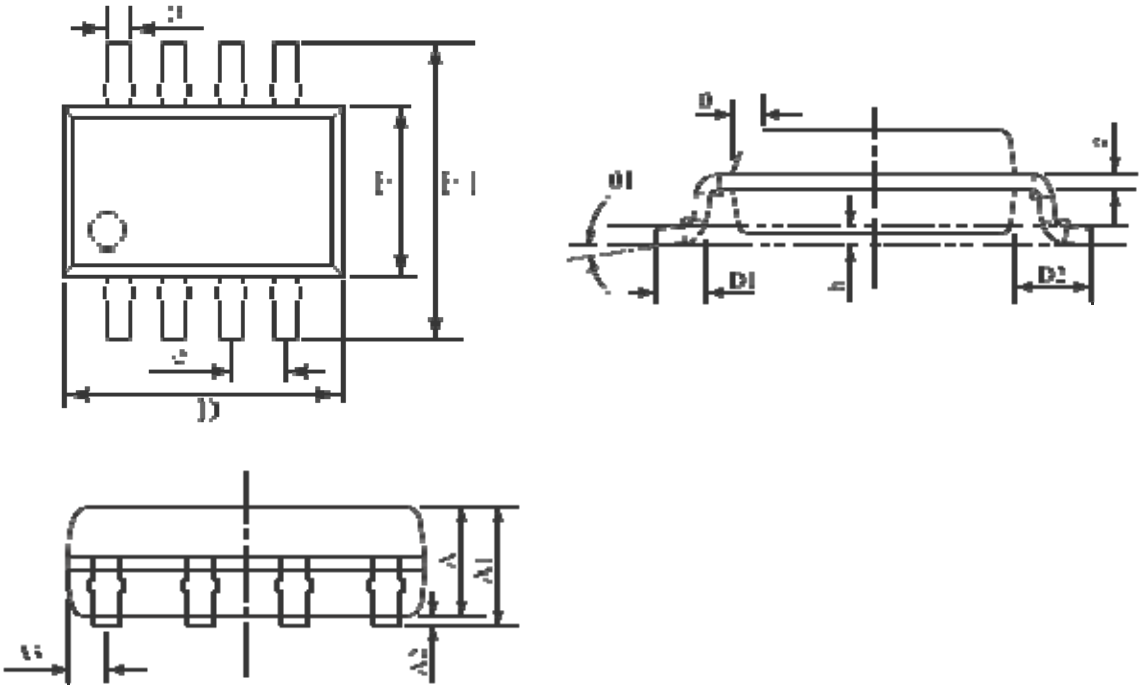
序号	最小值(mm)	标准值(mm)	最大值(mm)
A	0.70	0.75	0.80
A1	---	0.02	0.05
b	0.15	0.20	0.25
c	0.18	0.20	0.25
D	2.90	3.00	3.10
D2	1.55	1.65	1.75
e	0.40BSC		
Ne	1.60BSC		
Nd	1.60BSC		
E	2.90	3.00	3.10
E2	1.55	1.65	1.75
L	0.35	0.40	0.45
h	0.20	0.25	0.30

封装形式： **SOP16**



序号	最小值(mm)	标准值(mm)	最大值(mm)
A	1.500	1.600	1.700
A2	1.400	1.450	1.500
b	0.356	0.406	0.456
D1	9.70	9.90	10.10
D2	9.75	9.95	10.15
E	5.90	6.000	6.100
E1	3.800	3.900	4.000
E2	3.850	3.950	4.050
e	————	1.27	————
Z	————	0.505	————

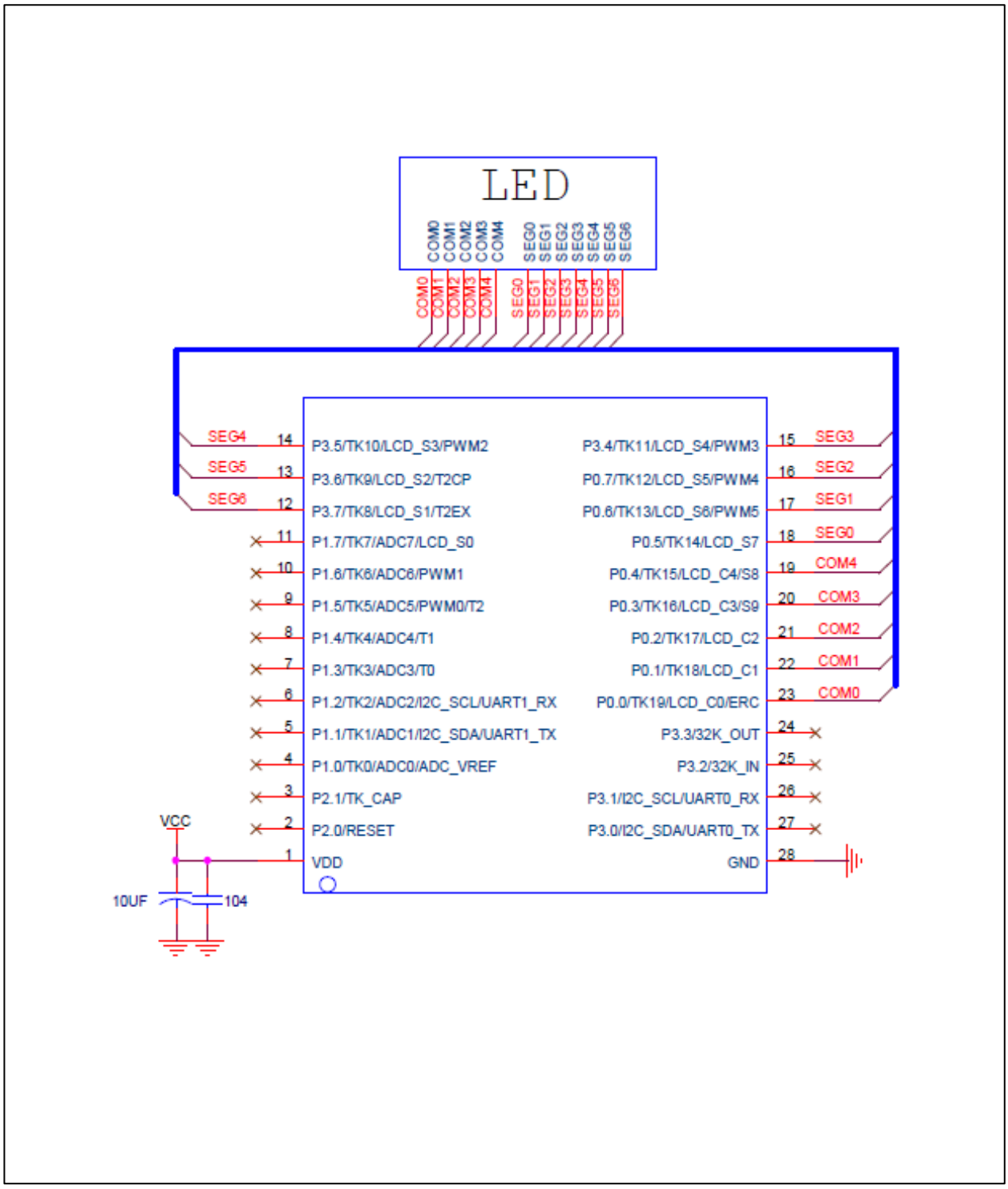
封装形式： **SOP8**



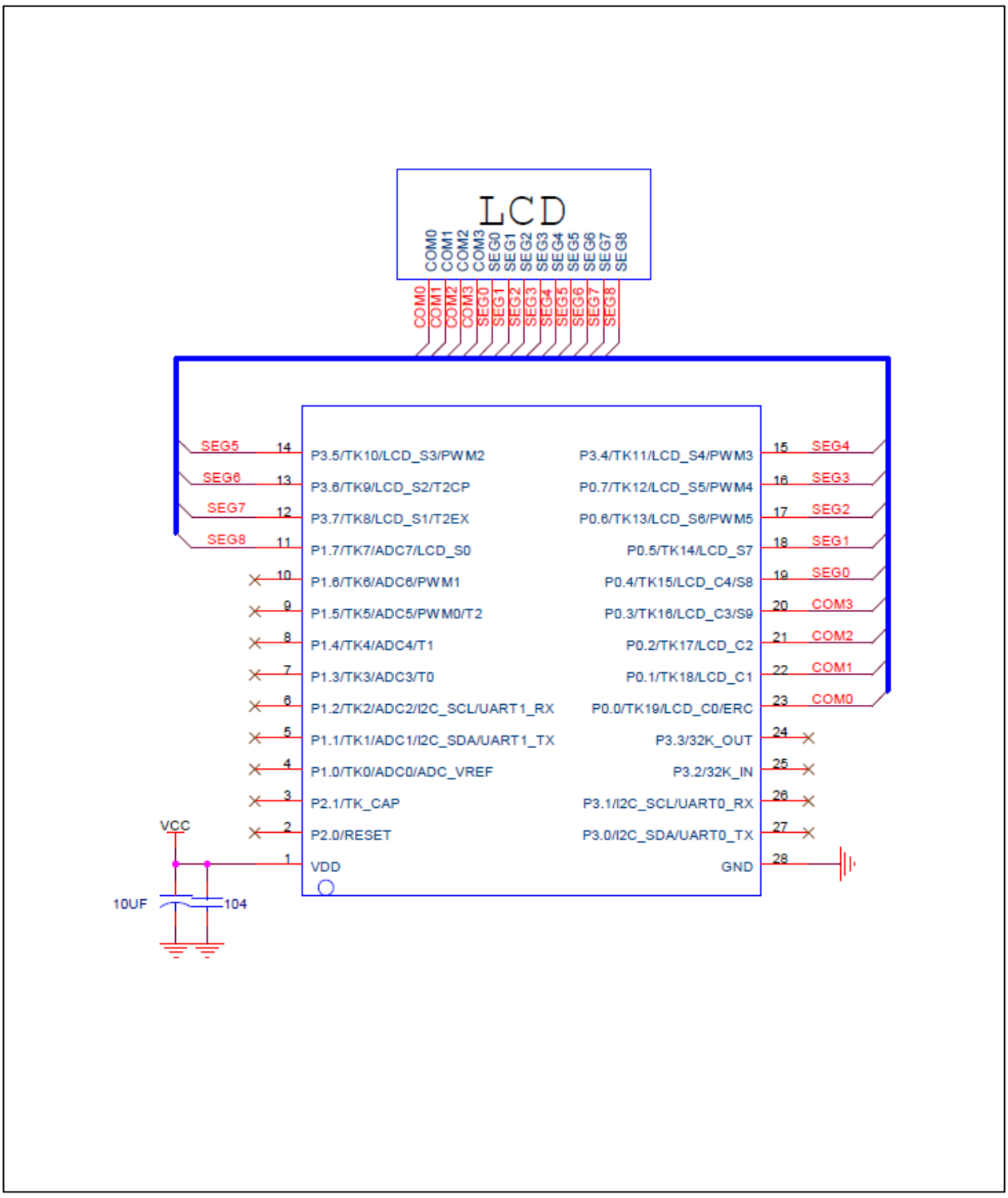
序号	最小值(mm)	标准值(mm)	最大值(mm)
A	1.40	1.45	1.50
A1	1.55	1.60	1.65
A2	0.10	0.15	0.20
A3	0.50	0.535	0.540
b	0.354	0.406	0.504
b1	0.155	0.150	0.175
c	0.20	0.203	0.210
D	4.830	4.880	4.910
D1	0.610	0.660	0.710
D2	1.045	1.050	1.0505
e	———	1.270	———
E	3.810	3.910	3.96
E1	5.900	6.000	6.10

26 典型应用参考电路

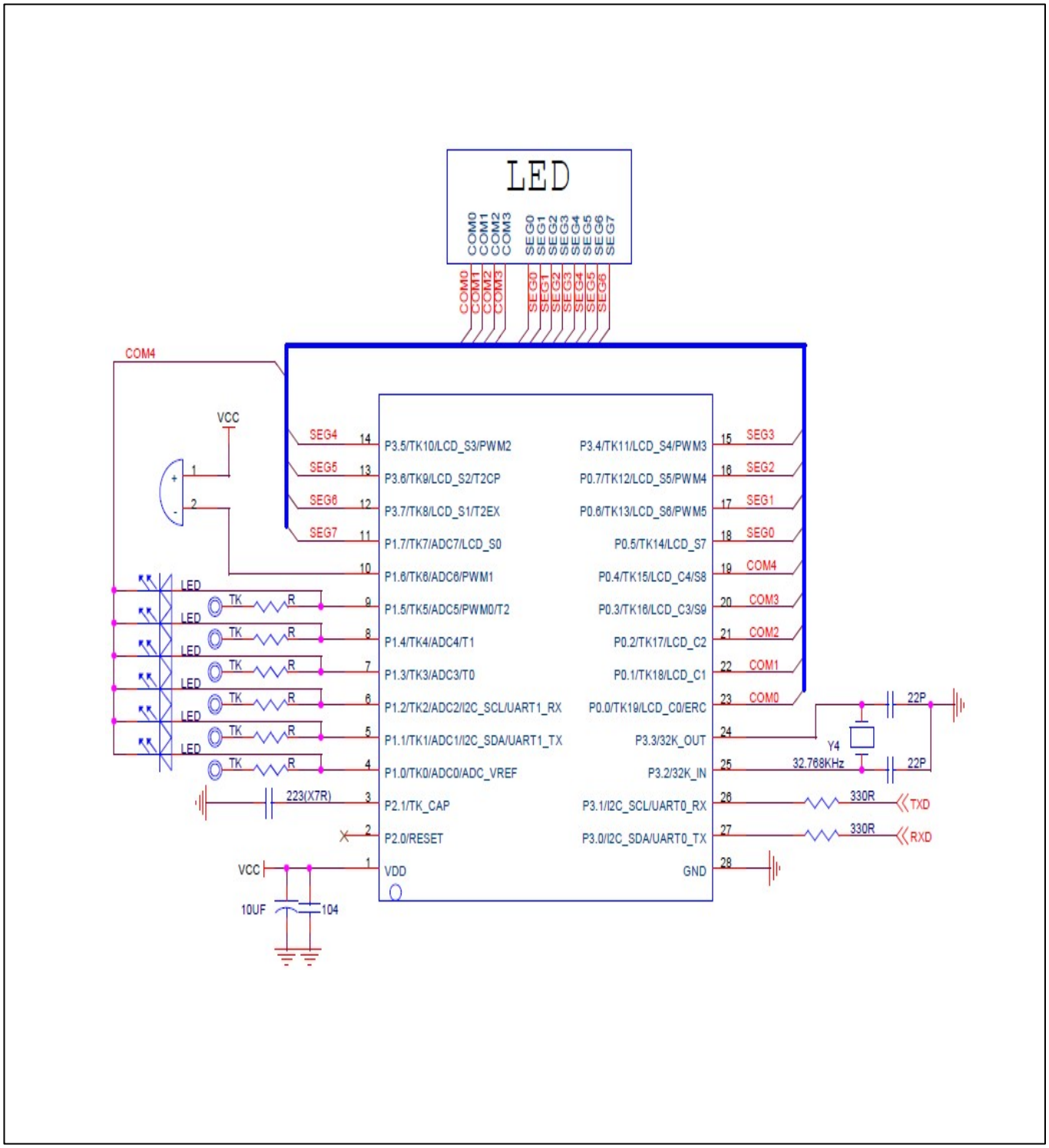
参考电路（一）



参考电路（二）


LCD
COM0
COM1
COM2
COM3
SEG0
SEG1
SEG2
SEG3
SEG4
SEG5
SEG6
SEG7
SEG8
P3.5/TK10/LCD_S3/PWM2
P3.6/TK9/LCD_S2/T2CP
P3.7/TK8/LCD_S1/T2EX
P1.7/TK7/ADC7/LCD_S0
P1.6/TK6/ADC6/PWM1
P1.5/TK5/ADC5/PWM0/T2
P1.4/TK4/ADC4/T1
P1.3/TK3/ADC3/T0
P1.2/TK2/ADC2/I2C_SCL/UART1_RX
P1.1/TK1/ADC1/I2C_SDA/UART1_TX
P1.0/TK0/ADC0/ADC_VREF
P2.1/TK_CAP
P2.0/RESET
VDD
P3.4/TK11/LCD_S4/PWM3
P0.7/TK12/LCD_S5/PWM4
P0.6/TK13/LCD_S6/PWM5
P0.5/TK14/LCD_S7
P0.4/TK15/LCD_C4/S8
P0.3/TK16/LCD_C3/S9
P0.2/TK17/LCD_C2
P0.1/TK18/LCD_C1
P0.0/TK19/LCD_C0/ERC
P3.3/32K_OUT
P3.2/32K_IN
P3.1/I2C_SCL/UART0_RX
P3.0/I2C_SDA/UART0_TX
GND
VCC
10UF
104

参考电路（三）


LED
COM0
COM1
COM2
COM3
SEG0
SEG1
SEG2
SEG3
SEG4
SEG5
SEG6
SEG7
COM4
VCC
LED
TK
R
223(X7R)
10UF
104
P3.5/TK10/LCD_S3/PWM2
P3.6/TK9/LCD_S2/T2CP
P3.7/TK8/LCD_S1/T2EX
P1.7/TK7/ADC7/LCD_S0
P1.6/TK6/ADC6/PWM1
P1.5/TK5/ADC5/PWM0/T2
P1.4/TK4/ADC4/T1
P1.3/TK3/ADC3/T0
P1.2/TK2/ADC2/I2C_SCL/UART1_RX
P1.1/TK1/ADC1/I2C_SDA/UART1_TX
P1.0/TK0/ADC0/ADC_VREF
P2.1/TK_CAP
P2.0/RESET
VDD
P3.4/TK11/LCD_S4/PWM3
P0.7/TK12/LCD_S5/PWM4
P0.6/TK13/LCD_S6/PWM5
P0.5/TK14/LCD_S7
P0.4/TK15/LCD_C4/S8
P0.3/TK16/LCD_C3/S9
P0.2/TK17/LCD_C2
P0.1/TK18/LCD_C1
P0.0/TK19/LCD_C0/ERC
P3.3/32K_OUT
P3.2/32K_IN
P3.1/I2C_SCL/UART0_RX
P3.0/I2C_SDA/UART0_TX
GND
Y4
32.768KHz
22P
330R
TXD
RXD

27 附录

附录 **1** 指令集速查表

指令	描述	说明	周期
数据传送指令			
MOV A,Rn	寄存器内容送入累加器	(A) ← (Rn)	1
MOV A,direct	直接地址单元中的数据送入累加器	(A) ← (direct)	1
MOV A,@Ri	间接 RAM 中的数据送入累加器	(A) ← ((Ri))	1
MOV A,#data8	8 位立即数送入累加器	(A) ← #data	1
MOV Rn,A	累加器内容送入寄存器	(Rn) ← (A)	1
MOV Rn,direct	直接地址单元中的数据送入寄存器	(Rn) ← (direct)	2
MOV Rn,#data8	8 位立即数送入寄存器	(Rn) ← #data	1
MOV direct,A	累加器内容送入直接地址单元	(direct) ← (A)	1
MOV direct,Rn	寄存器内容送入直接地址单元	(direct) ← (Rn)	2
MOV direct,direct	直接地址单元中的数据送入直接地址单元	(direct) ← (direct)	2
MOV direct,@Ri	间接 RAM 中的数据送入直接地址单元	(direct) ← ((Ri))	2
MOV direct,#data8	8 位立即数送入直接地址单元	(direct) ← #data	2
MOV @Ri,A	累加器内容送入间接 RAM 单元	((Ri)) ← (A)	1
MOV @Ri,direct	直接地址单元中的数据送入间接 RAM 单元	((Ri)) ← (direct)	2
MOV @Ri,#data8	8 位立即数送入间接 RAM 单元	((Ri)) ← #data	1
MOV DPTR,#data16	16 位立即数地址送入地址寄存器	(DPTR) ← #data116	2
MOV A,@A+DPTR	以 DPTR 为基地址变址寻址单元中的数据送入累加器	(A) ← ((A)) + (DPTR)	2
MOV A,@A+PC	以 PC 为基地址变址寻址单元中的数据送入累加器	(PC) ← (PC) + 1 (A) ← ((A) + (PC))	2
MOVX A,@Ri	外部 RAM(8 位地址)送入累加器	(A) ← ((Ri))	2
MOVX A,@DPTR	外部 RAM(16 位地址)送入累加器	(A) ← ((DPTR))	2
MOVX @Ri,A	累加器送入外部 RAM(8 位地址)	((Ri)) ← (A)	2
MOVX @DPTR,A	累加器送入外部 RAM(16 位地址)	(DPTR) ← (A)	2
PUSH direct	直接地址单元中的数据压入堆栈	(SP) ← (SP) + 1 ((SP)) ← (direct)	2
POP DIRECT	堆栈中的数据弹出到直接地址单元	(direct) ← ((SP)) (SP) ← (SP) - 1	2
XCH A,Rn	寄存器与累加器交换	(A) ↔ (Rn)	1
XCH A,direct	直接地址单元与累加器交换	(A) ↔ (direct)	1
XCH A,@Ri	间接 RAM 与累加器交换	(A) ↔ ((Ri))	1
XCHD A,@Ri	间接 RAM 与累加器进行低半字节交换	(A.3,...,A.0) ↔	1

		((Ri).3,...,(Ri).0)	
SWAP A	累加器半字节交换	(A.3,...,A.0) ↔ (A.7,...,A.4)	1
算术操作类指令			
ADD A, Rn	寄存器内容加到累加器	(A) ← (A) + (Rn)	1
ADD A, direct	直接地址单元加到累加器	(A) ← (A) + (direct)	1
ADD A, @Ri	间接 RAM 内容加到累加器	(A) ← (A) + ((Ri))	1
ADD A, #data8	8 位立即数加到累加器	(A) ← (A) + #data	1
ADDC A, Rn	寄存器内容带进位加到累加器	(A) ← (A) + (C) + (Rn)	1
ADDC A, direct	直接地址单元带进位加到累加器	(A) ← (A) + (C) + (direct)	1
ADDC A, @Ri	间接 RAM 内容带进位加到累加器	(A) ← (A) + (C) + ((Ri))	1
ADDC A, #data8	8 位立即数带进位加到累加器	(A) ← (A) + (C) + #data	1
SUBB A, Rn	累加器带借位减寄存器内容	(A) ← (A) - (C) - (Rn)	1
SUBB A, direct	累加器带借位减直接地址单元	(A) ← (A) - (C) - (direct)	1
SUBB A, @Ri	累加器带借位减间接 RAM 内容	(A) ← (A) - (C) - ((Ri))	1
SUBB A, #data8	累加器带借位减 8 位立即数	(A) ← (A) - (C) - #data	1
INC A	累加器加 1	(A) ← (A) + 1	1
INC Rn	寄存器加 1	(Rn) ← (Rn) + 1	1
INC direct	直接地址单元内容加 1	(direct) ← (direct) + 1	1
INC @Ri	间接 RAM 内容加 1	((Ri)) ← ((Ri)) + 1	1
INC DPTR	DPTR 加 1	(DPTR) ← (DPTR) + 1	2
DEC A	累加器减 1	(A) ← (A) – 1	1
DEC Rn	寄存器减 1	(Rn) ← (Rn) - 1	1
DEC direct	直接地址单元内容减 1	(direct) ← (direct) - 1	1
DEC @Ri	间接 RAM 内容减 1	((Ri)) ← ((Ri)) - 1	1
MUL AB	A 乘以 B	temp16 ← (A) X (B) (A)←(temp.7,temp.6,...,temp.0) (B)←(temp.15,temp.14,...,temp.8)	4
DIV AB	A 除以 B	QUO ← (A) /	4

		(B) .....REM (A) ← QUO (B) ← REM	
DA A	累加器进行十进制转换	IF (A.3,...,A.0) > 9 || AC = 1 THEN temp16 ← (A) + 0x06 (A) ← (temp.7,...,temp.0) IF (temp16) > 0xFF THEN CY ← 1 IF (A.7,...,A.4) > 9 || CY = 1 THEN temp16 ← (A) + 0x60 (A) ← (temp.7,...,temp.0) IF (temp16) > 0xFF THEN CY ← 1	1
逻辑操作类指令			
ANL A, Rn	累加器与寄存器相“与”	(A) ← (A) & (Rn)	1
ANL A, direct	累加器与直接地址单元相“与”	(A) ← (A) & (direct)	1
ANL A, @Ri	累加器与间接 RAM 内容相“与”	(A) ← (A) & ((Ri))	1
ANL A, #data8	累加器与 8 位立即数相“与”	(A) ← (A) & #data	1
ANL direct, A	直接地址单元与累加器相“与	(direct) ← (direct) & (A)	1
ANL direct, #data8	直接地址单元与 8 位立即数相“与”	(direct) ← (direct) & #data	2
ORL A, Rn	累加器与寄存器相“或”	(A) ← (A) | (Rn)	1
ORL A, direct	累加器与直接地址单元相“或”	(A) ← (A) | (direct)	1
ORL A, @Ri	累加器与间接 RAM 内容相“或”	(A) ← (A) | ((Ri))	1
ORL A, #data8	累加器与 8 位立即数相“或”	(A) ← (A) | #data	1
ORL direct, A	直接地址单元与累加器相“或”	(direct) ← (direct) | (A)	1

ORL direct, #data8	直接地址单元与 8 位立即数相“或”	(direct) ← (direct) | #data	2
XRL A, Rn	累加器与寄存器相“异或”	(A) ← (A) ^ (Rn)	1
XRL A, direct	累加器与直接地址单元相“异或”	(A) ← (A) ^ (direct)	1
XRL A, @Ri	累加器与间接 RAM 内容相“异或”	(A) ← (A) ^ ((Ri))	1
XRL A, #data8	累加器与 8 位立即数相“异或”	(A) ← (A) ^ #data	1
XRL direct, A	直接地址单元与累加器相“异或”	(direct) ← (direct) ^ (A)	1
XRL direct, #data8	直接地址单元与 8 位立即数相“异或”	(direct) ← (direct) ^ #data	2
CLR A	累加器清 0	(A) ← 0	1
CPL A	累加器求反	(A) ← /(A)	1
RL A	累加器循环左移	(A) ← (A.6,A.5,...,A.0,A.7)	1
RLC A	累加器带进位循环左移	C ← A.7 (A) ← (A.6,A.5,...,A.0,C)	1
RR A	累加器循环右移	(A) ← (A.0,A.7,...,A.2,A.1)	1
RRC A	累加器带进位循环右移	C ← A.0 (A) ← (C,A.7,...,A.2,A.1)	1
控制转移类指令			
ACALL addr11	绝对短调用子程序	(PC) ← (PC) + 2 (SP) ← (SP) + 1 ((SP)) ← (PC7-0) (SP) ← (SP) + 1 ((SP)) ← (PC15-8) (PC10-0) ← page address	2
LACLL addr16	长调用子程序	(PC) ← (PC) + 3 (SP) ← (SP) + 1 ((SP)) ← (PC7-0) ((SP)) ← (PC15-8) (PC) ←addr15-0	2
RET	子程序返回	(PC15-8) ← ((SP)) (SP) ← (SP) - 1 (PC7-0) ← ((SP)) (SP) ← (SP) - 1	2
RETI	中断返回	(PC15-8) ← ((SP)) (SP) ← (SP) - 1 (PC7-0) ← ((SP))	2

		(SP) ← (SP) - 1	
AJMP addr11	绝对短转移	(PC) ← (PC) + 2 (PC10-0) ← page address	2
LJMP addr16	长转移	(PC) ← (PC) + 3 (SP) ← (SP) + 1 ((SP)) ← (PC7-0) (SP) ← (SP) + 1 ((SP)) ← (PC15-8) (PC10-0) ←addr15-0	2
SJMP rel	相对转移	(PC) ← (PC) + 2 (PC) ← (PC) + rel	2
JMP @A+DPTR	相对于 DPTR 的间接转移	(PC) ← (A) + (DPTR)	2
JZ rel	累加器为零转移	(PC) ← (PC) + 2 IF (A) = 0 THEN (PC) ← (PC) + rel	2
JNZ rel	累加器非零转移	(PC) ← (PC) + 2 IF (A) <> 0 THEN (PC) ← (PC) + rel	2
CJNE A, direct, rel	累加器与直接地址单元比较，不等则转移	(PC) ← (PC) + 3 IF (A) <> (direct) THEN (PC) ← (PC) + relative offset IF (A) < (direct) THEN (C) ← 1 ELSE (C) ← 0	2
CJNE A, #data8, rel	累加器与 8 位立即数比较，不等则转移	(PC) ← (PC) + 3 IF (A) <> data THEN (PC) ← (PC) + relative offset IF (A) < data THEN (C) ← 1 ELSE (C) ← 0	2
CJNE Rn, #data8, rel	寄存器与 8 位立即数比较，不等则转移	(PC) ← (PC) + 3	2

		IF (Rn) <> data THEN (PC) ← (PC) + relative offset IF (Rn) < data THEN (C) ← 1 ELSE (C) ← 0	
CJNE @Ri, #data8, rel	间接 RAM 单元，不等则转移	(PC) ← (PC) + 3 IF ((Ri)) <> data THEN (PC) ← (PC) + relative offset IF ((Ri)) < data THEN (C) ← 1 ELSE (C) ← 0	2
DJNZ Rn, rel	寄存器减 1，非零转移	(PC) ← (PC) + 2 (Rn) ← (Rn) - 1 IF (Rn) <> 0 THEN (PC) ← (PC) + rel	2
DJNZ direct, rel	直接地址单元减 1，非零转移	(PC) ← (PC) + 2 (direct) ← (direct) - 1 IF (direct) <> 0 THEN (PC) ← (PC) + rel	2
NOP	空操作	(PC) ← (PC) + 1	1
布尔变量操作类指令			
CLR C	清进位位	(C) ← 0	1
CLR bit	清直接地址位	(bit) ← 0	1
SETB C	置进位位	(C) ← 1	1
SETB bit	置直接地址位	(bit) ← 1	1
CPL C	进位位求反	(C) ← /(C)	1
CPL bit	直接地址位求反	(bit) ← /(bit)	1
ANL C, bit	进位位和直接地址位相“与”	(C) ← (C) & (bit)	2
ANL C, /bit	进位位和直接地址位的反码相“与”	(C) ← (C) & /(bit)	2
ORL C, bit	进位位和直接地址位相“或”	(C) ← (C) | (bit)	2
ORL C, /bit	进位位和直接地址位的反码相“或”	(C) ← (C) | /(bit)	2
MOV C, bit	直接地址位送入进位位	(C) ← (bit)	1
MOV bit, C	进位位送入直接地址位	(bit) ← (C)	2

JC rel	进位位为 1 则转移(CY=O 不转移，=1 转移)	(PC) ← (PC) + 2 IF (C) = 1 THEN (PC) ← (PC) + rel	2
JNC rel	进位位为 0 则转移	(PC) ← (PC) + 2 IF (C) = 0 THEN (PC) ← (PC) + rel	2
JB bit, rel	直接地址位为 1 则转移	(PC) ← (PC) + 3 IF (bit) = 1 THEN (PC) ← (PC) + rel	2
JNB bit, rel	直接地址位为 0 则转移	(PC) ← (PC) + 3 IF (bit) = 0 THEN (PC) ← (PC) + rel	2
JBC bit, rel	直接地址位为 1 则转移，该位清零	(PC) ← (PC) + 3 IF (bit) = 1 THEN (bit) ← 0 (PC) ← (PC) + rel	2

伪指令	
ORG	设置程序起始地址
END	标志源代码结束
EQU	定义常数
SET	定义整型数
DATA	给数据地址定值
BYTE	给字节类型符号定值
WROD	给字类型符号定值
BIT	给位地址取名
ALTNAME	用自定义名取代保留字
DB	给一块连续的存储区装载字节型数据
DW	给一块连续的存储区装载字型数据
DS	预留一个连续的存储区或装入指定字节
INCLUDE	将一个源文件插入程序中
TITLE	列表文件中加入标题行
NOLIST	汇编时不产生列表文件
NOCODE	条件汇编时，条件为假的不产生清单